

SECR2033
Computer Organization and Architecture

Lecture slides prepared by “Computer Organization and Architecture”, 9/e, by William Stallings, 2013.

Mid Term Test - COA



- Date: 21 May 2025 (Wednesday)
- Time: 8 pm – 10 pm
- Venue: to be informed later

- Coverage:
  - Chapters 1 - 4

Module 5
Central Processing Unit (CPU)

Objectives:

- To learn the components common to every CPU.
- Be able to explain how each component in CPU contributes to instruction cycle.
- To understand the concept of pipelining in CPU execution.
- Be able to understand micro-operations basis for the design and implementation of the control unit.

Module 5
Central Processing Unit (CPU)

5.1 Processor Organization
5.2 Register Organization
5.3 Instruction Cycles
5.4 Instruction Pipelining
5.5 Control Unit Operation
5.6 Microprogrammed Control
5.7 Summary

Module 5a
Central Processing Unit (CPU)

5.1 Processor Organization

- Overview

5.2 Register Organization

5.3 Instruction Cycles

5.4 Instruction Pipelining

5.5 Control Unit Operation

5.6 Microprogrammed Control

5.7 Summary

Overview

- Basic function of a computer is running a program (stored in RAM)
- The CPU:
  - fetches (OR *reads*) instructions from RAM,
  - decodes it,
  - and executes it.
- Thus, program execution is repeating the process of instruction fetch and execute

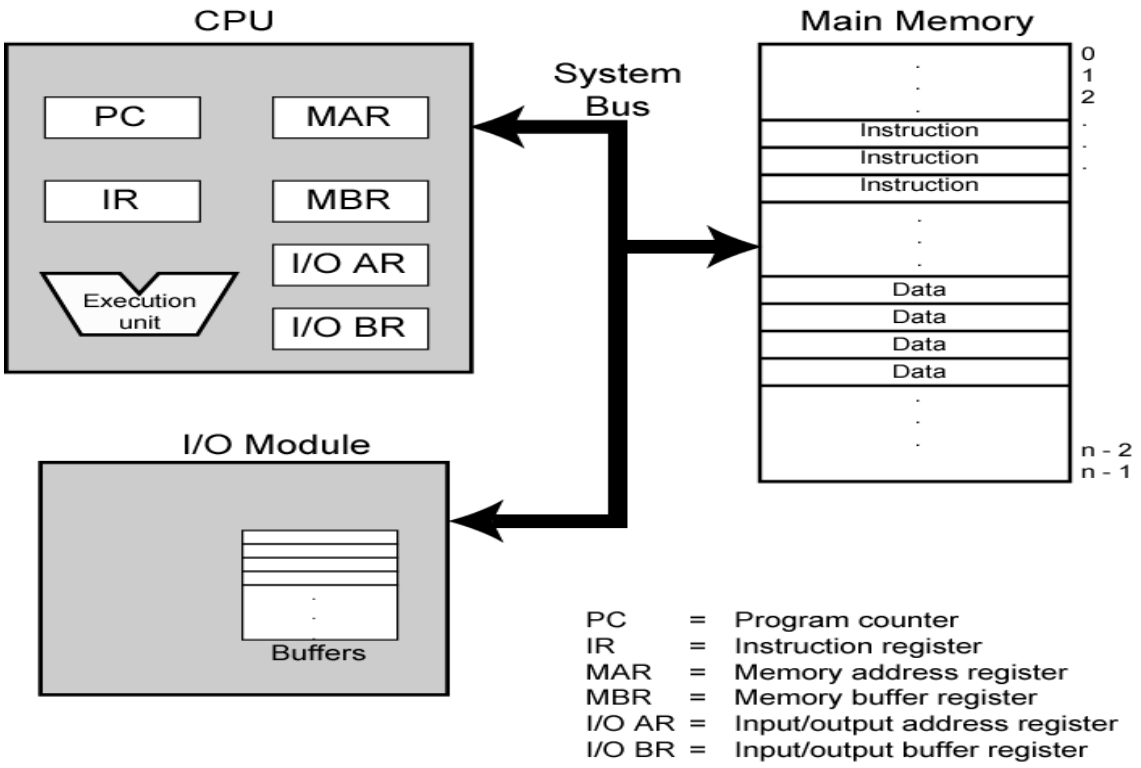


Figure 3.2 Computer Components: Top-Level View

- To understand the organization of the processor, let us consider the requirements placed on the processor, the things that it must do:
  - *Fetch instruction*: reads an instruction from memory (register, cache, main memory).
  - *Interpret instruction*: The instruction is decoded to determine what action is required.
  - *Fetch data*: The execution of an instruction may require reading data from memory or an I/O module.

5

- *Process data*: The execution of an instruction may require performing some arithmetic or logical operation on data.
- *Write data*: The results of an execution may require writing data to memory or an I/O module.

- To do these things, the processor needs a small internal memory to store some data and instruction temporarily while an instruction is being executed.

Minimal internal memory, consisting of a set of storage locations, called registers.

ALU does the actual computation or processing of data

Control unit controls the movement of data and instructions into / out of the processor and controls the operation of the ALU

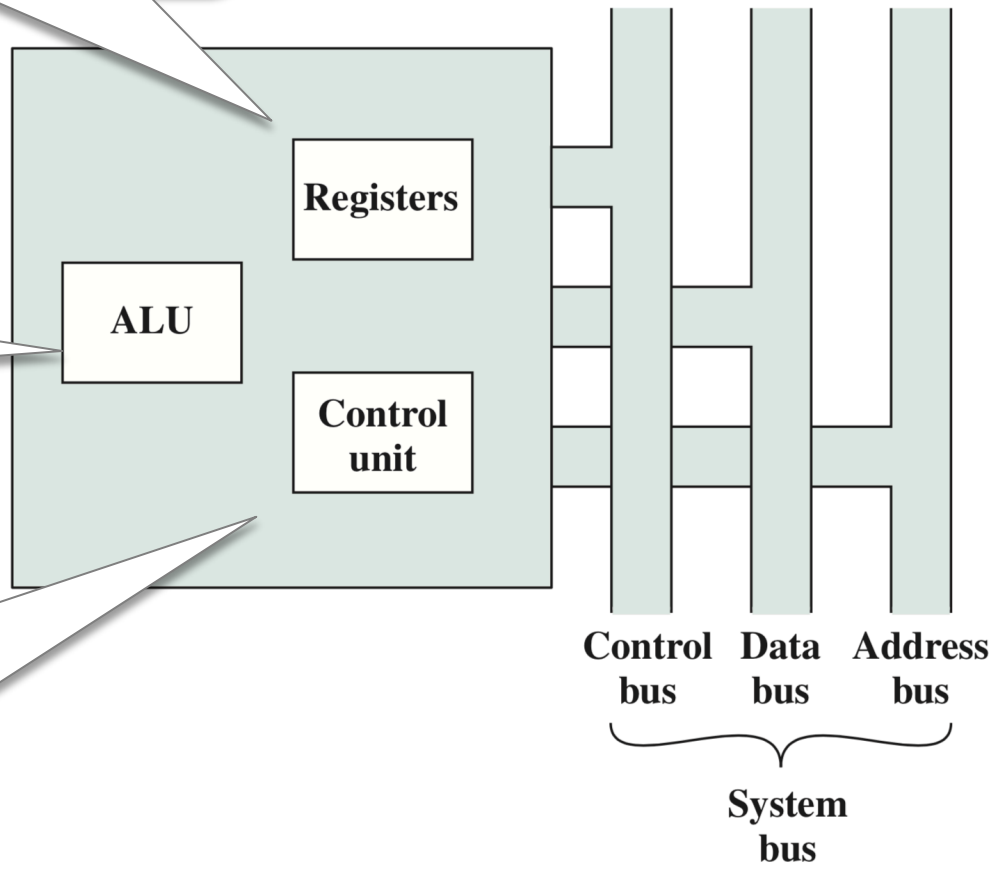


Figure: The CPU with the System Bus.

- The two principal parts of the CPU:
  - data path
  - control unit

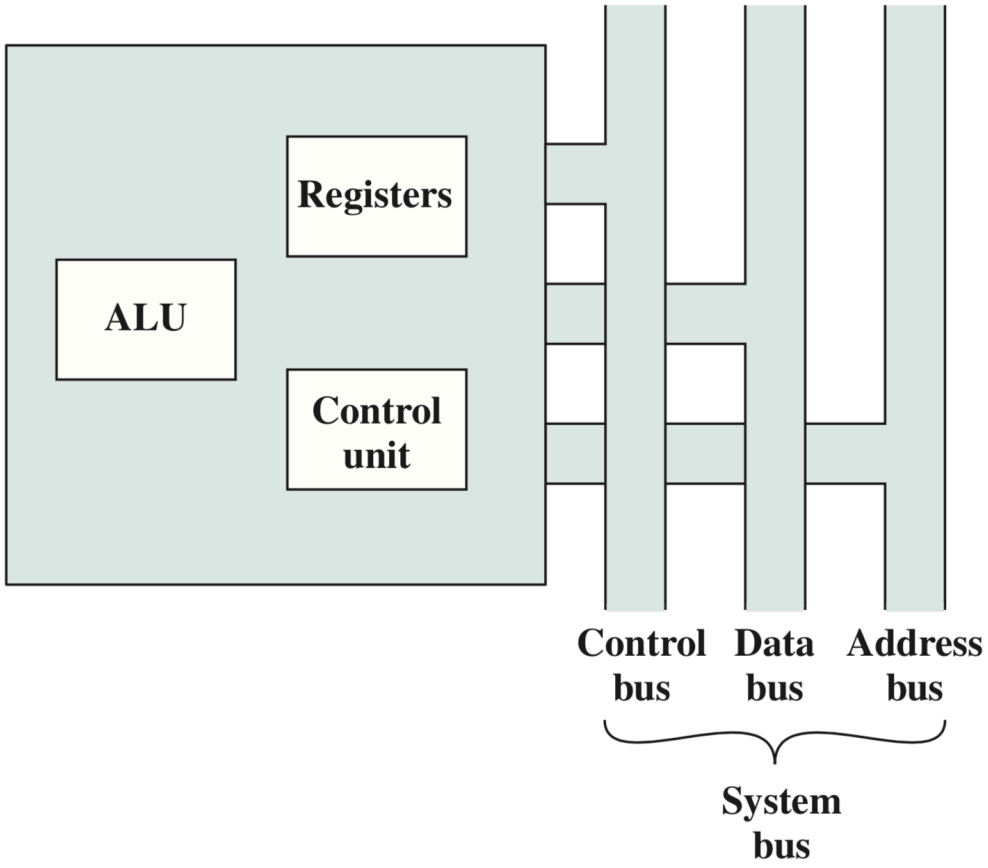


Figure: The CPU with the System Bus.

1) The **data path** consists of an *ALU* and storage units (*registers*) that are interconnected by a *data bus* that is also connected to *main memory*.

2) The **control unit** provides signals to tell the **data path**, *memory*, and *I/O* devices what to do according the instructions of the program.

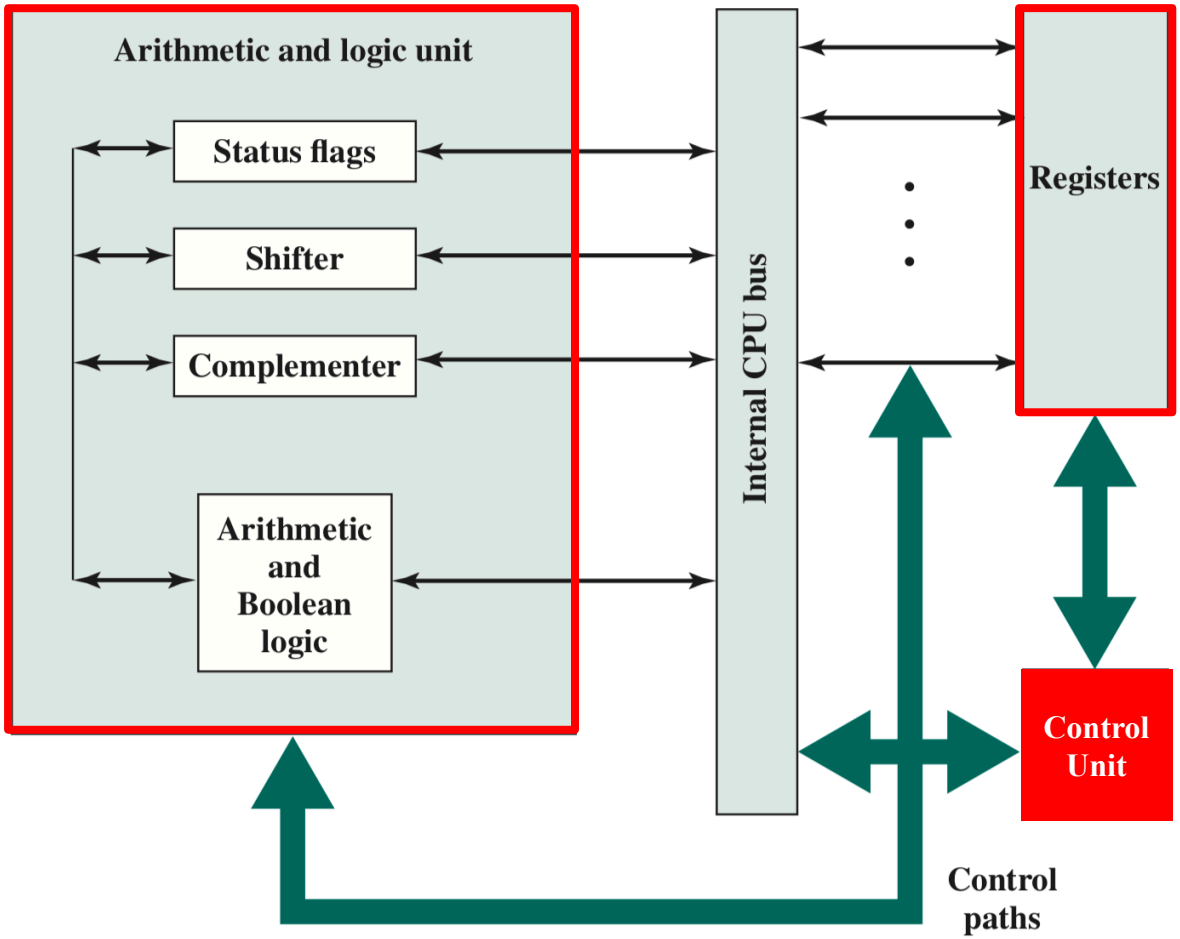
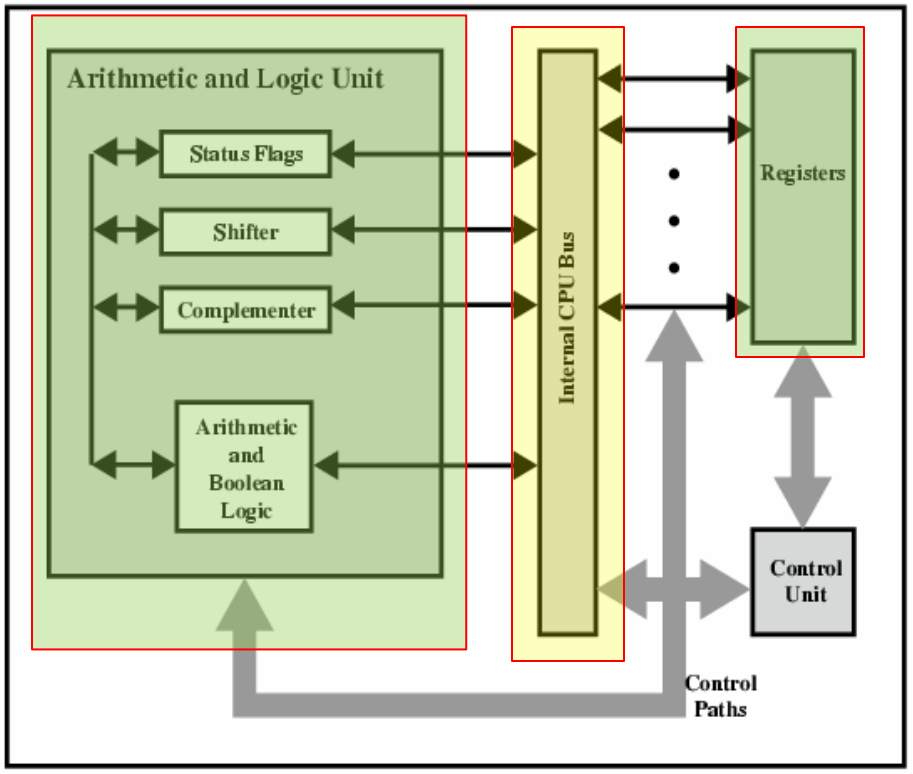


Figure: Internal Structure of the CPU.

Internal CPU Bus

- The figure shows typical basic elements of the ALU.
- *Internal CPU bus* is needed to transfer data between the various registers and the ALU
  - the ALU operates only on data in the registers



- The internal CPU bus permits data transfer between these components without the use of the main system bus.

Extra

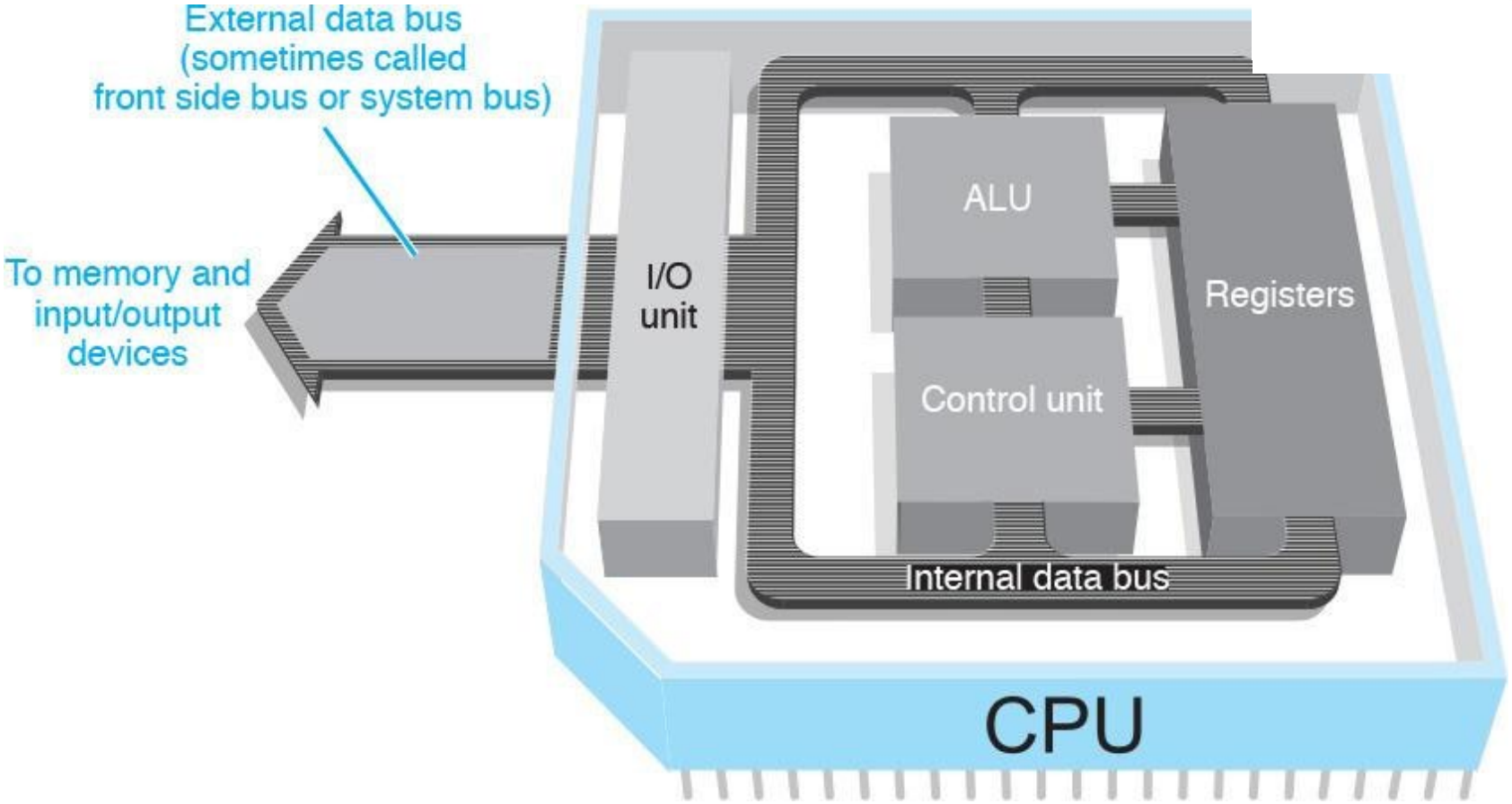
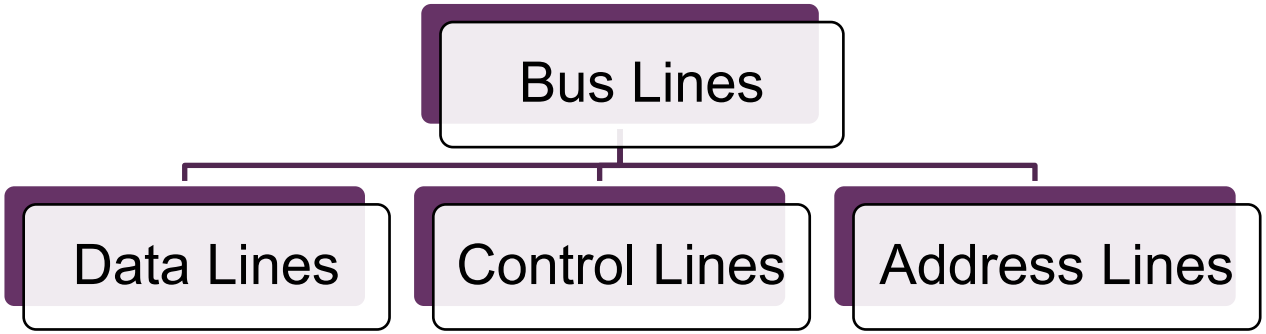


Figure: Internal and external data buses

External Bus

- A bus is a set of wires that simultaneously convey a single bit along each line.



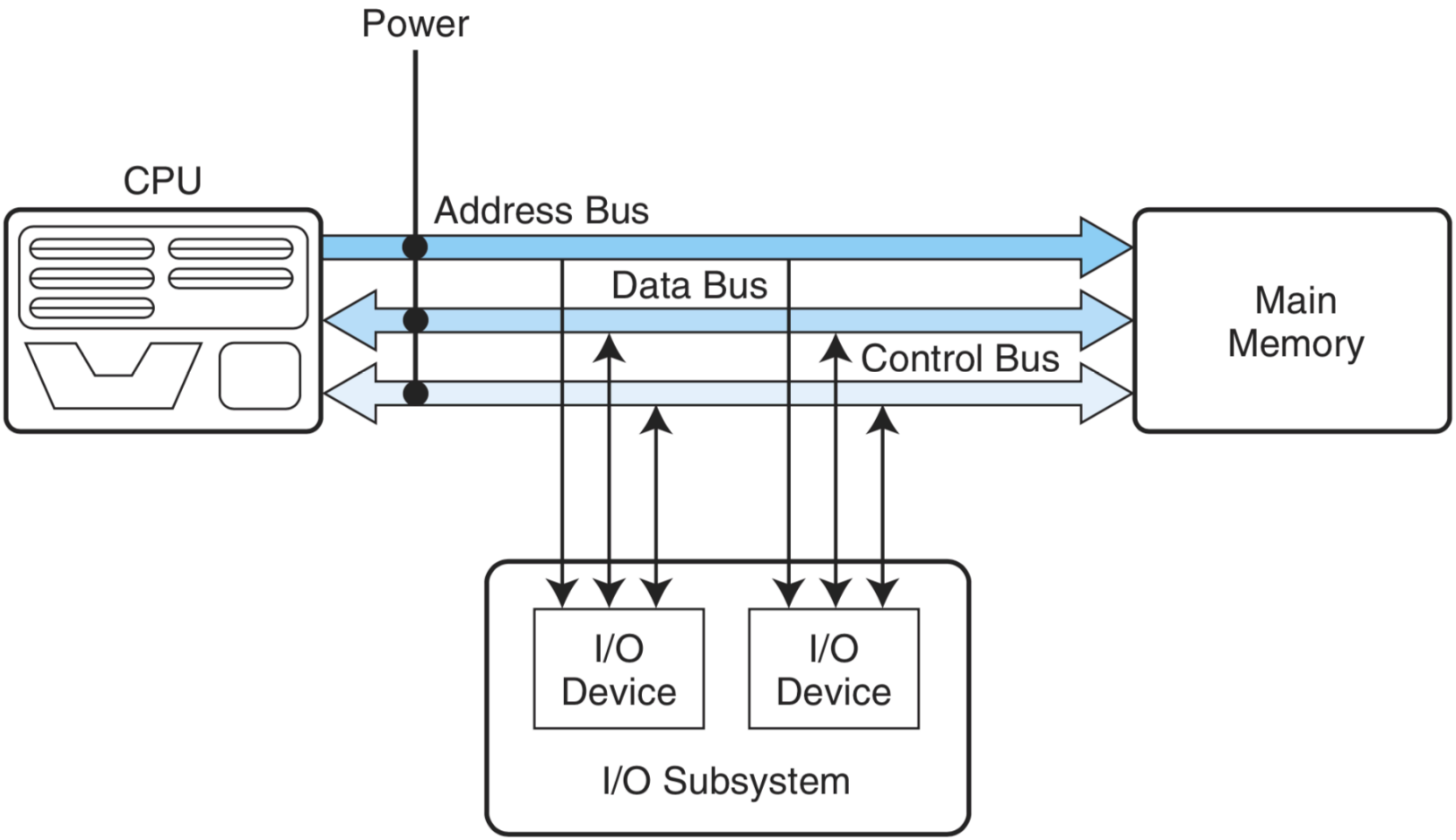


Figure: The Components of a Typical Bus

5

1) Data lines

- convey bits from one device to another.
- The number of lines (32, 64,128) gives the width of the data bus.
- **Example**: width = 32 bit, instruction length = 64 bits → this means the processor must access the memory modules twice during each instruction cycle.

2) Control lines

- determine the direction of data flow, and when each device can access the bus.
- Uses control signals, timing signals, command signals → memory write, memory read, bus request and bus grant.



3) Address lines

- determine the location of the source or destination of the data.

- **Example**: want to read data in memory, put address on this bus, go to memory, get the content (*i.e.* the data) and put it on to data bus.

Module 5a
Central Processing Unit (CPU)

5.1 Processor Organization

5.2 Register Organization

- Overview
- User-Visible Registers
- Control & Status Registers

5.3 Instruction Cycles

5.4 Instruction Pipelining

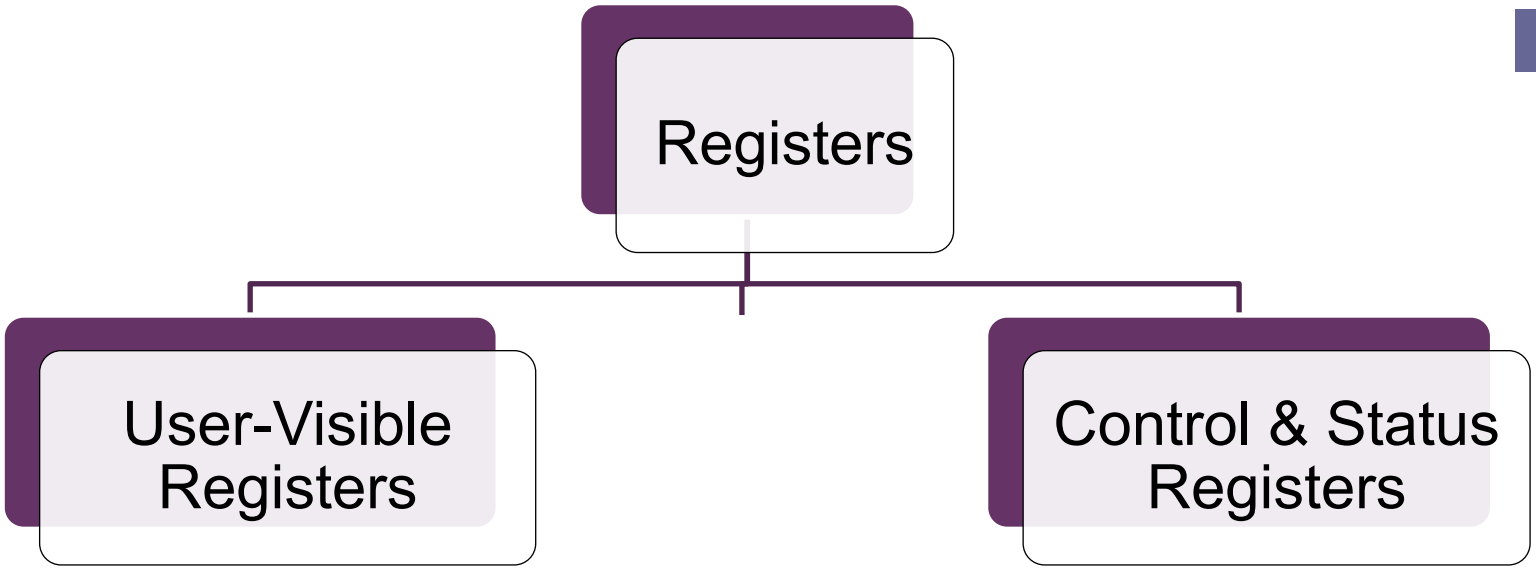
5.5 Control Unit Operation

5.6 Microprogrammed Control

5.7 Summary

5 Overview

- Registers form the highest level of the memory hierarchy.
  - Small set of high speed storage locations.
  - Temporary storage for data and control information.
- Registers hold data that can be readily accessed by the CPU.
- They can be implemented using D flip-flops.
  - A 32-bit register requires 32 D flip-flops.



(*General Purpose*)

May be referenced by assembly-level instructions and are thus "visible" to the user.

(*Special Purpose*)

Used to control the operation of the CPU. Most are not visible to the user.

5

(a) User-Visible Registers

- This register will be referenced by means of the machine language that the processor executes.

- Categories:
  - General purpose
  - Data
  - Address
  - Condition codes

General purpose registers:

- Can be assigned to a variety of functions.
- Defined to the operations within the instructions.
- Can be used for addressing functions.
- **Examples**: accumulator, base, count, data.

Data registers:

- Hold data and cannot be used in the calculation of an operand address.
- **Example**: accumulator.



Address registers:

- Hold address information.
- **Examples**: general purpose address registers, segment pointers, stack pointers, index registers.

Condition codes or Flags:

- Bits set by the processor hardware as a result of operations.
- Can be accessed by a program but not changed directly.
- **Examples**: *Sign Flag* (SF), *Zero Flag* (ZF), *Overflow Flag* (OF).
- Bit values are used as the basis for conditional jump instructions.

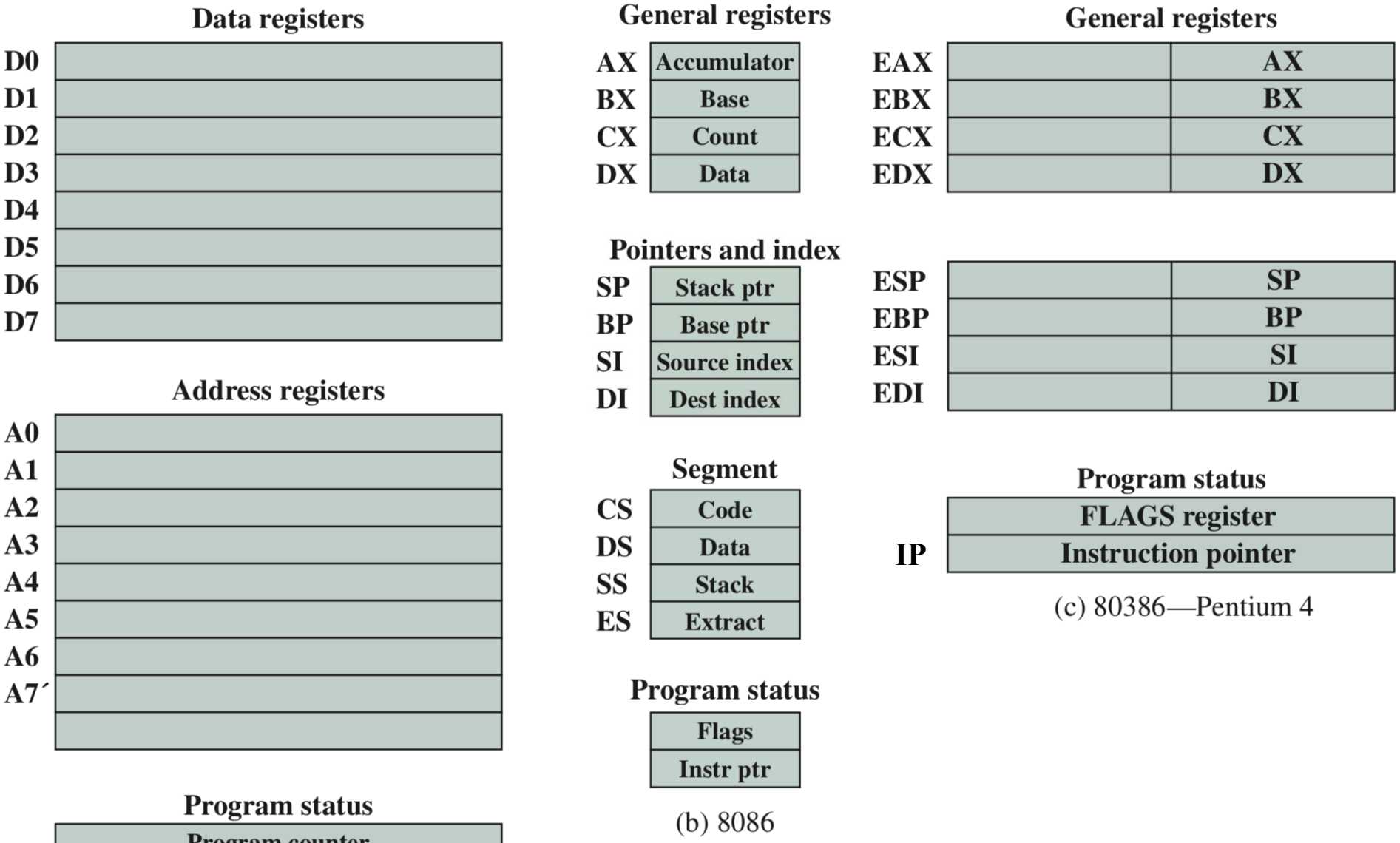
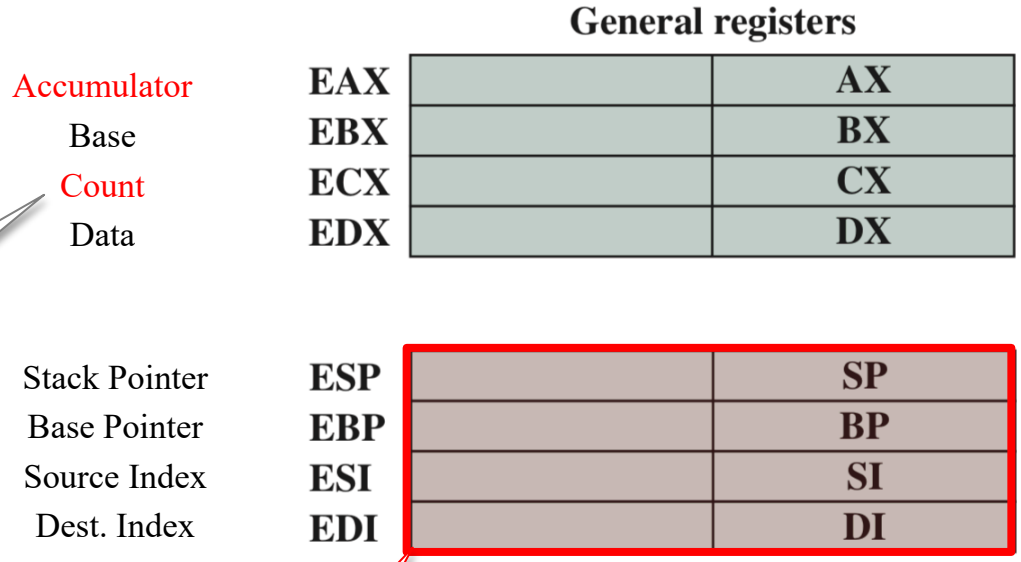


Figure: Example Microprocessor Register Organizations.

EAX *automatically used by* `MUL` *and* `DIV` *instructions*

ECX *automatically used by processor as a* `LOOP` *counter.*

Avoid usage for generic calculation !





- Address registers should be wide enough to hold the longest address!
- Data registers should be wide enough to hold most data types.
- Would not want to use 64-bit registers if the vast majority of data operations used 16 and 32-bit operands.
- Related to width of memory data bus.

- **Solution**: Concatenate registers together to store longer formats (DX:AX)

Example:

```
mul   ax
```

Assuming the following *DumpRegs*. Update the related register once the instruction executed:

```
EAX=770C0400  EBX=7FFD7098  ECX=00000000  EDX=00401005
ESI=00000000  EDI=00000000  EBP=0012FF94  ESP=0012FF8C
EIP=00401025  EFL=00000A82  CF=0  SF=1  ZF=0  OF=1
```

Solutions:

```
mul   ax
```

→ ax = 0400h

→ ax * ax = 0400 * 0400 = 0010 0000h

DX : AX

```
EAX=770C0000  EBX=7FFD7098  ECX=00000000  EDX=00400010
ESI=00000000  EDI=00000000  EBP=0012FF94  ESP=0012FF8C
EIP=00401025  EFL=00000A82  CF=0  SF=1  ZF=0  OF=1
```

Example:

```
div   bx
```

Assuming the following *DumpRegs*. Update the related register once the instruction executed:

```
EAX=770C0000  EBX=7FFD0020  ECX=00000000  EDX=00400010
ESI=00000000  EDI=00000000  EBP=0012FF94  ESP=0012FF8C
EIP=00401025  EFL=00000A82  CF=0  SF=1  ZF=0  OF=1
```

Solutions:

```
div   bx
```

→ dx : ax = 00100000h, bx = 0020h

→ [dx : ax] / bx = 100000 / 0020 = 0000 8000h

DX : AX

```
EAX=770C8000  EBX=7FFD0020  ECX=00000000  EDX=00400000
ESI=00000000  EDI=00000000  EBP=0012FF94  ESP=0012FF8C
EIP=00401025  EFL=00000A82  CF=0  SF=1  ZF=0  OF=1
```

(b) Control & Status Registers

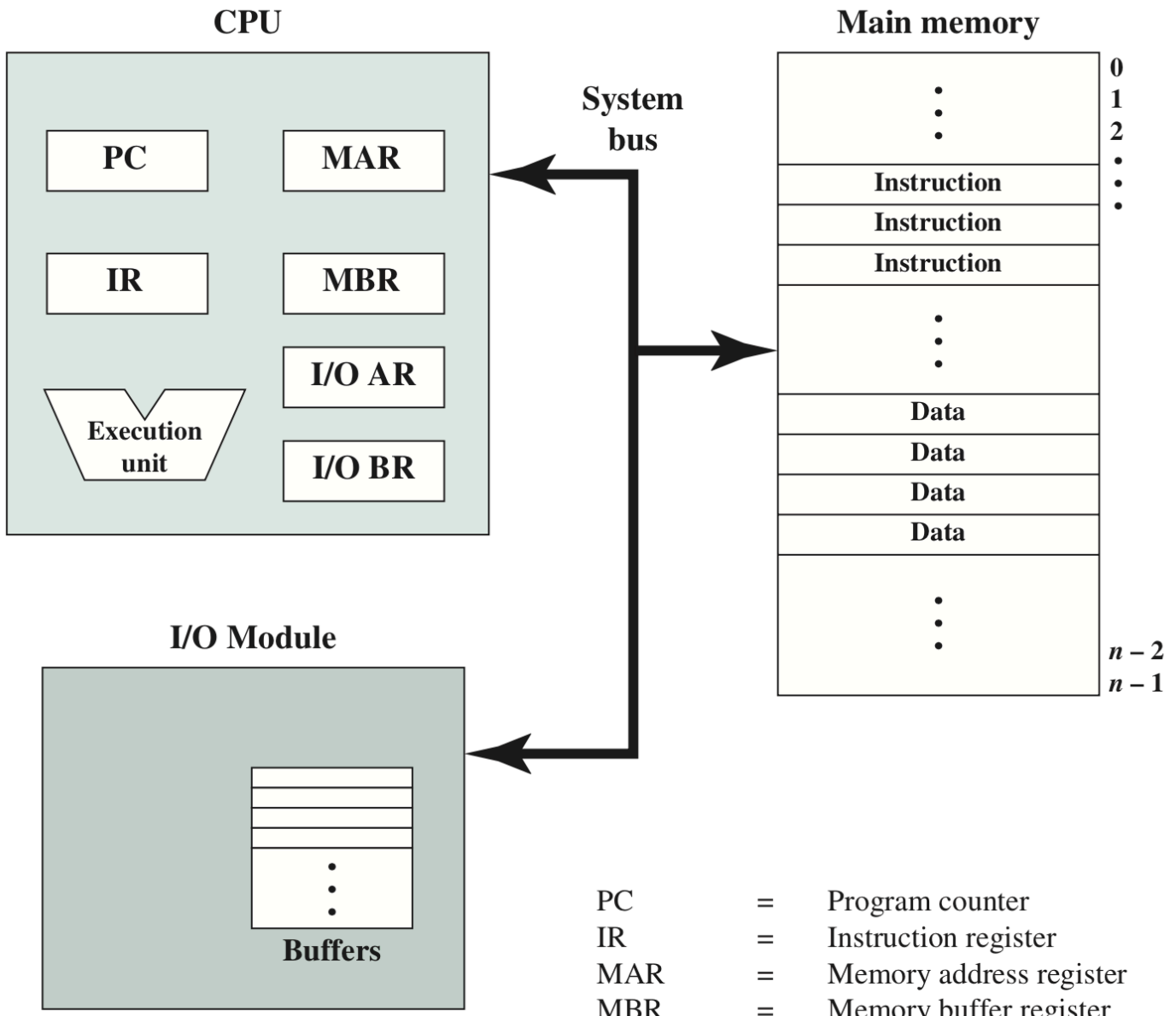
- There are a variety of processor registers that are employed to control the operation of the processor.
- Most of these, on most machines, are not visible to the user.
- Some of them may be visible to machine instructions executed in a control or operating system mode.

Different machines will have different register organizations and use different terminology.

Table: Four essential registers for instruction execution.

Registers	Function
Program Counter (PC)	Contains address of next instruction to be fetched.
Instruction Register (IR)	Contains the instruction being executed.
Memory Address Register (MAR)	Contains the address of the next memory access.
Memory Buffer Register (MBR)	Contains the current data fetched from memory.

Figure: Computer components: The CPU exchanges data with memory.



PC = Program counter
IR = Instruction register
MAR = Memory address register
MBR = Memory buffer register
I/O AR = Input/output address register
I/O BR = Input/output buffer register

Instruction Pointer

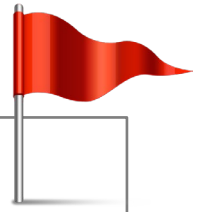


- The *Extended Instruction Pointer* (EIP) register holds the address of the next instruction to be executed.
- The EIP register corresponds to the *Program Counter* (PC) register in other architectures.
- EIP can be manipulated for certain instructions (*e.g.* call, `jmp`, `ret`) to branch to a new location.

Program Status

- Many processor designs include a register or set of registers, often known as the *Program Status Word* (PSW), that contain status information and condition codes.
- The ALU has a number of status flags that reflect the outcome of arithmetic (and bitwise) operations.

Status flags that reflect the outcome of arithmetic (contents of the destination operand)



- Common status flags:
  - *Zero flag* (ZF): set when destination (result) equals zero.
  - *Sign flag* (SF): set when destination (result) is negative.
  - *Parity flag* (PF): set when destination (result) has even number of 1's.
  - *Carry flag* (CF): set when unsigned value is out of range.
  - *Overflow flag* (OF): set when signed value is out of range.
  - *Auxiliary flag* (AF): set when there is carry from lower nibble to higher nibble in the lower byte.

Example 1: Flags

Assuming that the system uses 8-bit system.

→ 06h + 54h

ZF :	0
SF :	0
CF :	0
OF :	0
AF :	0
PF :	1

```
     0000 0110 (06h)
+    0101 0100 (54h)
   ----------------
     0101 1010 (5Ah)
```

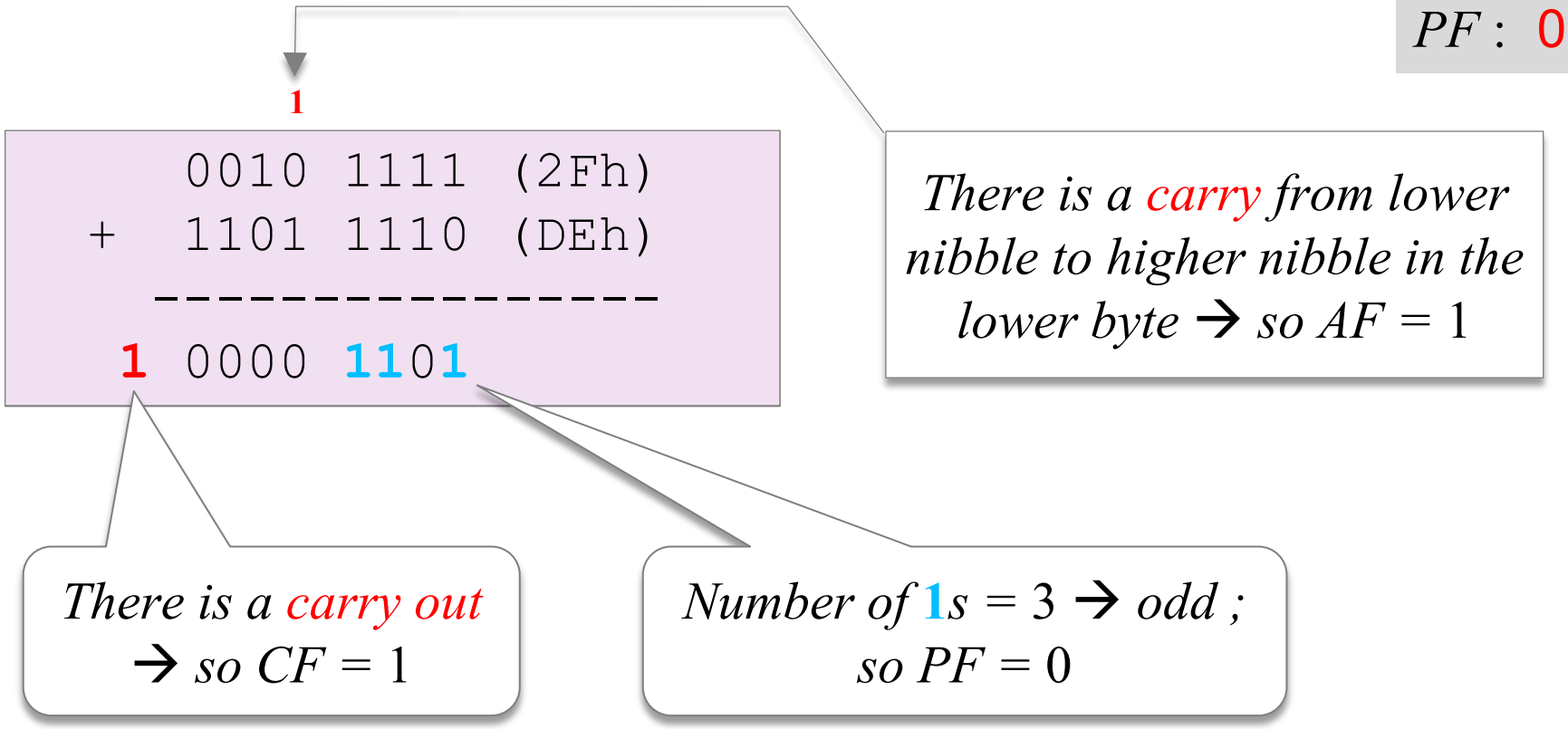
Operation result → *non zero* (0) *; so ZF* = 0

Number of **1***s* = 4 → *even ; so PF* = 1

Example 2: Flags

Assuming that the system uses 8-bit system.
→ 2Fh + DEh

ZF :	0
SF :	0
CF :	1
OF :	0
AF :	1
PF :	0

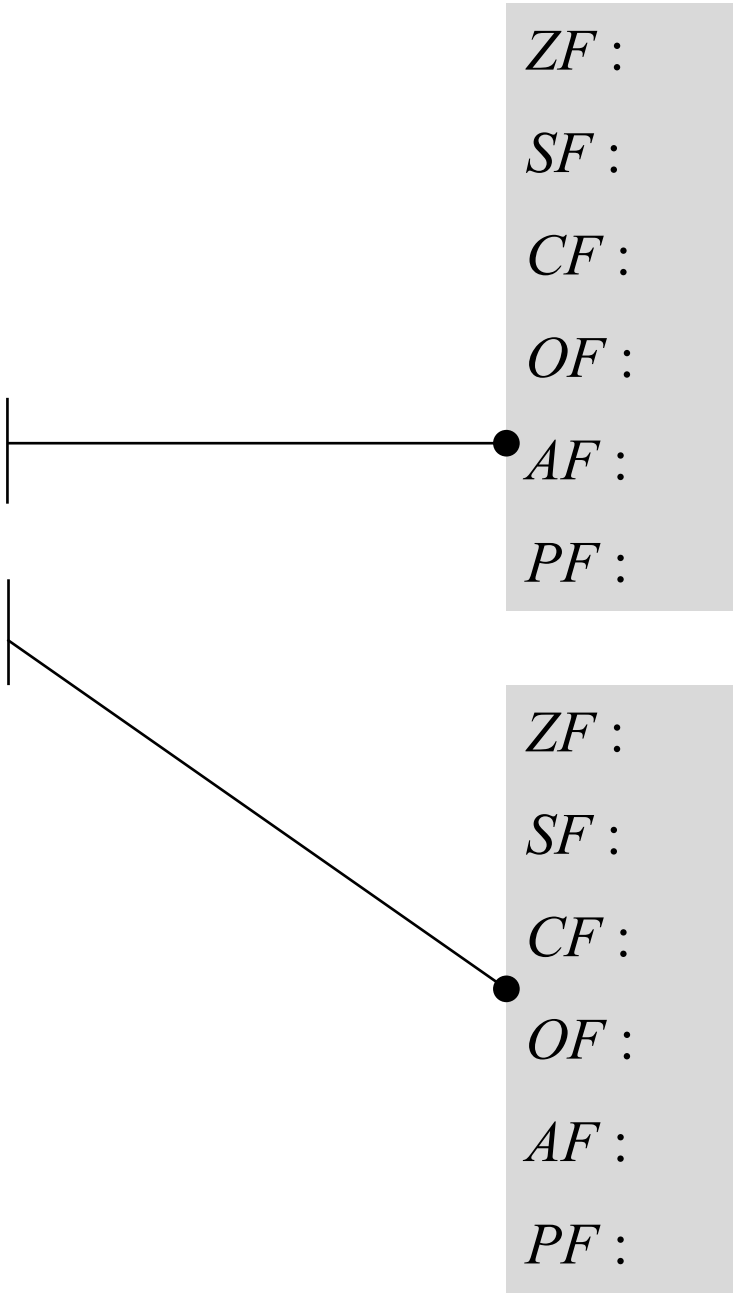


Example 3: Flags

a) Assuming that the system uses a word sized number for:

b) Assuming that the system uses a doubleword sized number for:

→ 5433h + 7098h



Solutions (a):

a) Assuming that the system uses a word sized number for:

$ZF: 0$

$SF: 1$

$CF: 0$

$OF: 1$

$AF: 0$

$PF: 1$

→ 5433h + 7098h

```
    0101 0100 0011 0011
+   0111 0000 1001 1000
  ---------------------
    1100 0100 1100 1011
```

Solution (b):

b) Assuming that the system uses a doubleword sized number for:

→ 00005433h + 00007098h

```
  0000 0000 0000 0000 0101 0100 0011 0011
+ 0000 0000 0000 0000 0111 0000 1001 1000
  ---------------------------------------
  0000 0000 0000 0000 1100 0100 1100 1011
```

ZF : 0

SF : 0

CF : 0

OF : 0

AF : 0

PF : 1

Example 4 : Flags

Assuming that the system uses **even parity** for:

a) 3Ch – 28h

b) 5Ah – 7Fh

a)
```
    0011 1100
  + 1101 1000
  -------------
  1 0001 0100  (14h)
```

2's for 28h

ZF : 0
SF : 0
CF : 1
OF : 0
AF : 1
PF : 1

b)
```
    0101 1010
  + 1000 0001
  -------------
    1101 1011  (DBh)
```

2's for 7Fh

ZF : 0
SF : 1
CF : 0
OF : 0
AF : 0
PF : 1

Module 5a
Central Processing Unit (CPU)

5.1 Processor Organization
5.2 Register Organization
5.3 Instruction Cycles
- Overview
- The Indirect Cycle
- Data Flow

5.4 Instruction Pipelining
5.5 Control Unit Operation
5.6 Microprogrammed Control
5.7 Summary

Overview: Instruction Cycle

- The *fetch-decode-execute cycle* is a series of steps that a computer carries out when it runs a program.
- **First**, CPU have to *fetch* an instruction from memory, and place it into the **IR**.

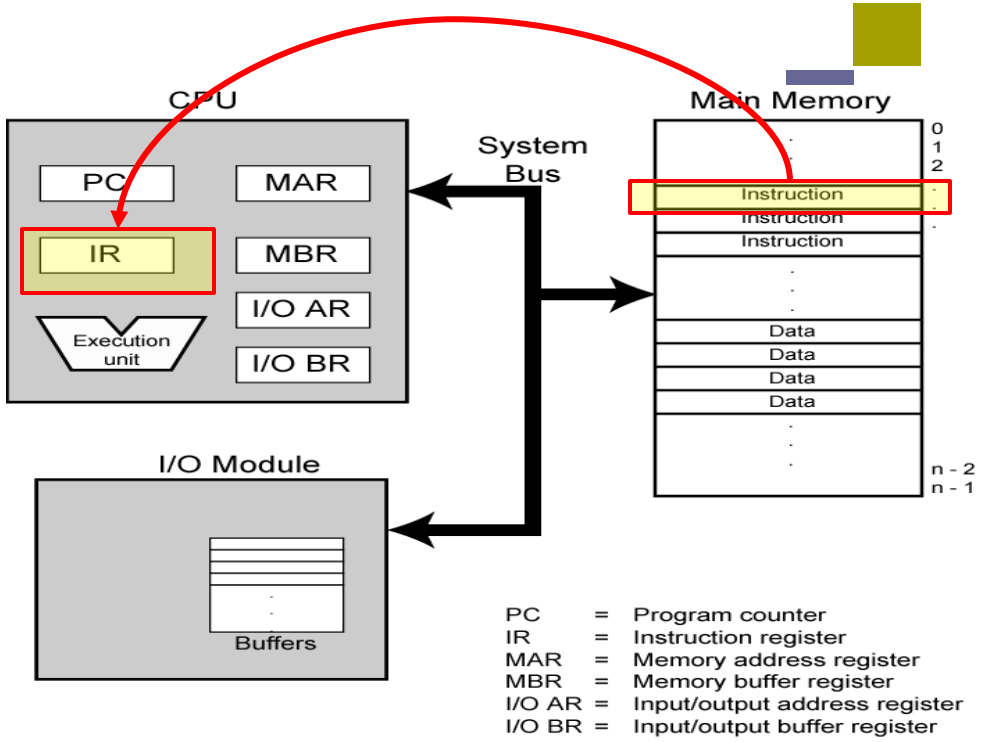


Figure 3.2 Computer Components: Top-Level View

- Once the instruction in the **IR**, it is *decoded* to determine what needs to be done next.
- If a memory value (operand) is involved in the operation, it is retrieved and placed into the **MBR**.
- With everything in place, the instruction is *executed*.

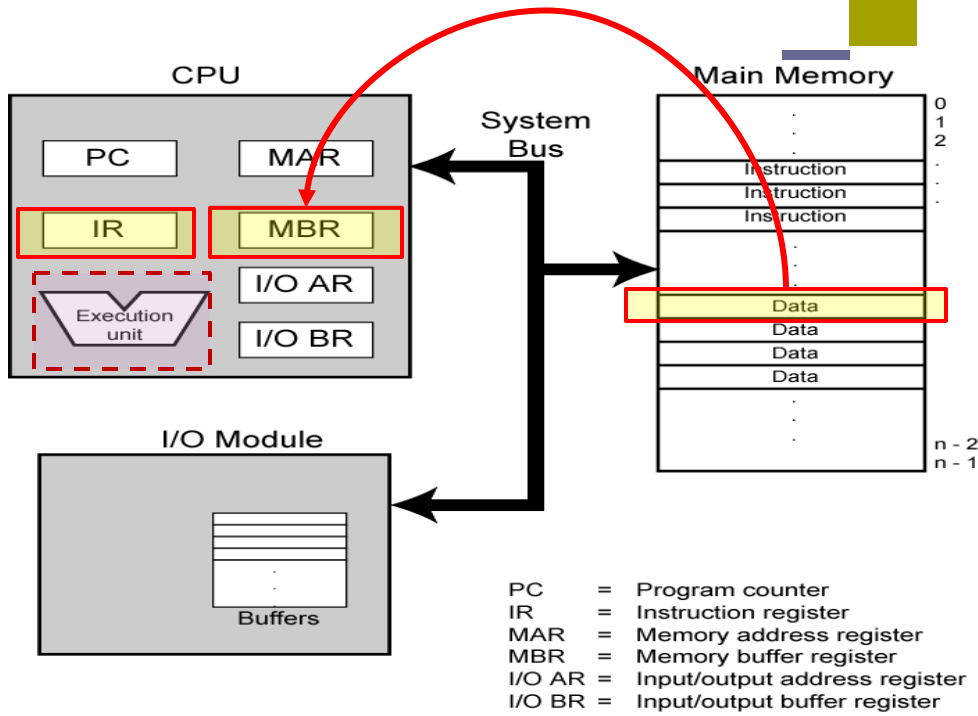
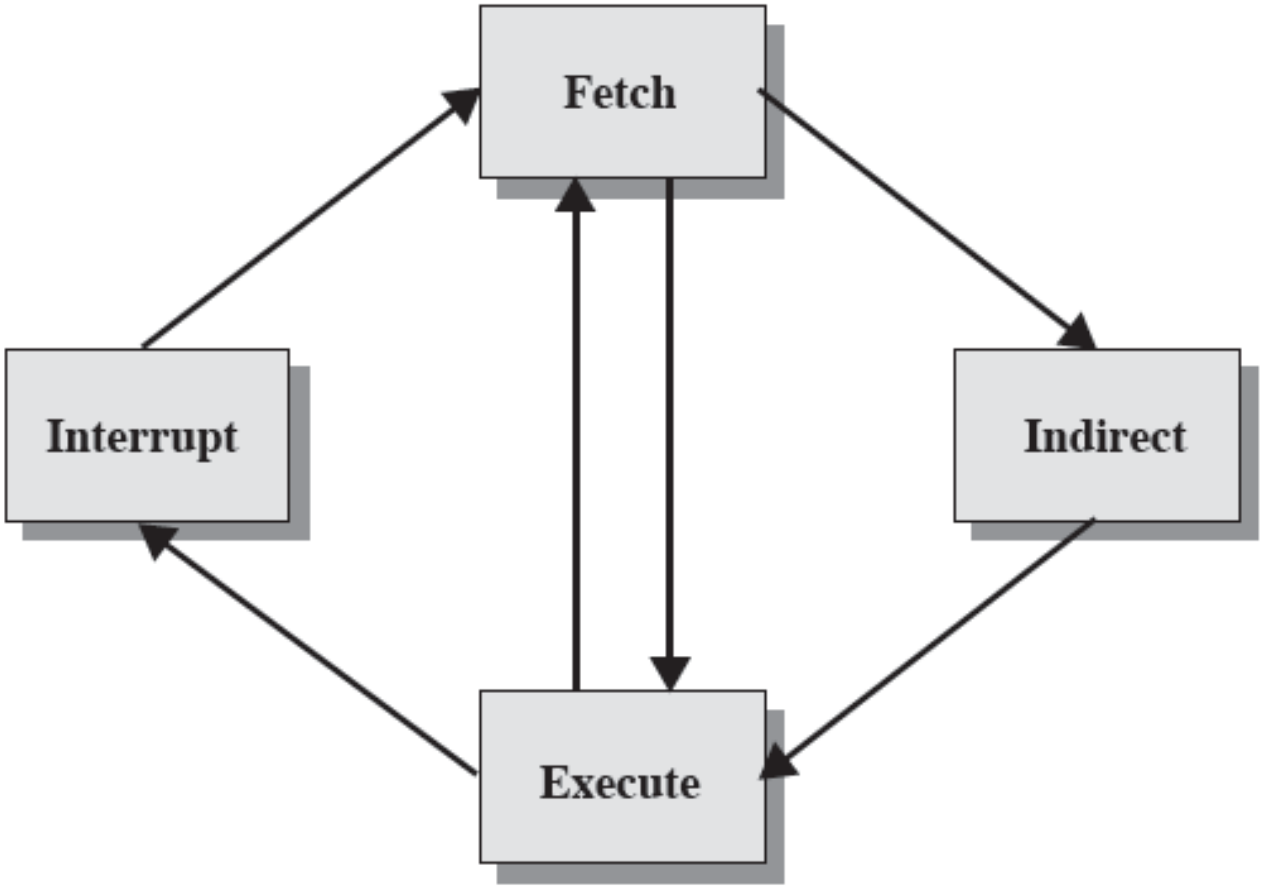


Figure 3.2 Computer Components: Top-Level View

Stages in Instruction Cycle



Stages in Instruction Cycle

- Fetch the instruction
- Decode it
- Fetch operands, if required.

Fetch

Indirect

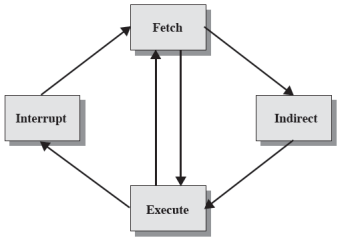
- Perform the operation
- Store results, if required

Execute

- Recognize pending interrupts

Interrupt

The Fetch-Decode-Execute Cycle



- represents the steps that a computer follows to run a program.
- This cycle can be illustrated in this flowchart

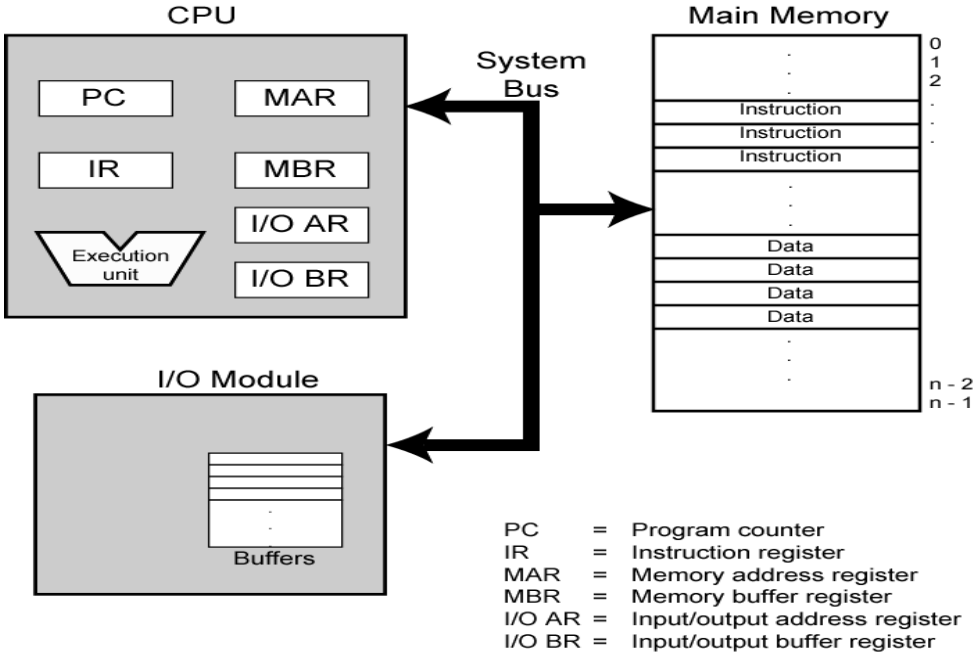
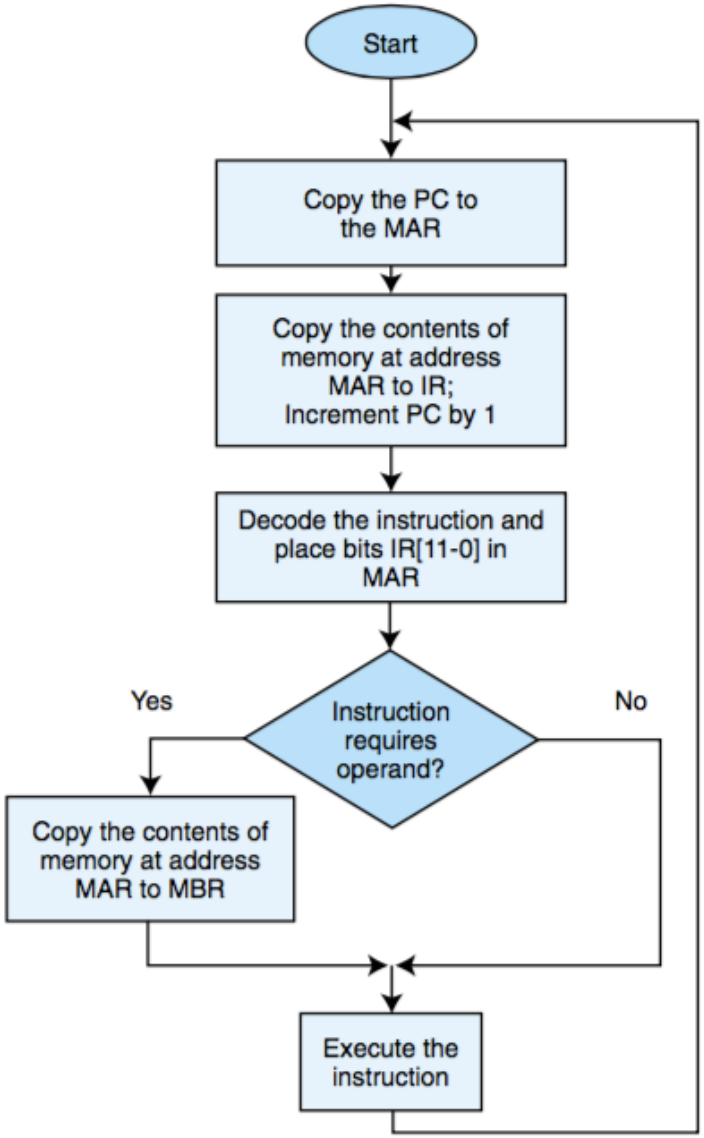
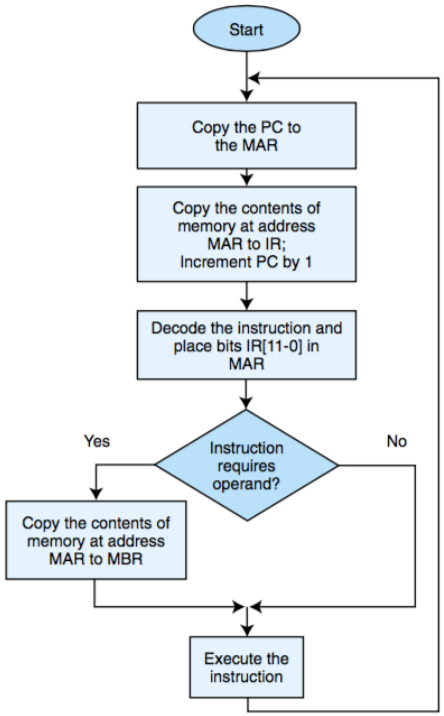


Figure 3.2 Computer Components: Top-Level View



- the flowchart is based on an imaginary machine called MARIE (Machine Architecture that is Really Intuitive and Easy)

Ref: Linda Null



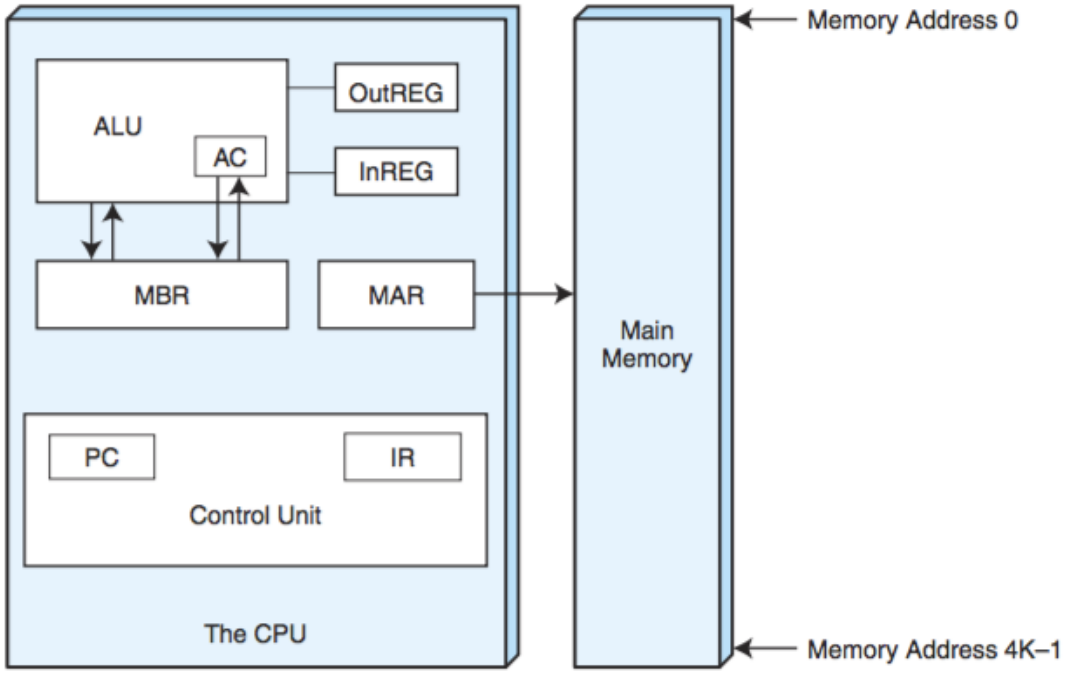


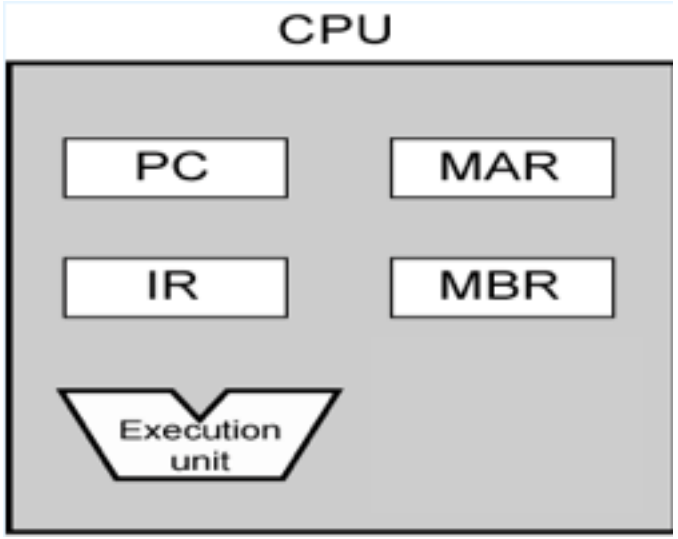
FIGURE 4.8 **MARIE's Architecture**



FIGURE 4.10 **MARIE's Instruction Format**

Data Flow Diagram (DFD)

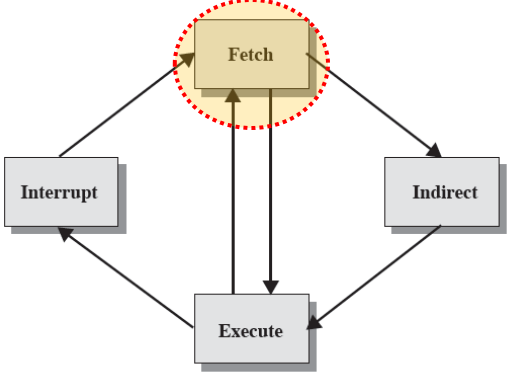
- DFD is used to illustrate the sequence of events during the execution of instruction cycle (i.e. *fetch-decode-execute* cycle)
- We assume the CPU has the following core components:
  - a memory address register (MAR)
  - a memory buffer register (MBR)
  - a program counter (PC), and
  - an instruction register (IR)
  - an execution unit (ALU)



Micro-Operations

- **Every step** of the *instruction cycle* consist of **micro-operation(s)** that are **executed at specified time**.
- Every micro-operations is activated and controlled based on the signals from the control unit.
- Example of micro-operations:
  - Transferring data from one register to another
  - Activating the functional unit of ALU
- Conventional format using **register transfer language (RTL)**
  - Example: IP $\leftarrow$ IP +1 or PC $\leftarrow$ PC +1

Data Flow, Fetch Cycle



- 4 registers are involved:
  - PC, MAR, MBR and IR
- The **fetch cycle** can be written symbolically using RTL as

t1:	MAR ← IP/PC
t2:	MBR ← [MAR];
	IP/PC ← IP/PC + 1
t3:	IR ← MBR

t1, t2, t3 is the timing sequence

Data Flow, Fetch Cycle

What happens:

t1: MAR ← IP/PC

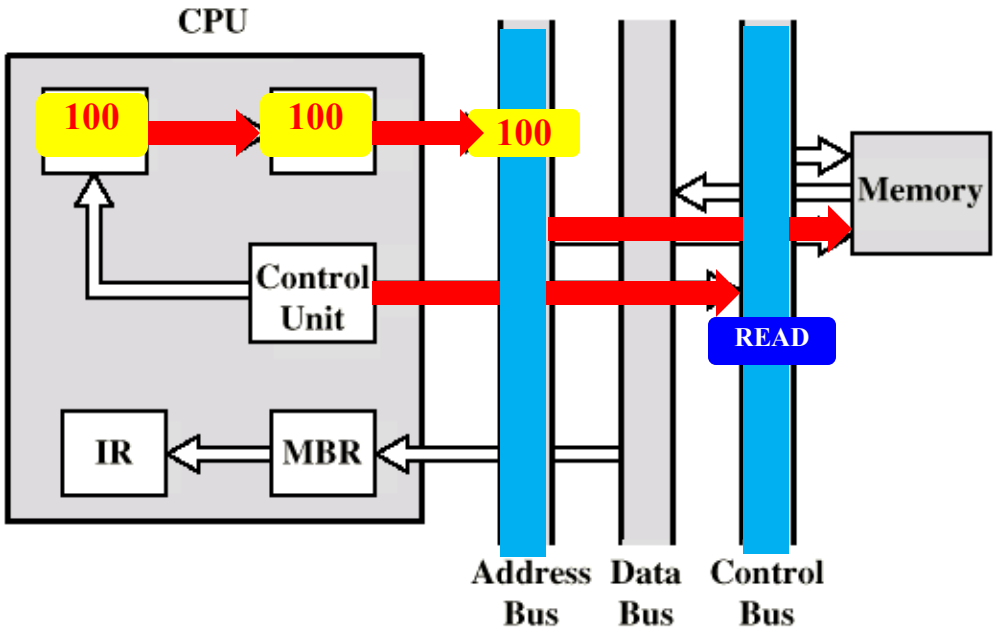
t2: MBR ← [MAR];
IP/PC ← IP/PC + 1

t3: IR ← MBR

Data Flow, Fetch Cycle

What happens:

t1: MAR ← IP/PC



MBR = Memory buffer register
MAR = Memory address register
IR = Instruction register
PC = Program counter

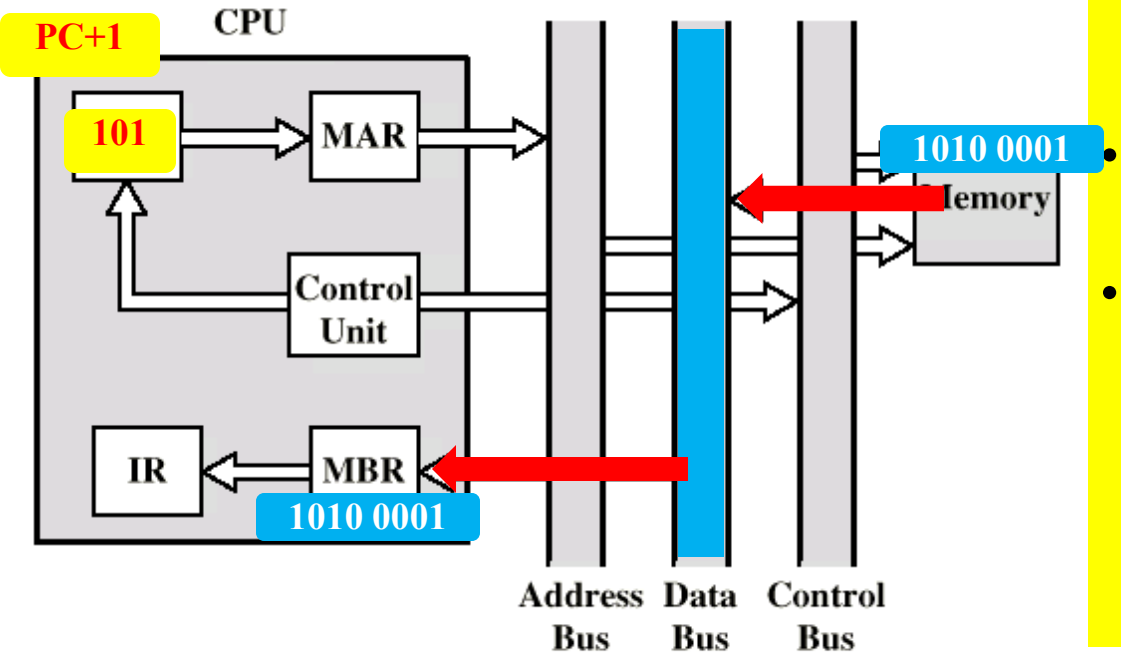
t1:

- Content of IP/PC is loaded into MAR
- Then transferred to memory on the address bus.
- Next, CU will activate the read signal of the memory.

Data Flow, Fetch Cycle

What happens:

t2: MBR ← [MAR];
IP/PC ← IP/PC + 1



MBR = Memory buffer register
MAR = Memory address register
IR = Instruction register
PC = Program counter

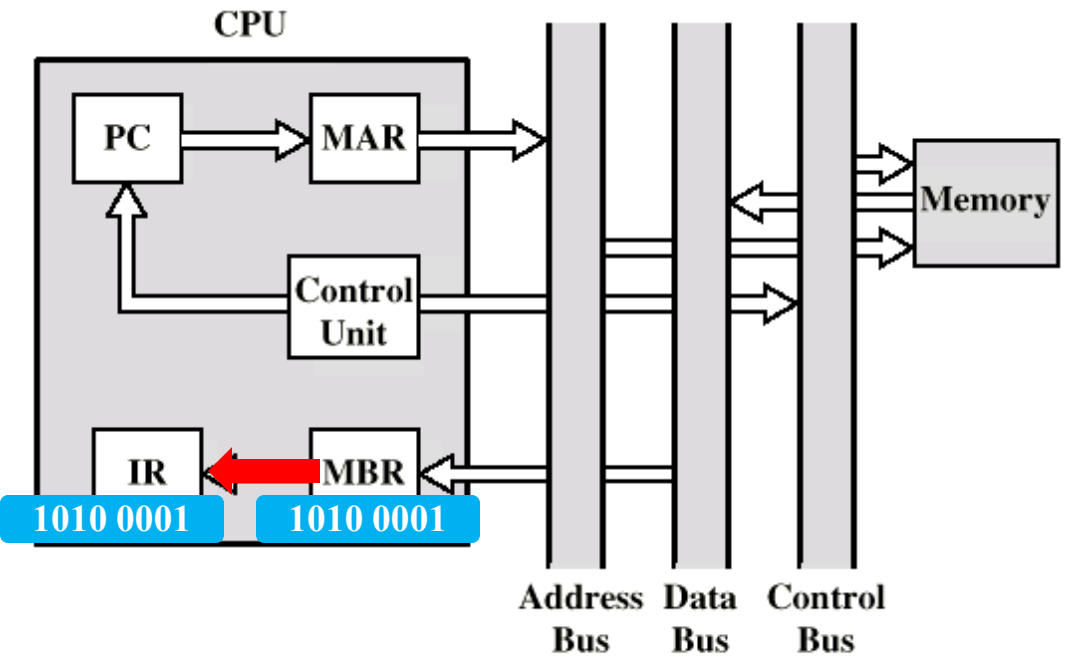
t2

- Instruction is accessed from the memory (@address 100) and transfer to CPU on the data bus.
- This instruction will be loaded into MBR.
- At the same time, IP/PC will be incremented (point to the next instruction)
  - i.e. PC = 100 + 1
    = 101

Data Flow, Fetch Cycle

What happens:

t3: IR ← MBR



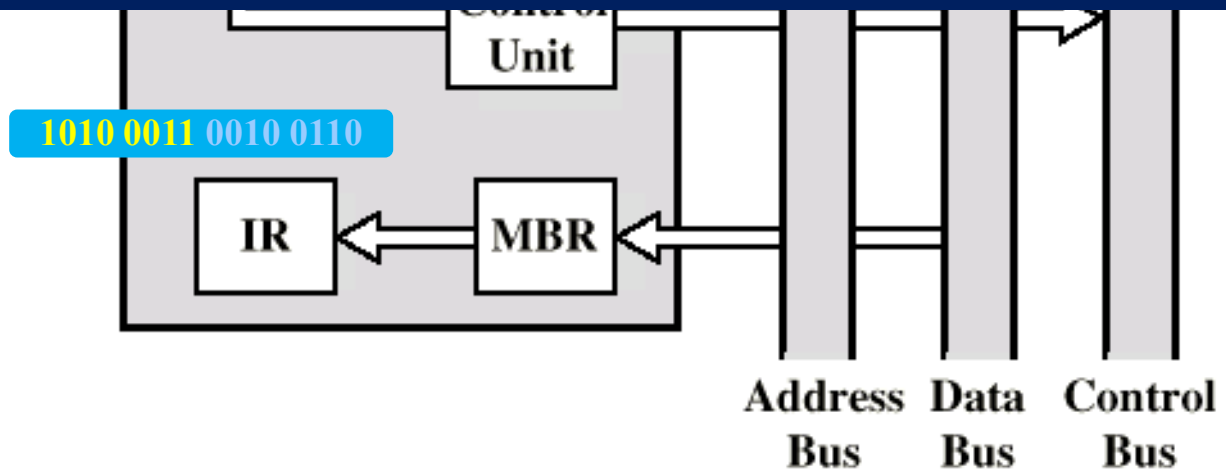
MBR = Memory buffer register
MAR = Memory address register
IR = Instruction register
PC = Program counter

t3:

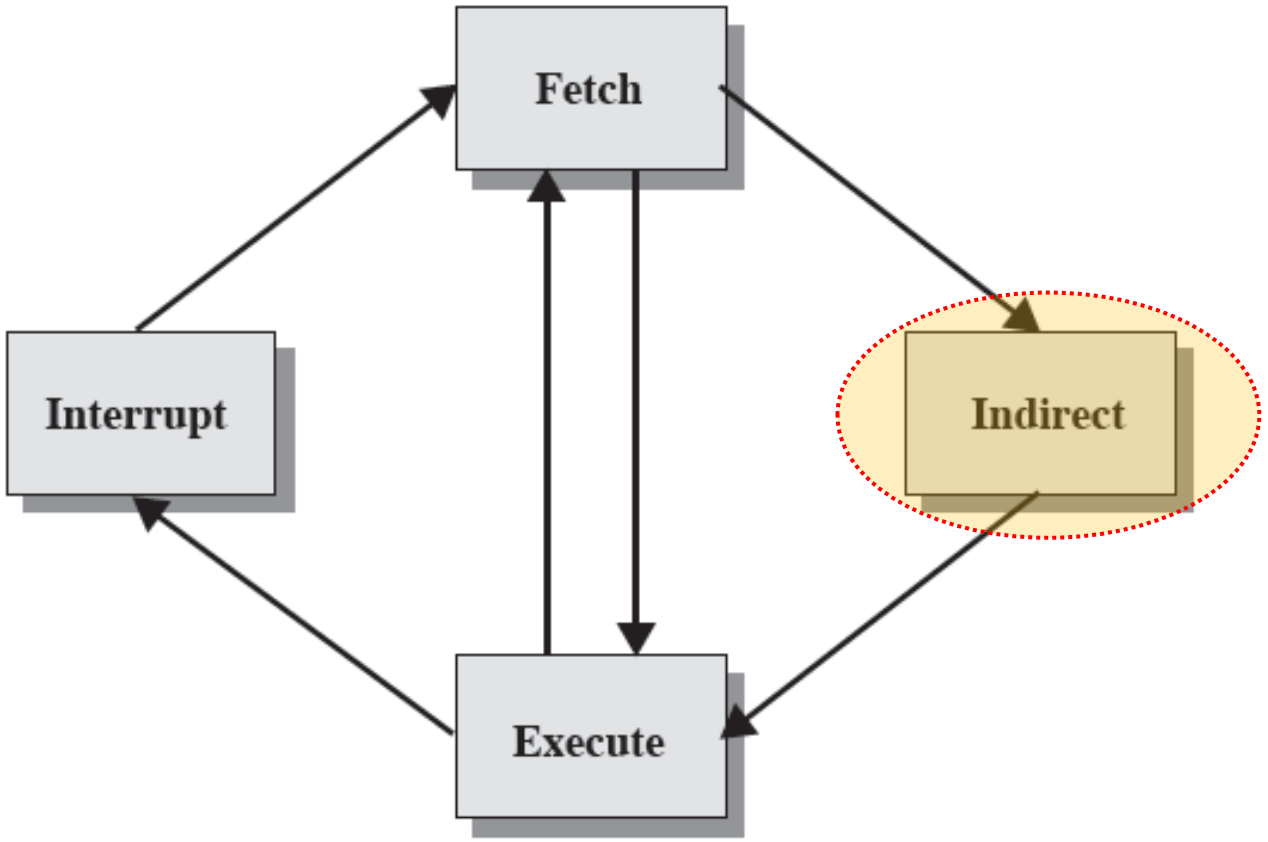
- Instruction in MBR is loaded into IR.
- The opcode of the instruction will be decoded and translated so as to determine the micro-operations for the particular instruction.

Data Flow, Indirect Cycle

- Once the **fetch cycle** is over, the control unit examines the content of **IR** ;
  - IF there is operand using direct addressing,
    - an indirect cycle is performed
    - example: mov AX, val1
  - ELSE
  - run the execute cycle
    - i.e. operand uses immediate addressing
    - example: add AX, 2000h



Data Flow, Indirect Cycle



Data Flow, Indirect Cycle

- Involve with *fetching data from memory* before initiating the execute cycle.

  Example: **ADD AX, Val1**

- In this cycle, the *operand bits* of the instruction in **IR** are transferred to **MAR**.
  - i.e., 2nd operand, **Val1**, is located in the RAM
- The *read* control line is activated, and the data associated with the address will be read and transfer to **MBR**.

Data Flow, Indirect Cycle

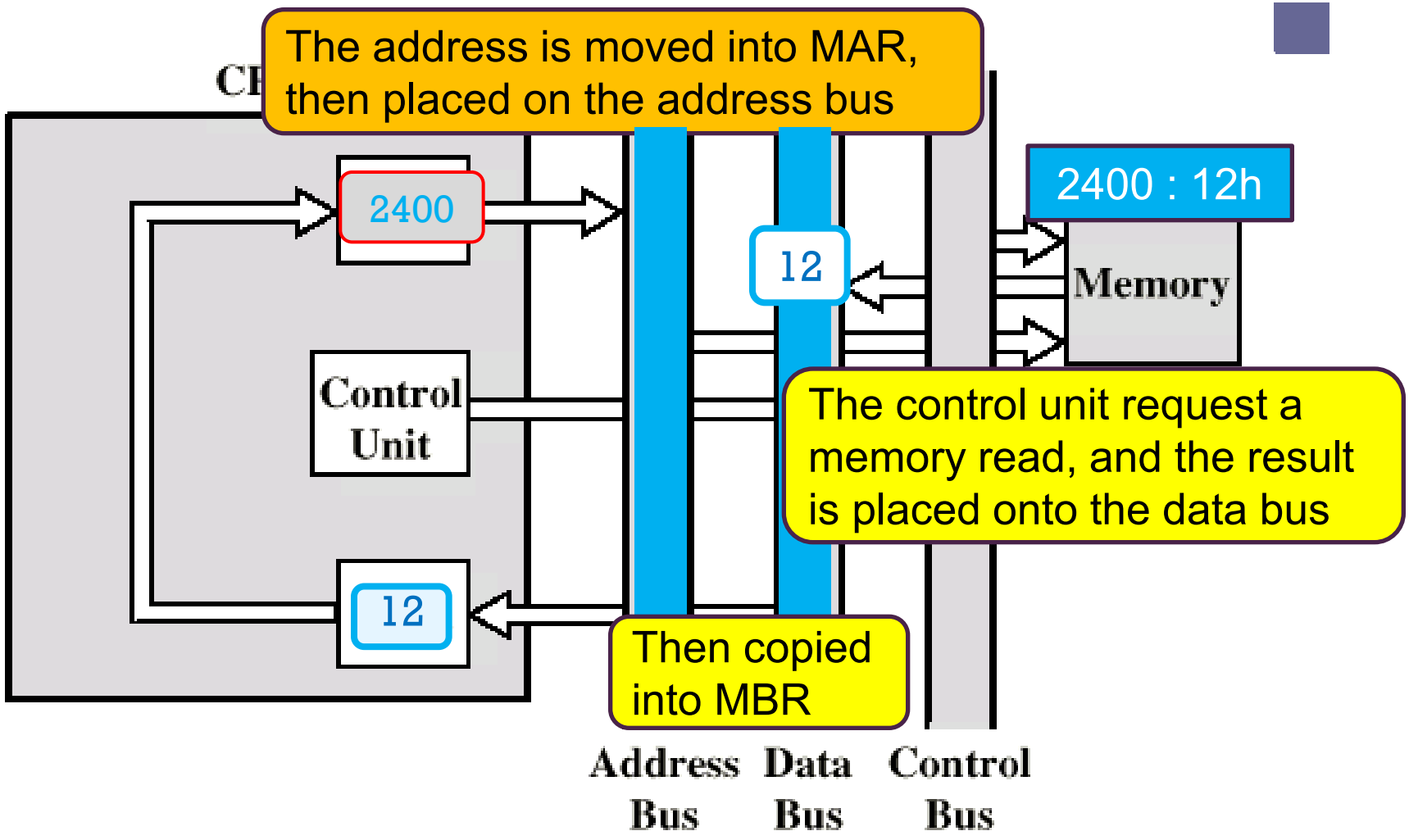
- The **indirect cycle** can be written symbolically using RTL as:

t1 : MAR ← IR [operand/address]

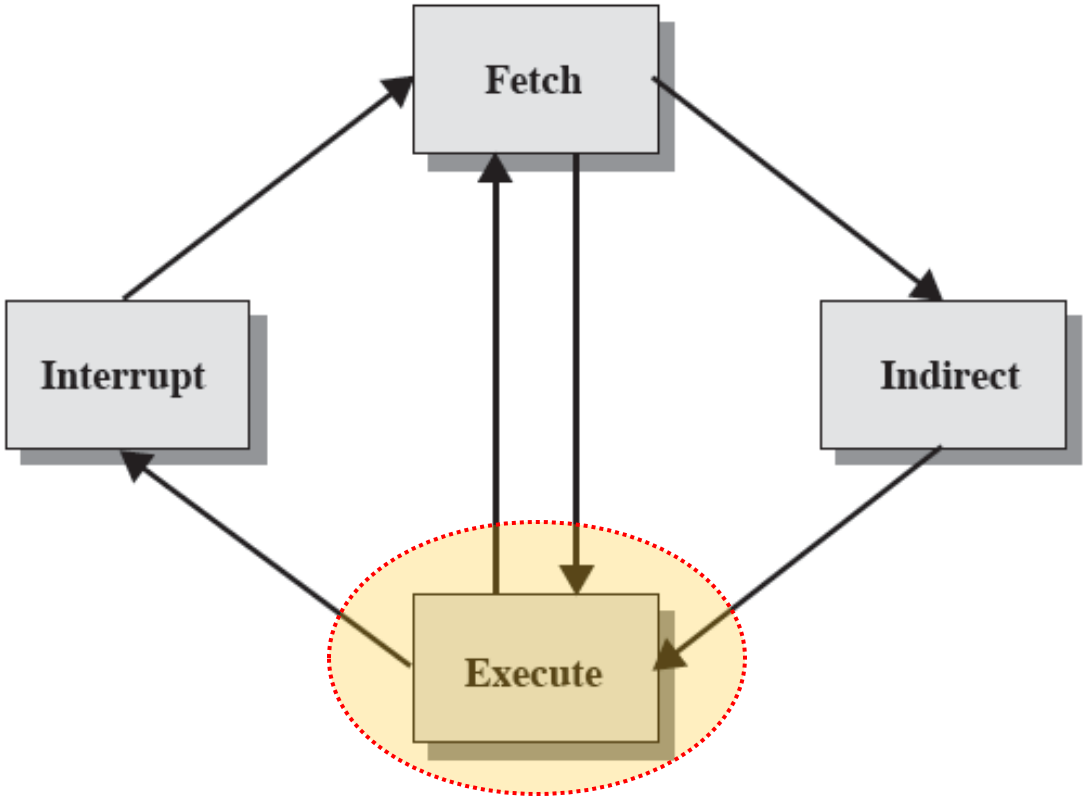
t2 : MBR ← Mem[MAR]

Data Flow, Fetch: Indirect Cycle

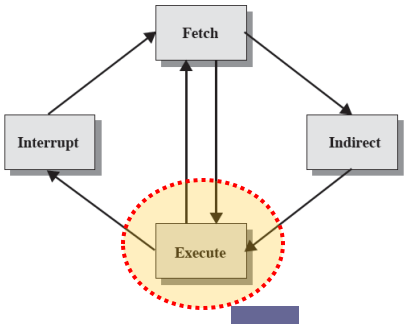
Example: add AX, Val1 ; *[2400]*



Data Flow, Execute Cycle



Data Flow, Execute Cycle



- The opcode of the instruction in **IR** will be decoded and related control signals will be generated.

Type of opcode	Operation
Processor-memory	data transfer between CPU and main memory
Processor I/O	Data transfer between CPU and I/O module
Data processing	Some arithmetic or logical operation on data
Control	Alteration of sequence of operations e.g. jump
	Combination of above

Example of Program Execution

Program Execution:

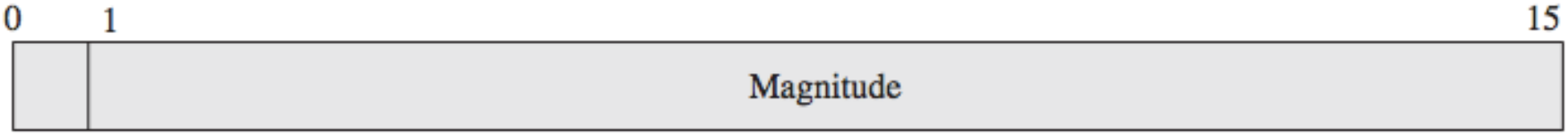
Example 1

Example 1: Program Execution Using MARIE CPU (Linda Null)

ISA for MARIE



(a) Instruction format



(b) Integer format

Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

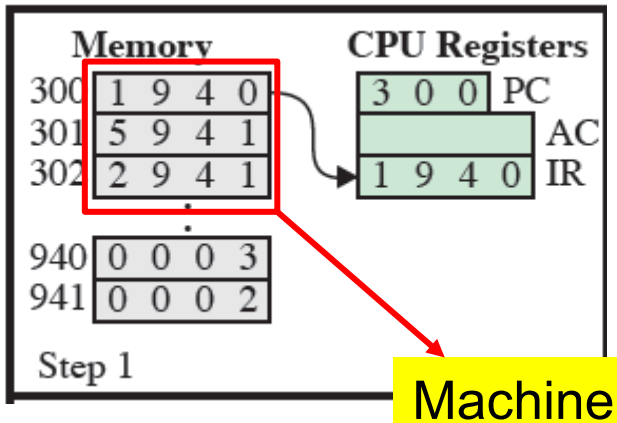
(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory

(d) Partial list of opcodes

Assembly code:

```
LOAD   var1
ADD    var2
STORE  var2
```



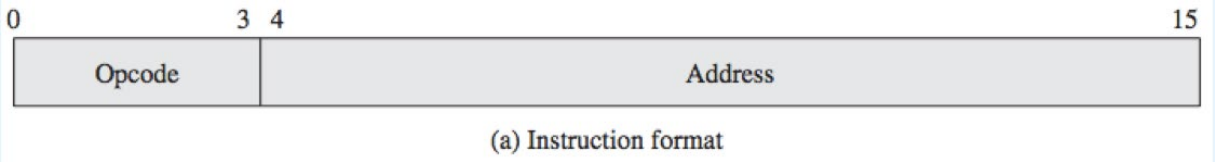
Machine code OR bytecodes (in hexadecimal)

Opcode:

1 = LOAD

5 = ADD

2 = STORE



Assembly code:

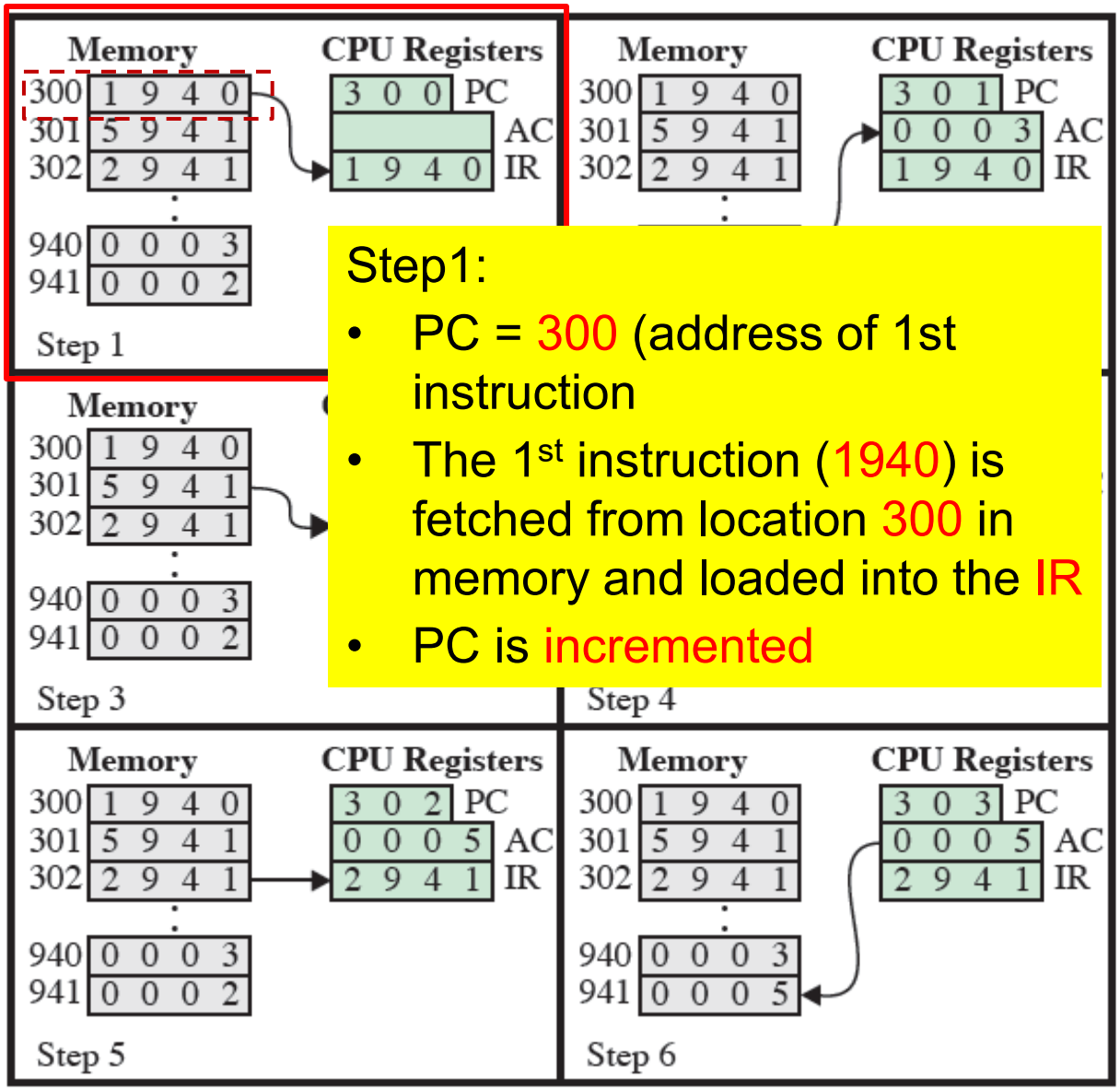
```
LOAD   var1
ADD    var2
STORE  var2
```

Opcode:

1 = LOAD

5 = ADD

2 = STORE



Step1:

- PC = 300 (address of 1st instruction
- The 1st instruction (1940) is fetched from location 300 in memory and loaded into the IR
- PC is incremented

Assembly code:

```
LOAD   var1
ADD    var2
STORE  var2
```

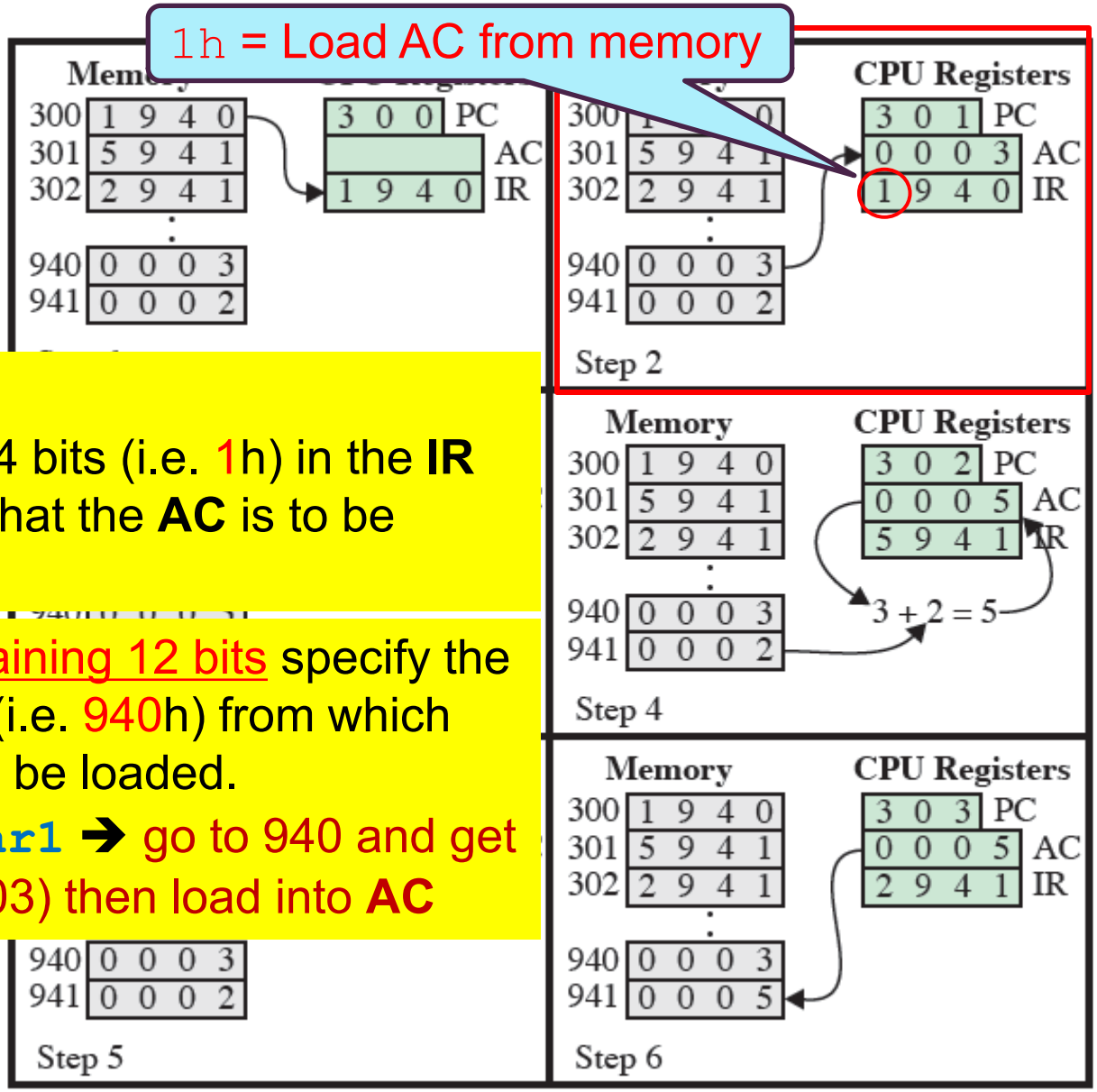
Opcode:

1 = LOAD
5 = ADD
2 = STORE

Step2:

- The first 4 bits (i.e. 1h) in the **IR** indicate that the **AC** is to be loaded.

- The remaining 12 bits specify the address (i.e. 940h) from which data is to be loaded.
- **LOAD var1** ➔ go to 940 and get data (0003) then load into **AC**



Assembly code:

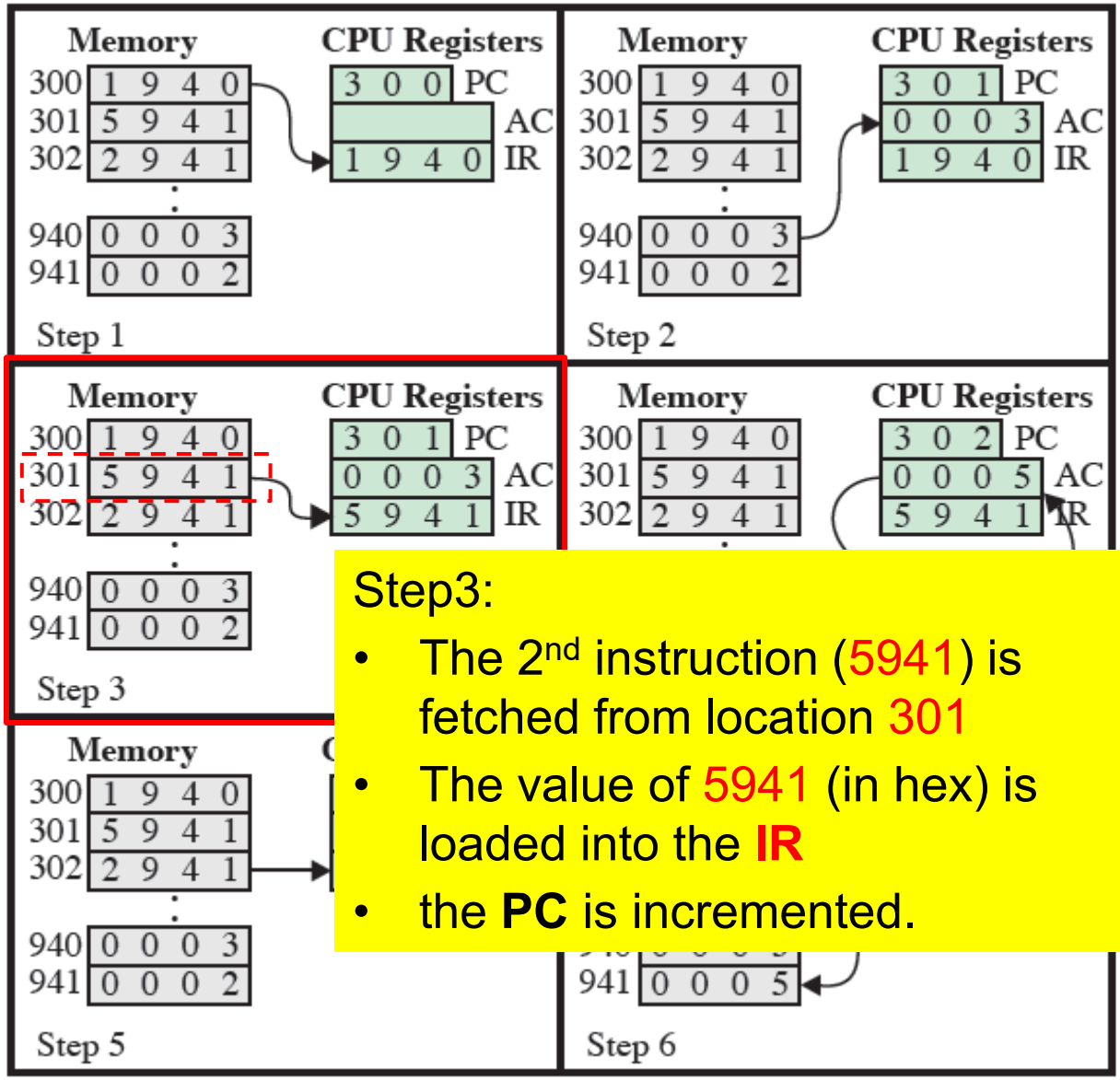
```
LOAD   var1
ADD    var2
STORE  var2
```

Opcode:

1 = LOAD

5 = ADD

2 = STORE



Step3:

- The 2nd instruction (5941) is fetched from location 301
- The value of 5941 (in hex) is loaded into the **IR**
- the **PC** is incremented.

Assembly code:

```
LOAD   var1
ADD    var2
STORE  var2
```

O

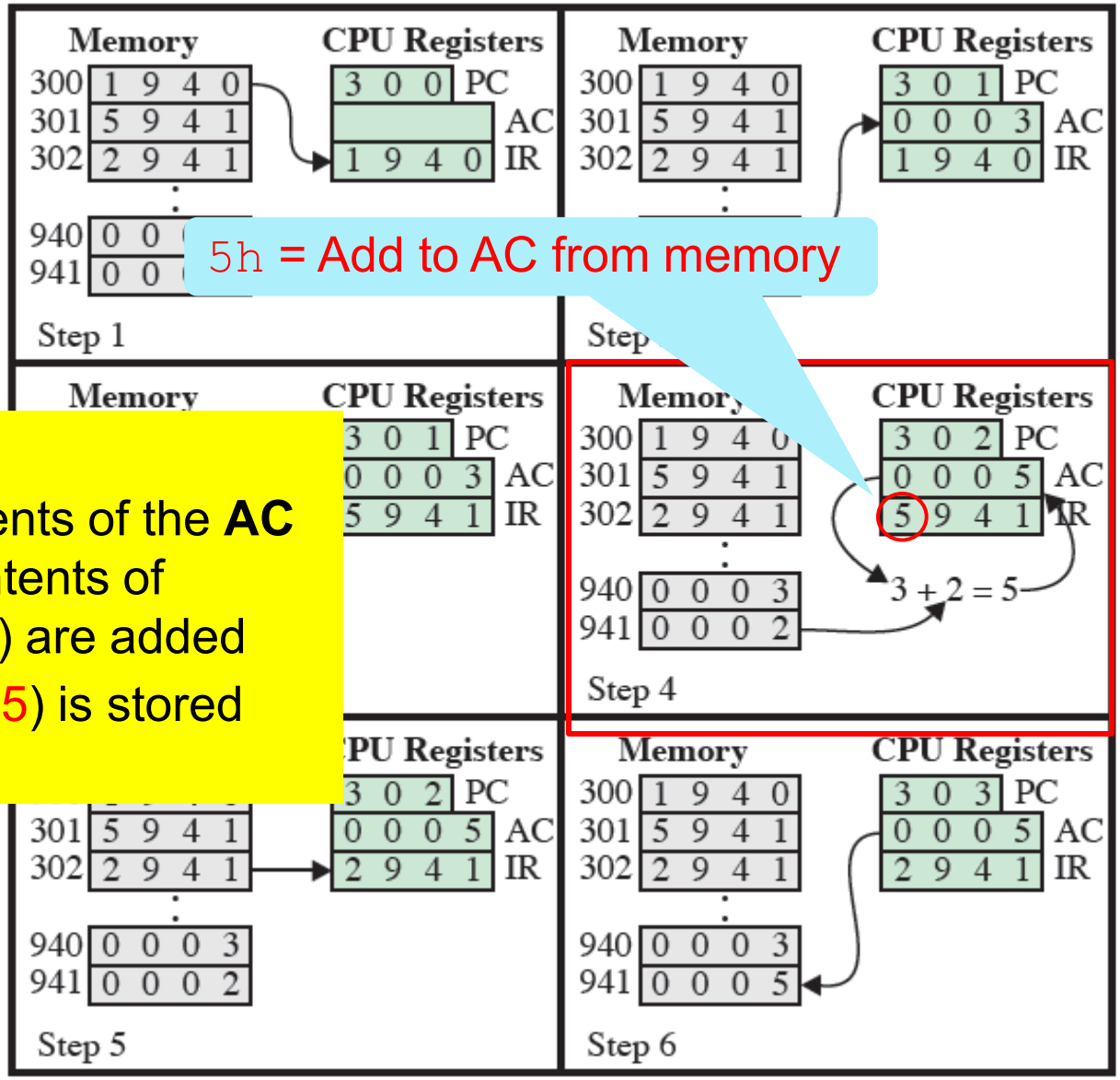
1

5

2

Step4:

- The previous contents of the **AC** (0003) and the contents of location 941 (0002) are added
- and the result (0005) is stored back into the **AC**



Assembly code:

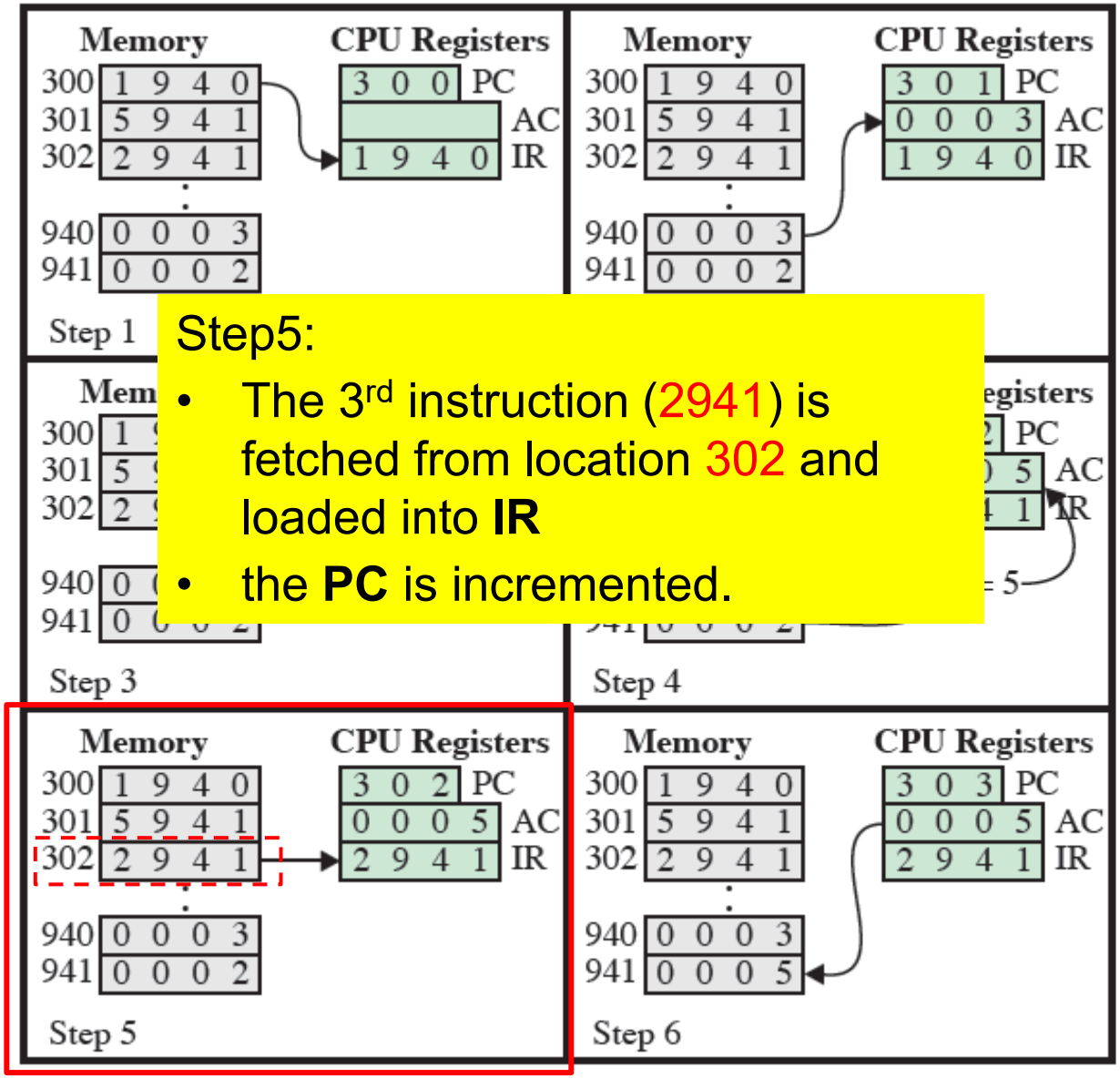
```
LOAD   var1
ADD    var2
STORE  var2
```

Opcode:

1 = LOAD

5 = ADD

2 = STORE



Step5:

- The 3rd instruction (2941) is fetched from location 302 and loaded into **IR**
- the **PC** is incremented.

Assembly code:

```
LOAD   var1
ADD    var2
STORE  var2
```

Opcode:

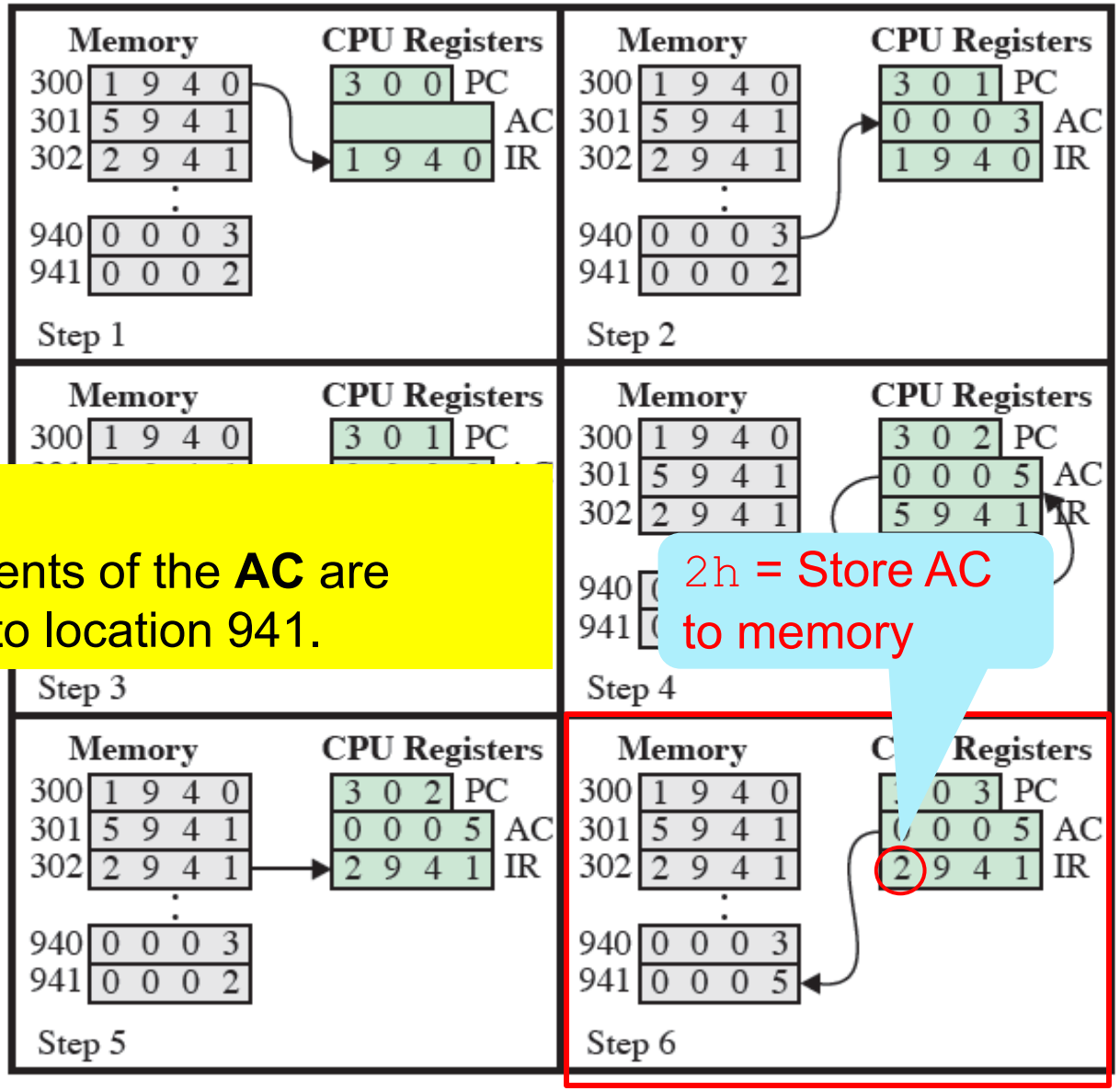
1 = LOAD

5 = ADD

2 = STORE

Step6:

- The contents of the **AC** are stored into location 941.



Assembly code:

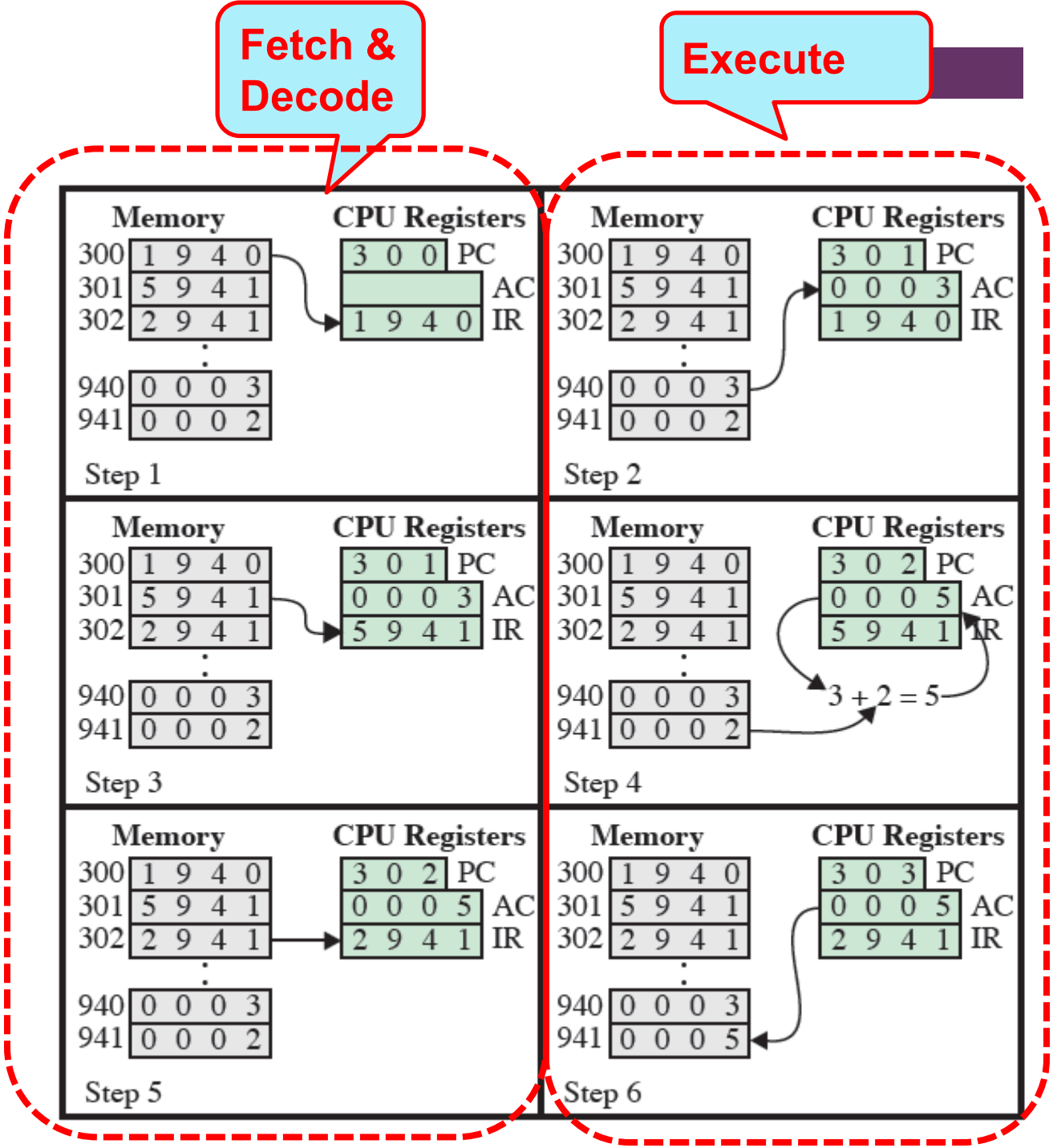
```
LOAD   var1
ADD    var2
STORE  var2
```

Opcode:

1 = LOAD

5 = ADD

2 = STORE

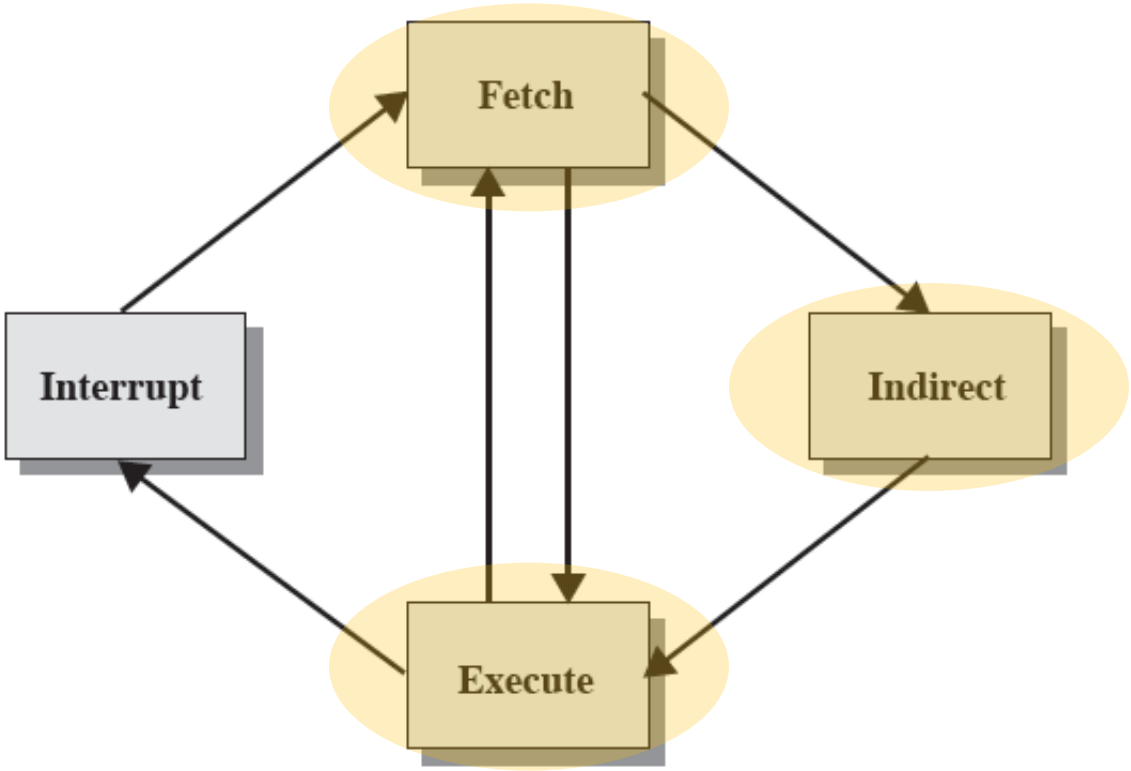


Example of Program Execution

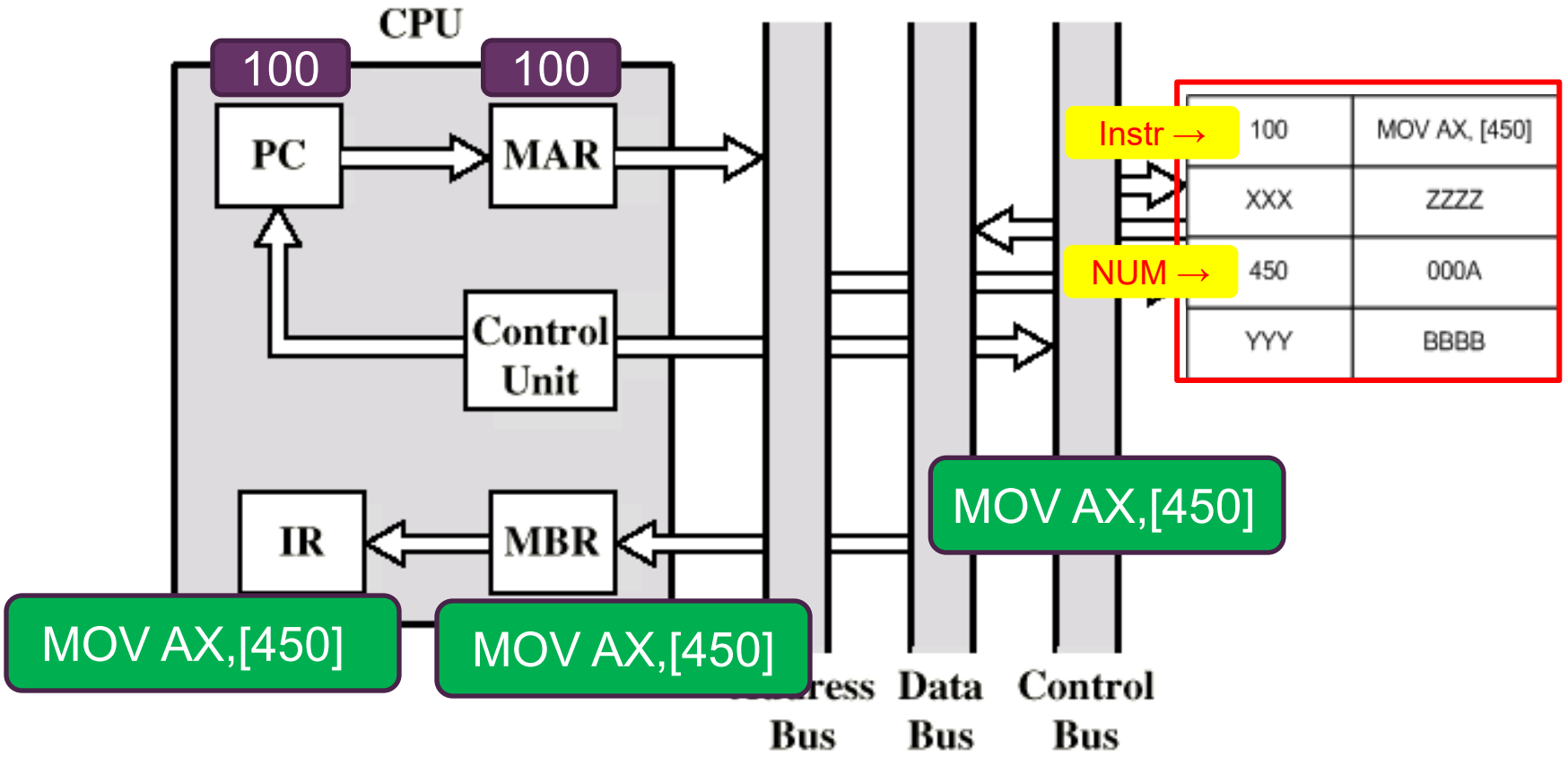
Example 2:

MOV AX, NUM

Example 2: MOV AX, NUM

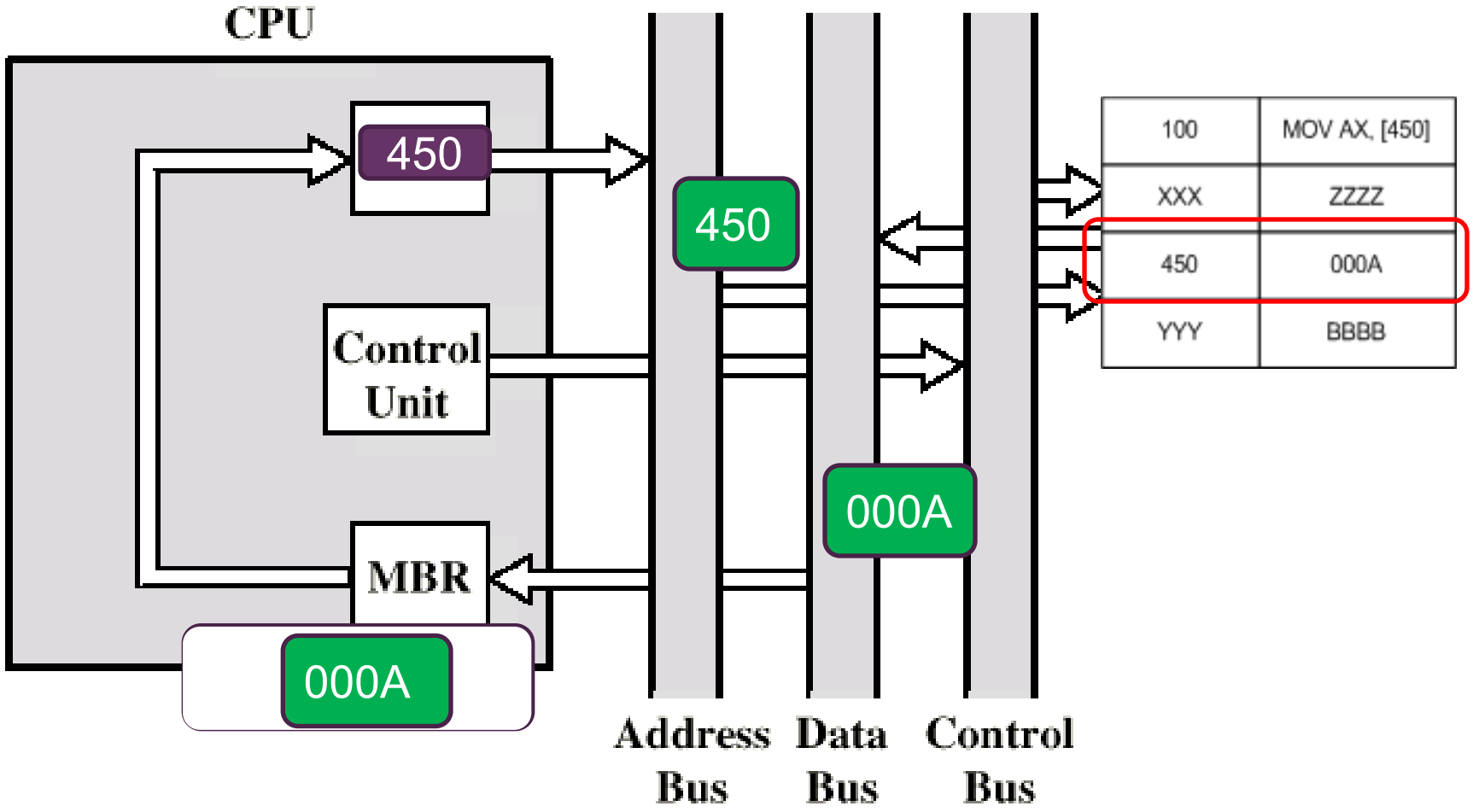


Example 2: MOV AX, NUM (Cycle: fetch)

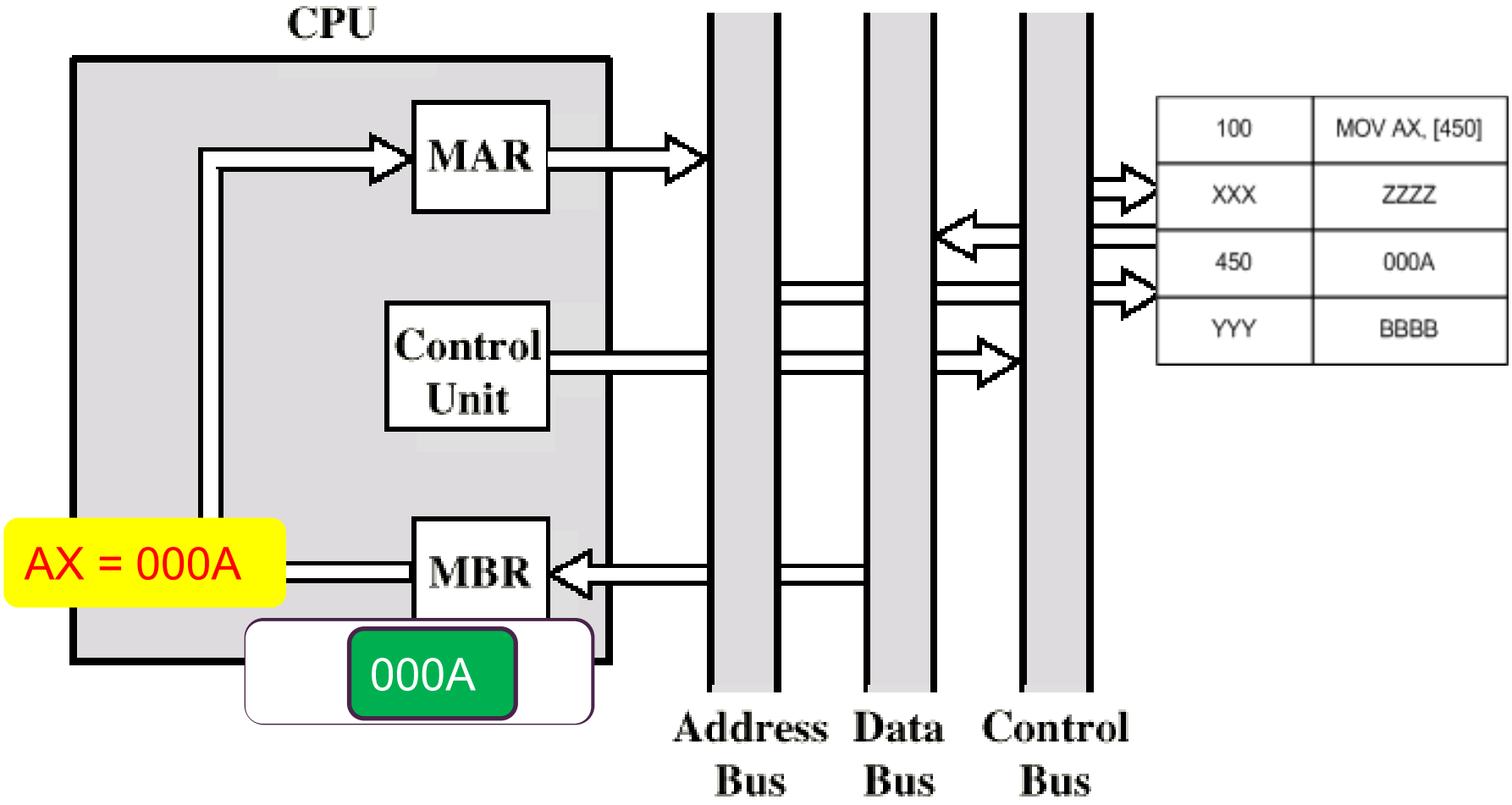


MBR = Memory buffer register
MAR = Memory address register
IR = Instruction register
PC = Program counter

Example 2: MOV AX, NUM (Cycle : indirect)



Example 2: MOV AX, NUM (Cycle : execute)



Example 2: MOV AX, NUM
- *A Closer Look using RTL*

Clock	IP/PC	MAR	MBR	IR	AX	μ operation
t_0	100					IP/PC= 100
t_1		100				**fetch cycle:** MAR ← IP/PC;
t_2	101	100	4450			MBR ← [MAR]; IP/PC ← IP/PC +1;
t_3	101	100	4450	4450		IR ← MBR
t_4	101	450	4450	4450		**indirect cycle:** MAR← IR[operand/address];
t_5	101	450	000A	4450		MBR ← Mem[MAR];
t_6	101	450	000A	4450	000A	**execution cycle:** AX ← MBR;



Exercise 5.2 :

Trace the execution of the instruction by showing all the changes in CPU registers (control and general-purpose registers) as well as the *micro-operations* related to the instructions. Use the given initial table.

Memory address	Memory Content	Instruction/Data
39D	A450	L1: XCHG CX, NUM
39E	B451	SUB VAL1,CX
39F	C39D	JMP L1:
450	100	NUM
451	500	VAL1

Clock	IP/PC	MAR	MBR	IR	CX	NUM	VAL1	Micro-operation
t_0					200	100		

(c) Interrupt Cycle

- All computers provide a way of interrupting the fetch-decode-execute cycle.
- Interrupts occur when:
  - A user break (*e.g.*, Control+C) is issued.
  - I/O is requested by the user or a program.
  - A critical error occurs.
- Interrupts can be caused by hardware or software.

→ Software interrupts are also called *traps*.

Interrupts :: Definition

- A mechanism by which other system modules (e.g. I/O devices, memory) may interrupt the normal processing of the CPU.
- These devices/modules are much slower than the CPU
  - CPU can waste vast amounts of processing cycles waiting for these slow devices to perform their tasks
- Interrupts let the CPU pause its normal instruction execution sequence to service the external devices

Interrupt Cycle

- Interrupts are processed in an interrupt cycle within the overall instruction cycle.

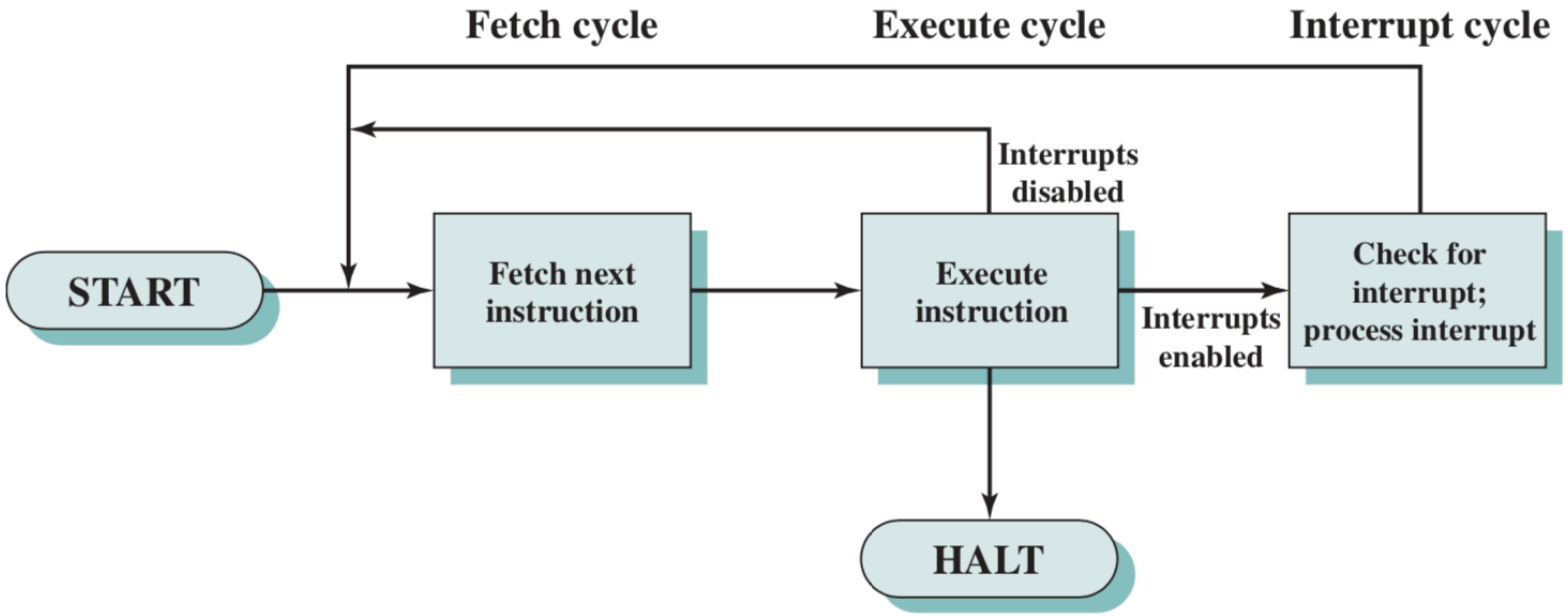


Figure 3.9 Instruction Cycle with Interrupts

Interrupt Processing: Flowchart

At the end of an instruction cycle, CPU will check if any interrupts are pending:

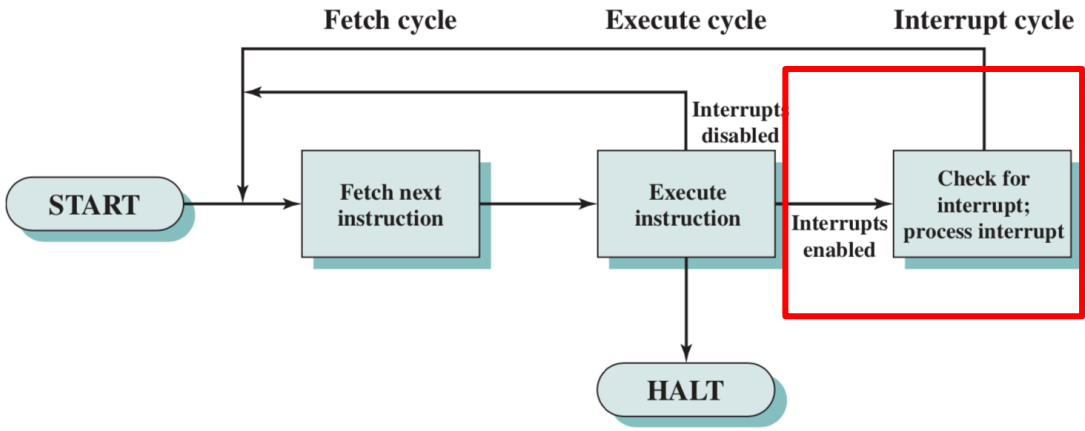
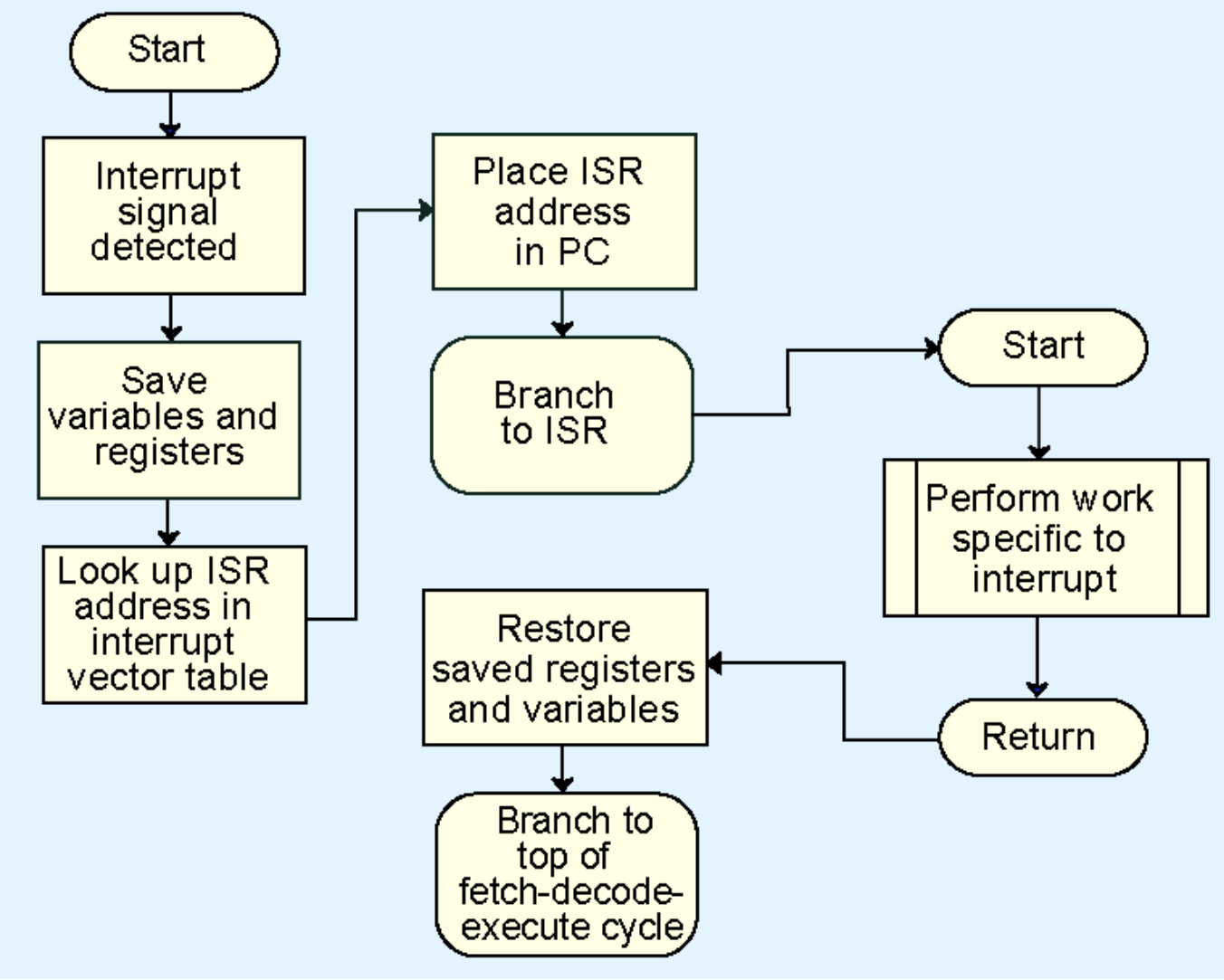


Figure 3.9 Instruction Cycle with Interrupts

- If NO, proceed with the next instruction.
- If **YES**, then:
  - Suspend execution of the program and save its "state".
  - Jump to the **interrupt service routine (ISR)** and run the normal instruction cycle for ISR.
  - When the ISR is completed, restore the state of the program and resume its operation.

…next slide show the details using ***flowchart***

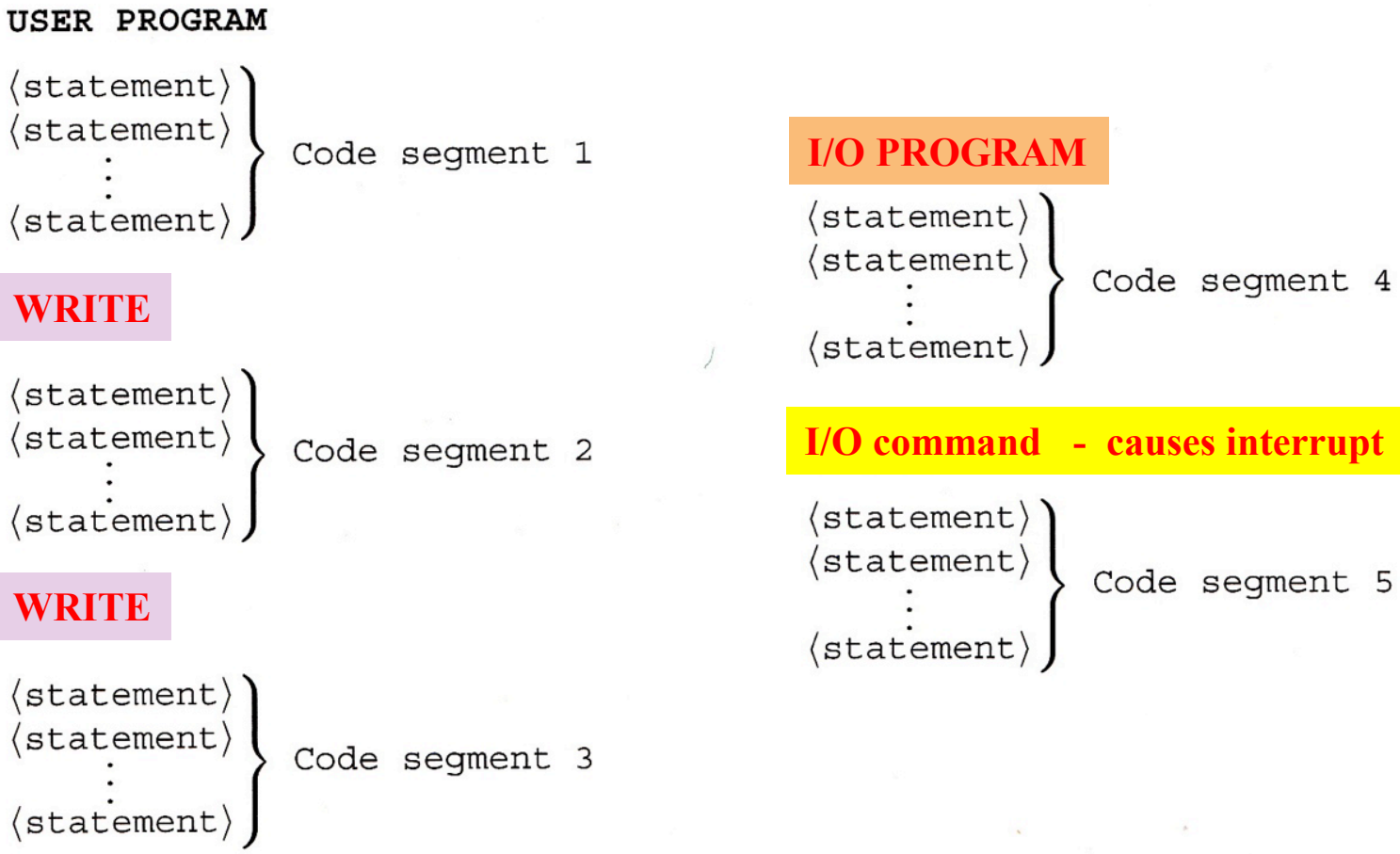
Interrupt Processing: Flowchart



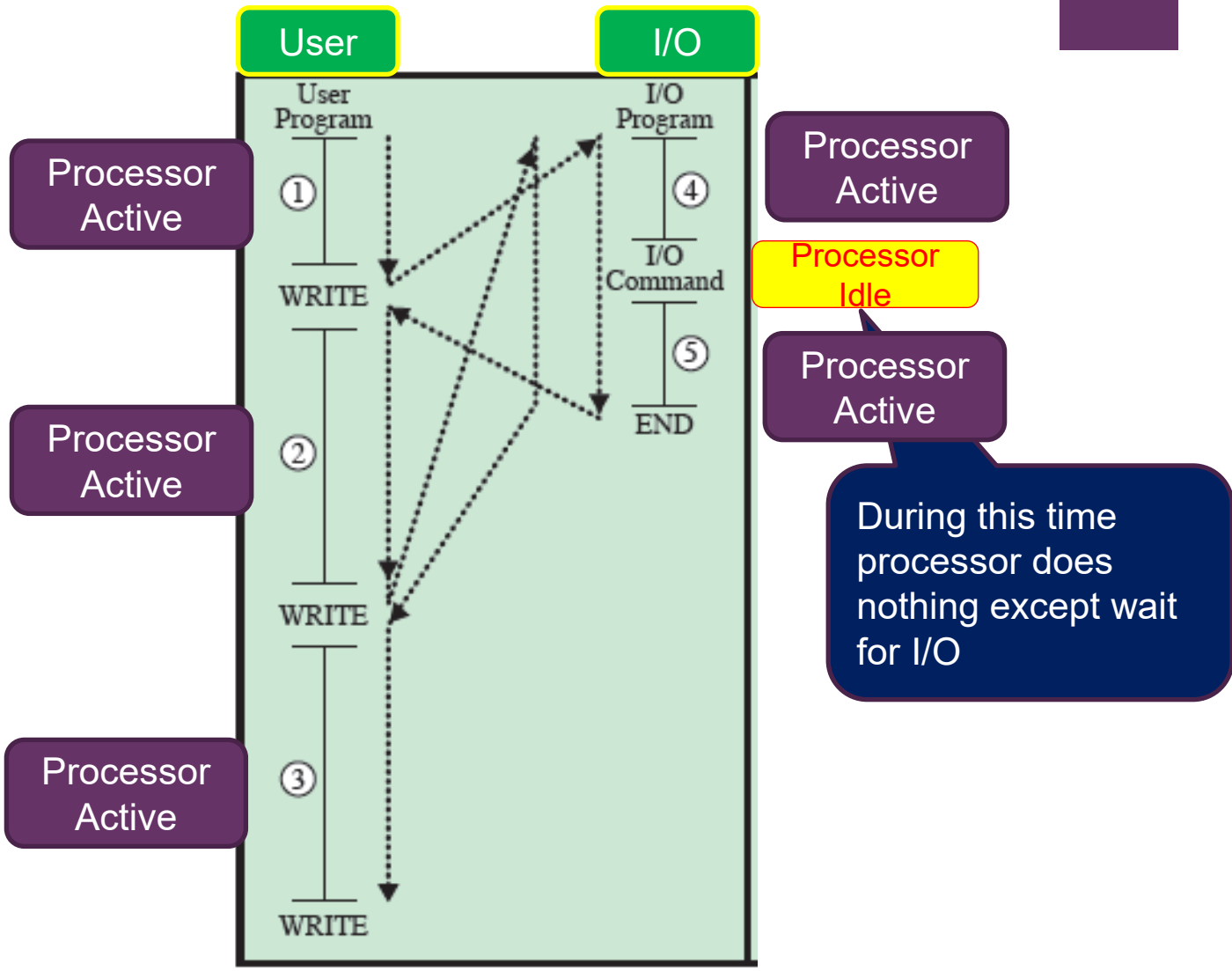
Interrupt Processing

- The *processor and the O/S* are responsible for
  - recognizing an interrupt,
  - suspending the user program,
  - servicing the interrupt,
  - and then resuming the user program.
- Interrupts provide a way to *improve CPU processing efficiency*
  - due to I/O processing is TOO SLOW

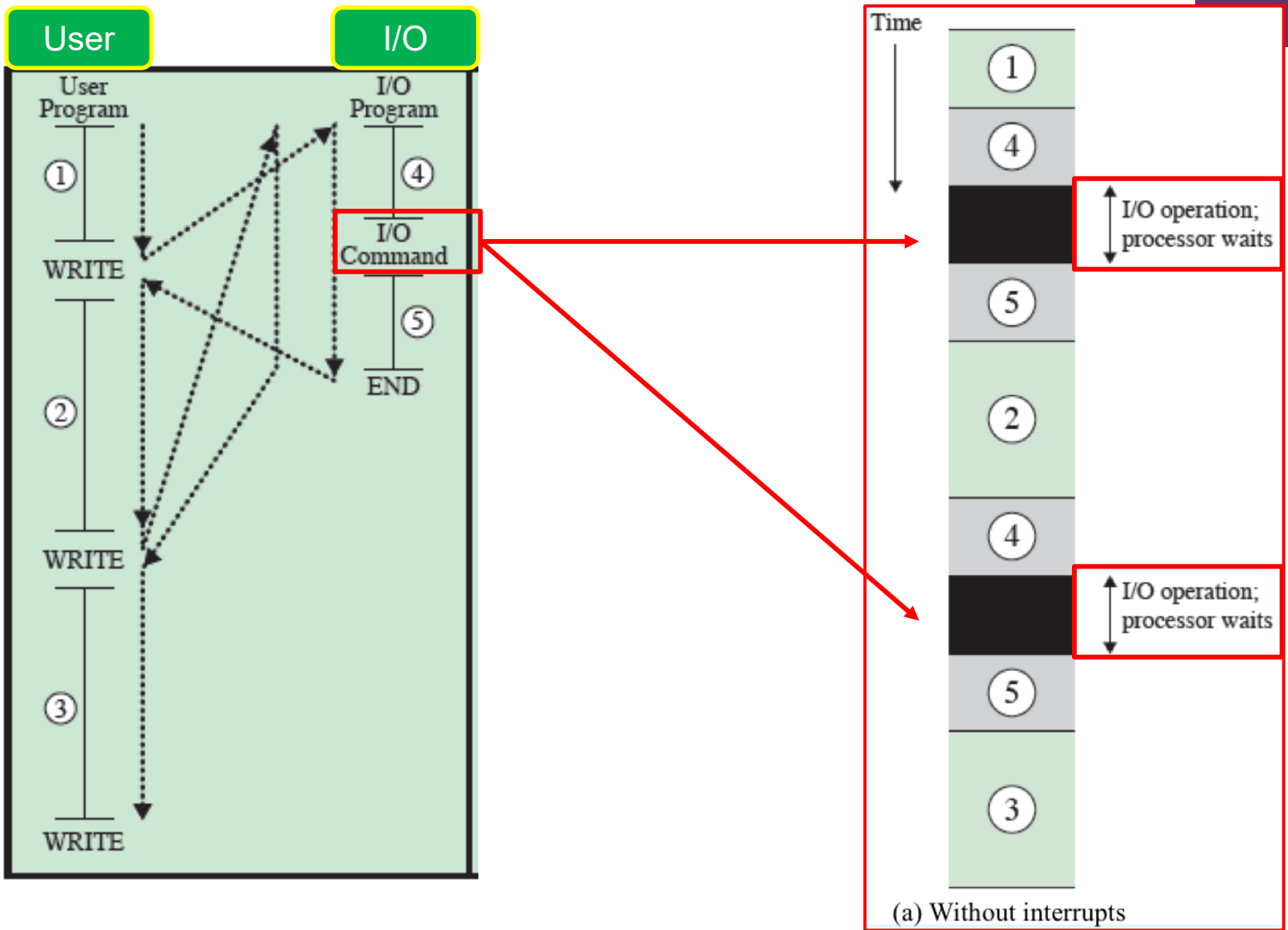
Interrupts :: Example



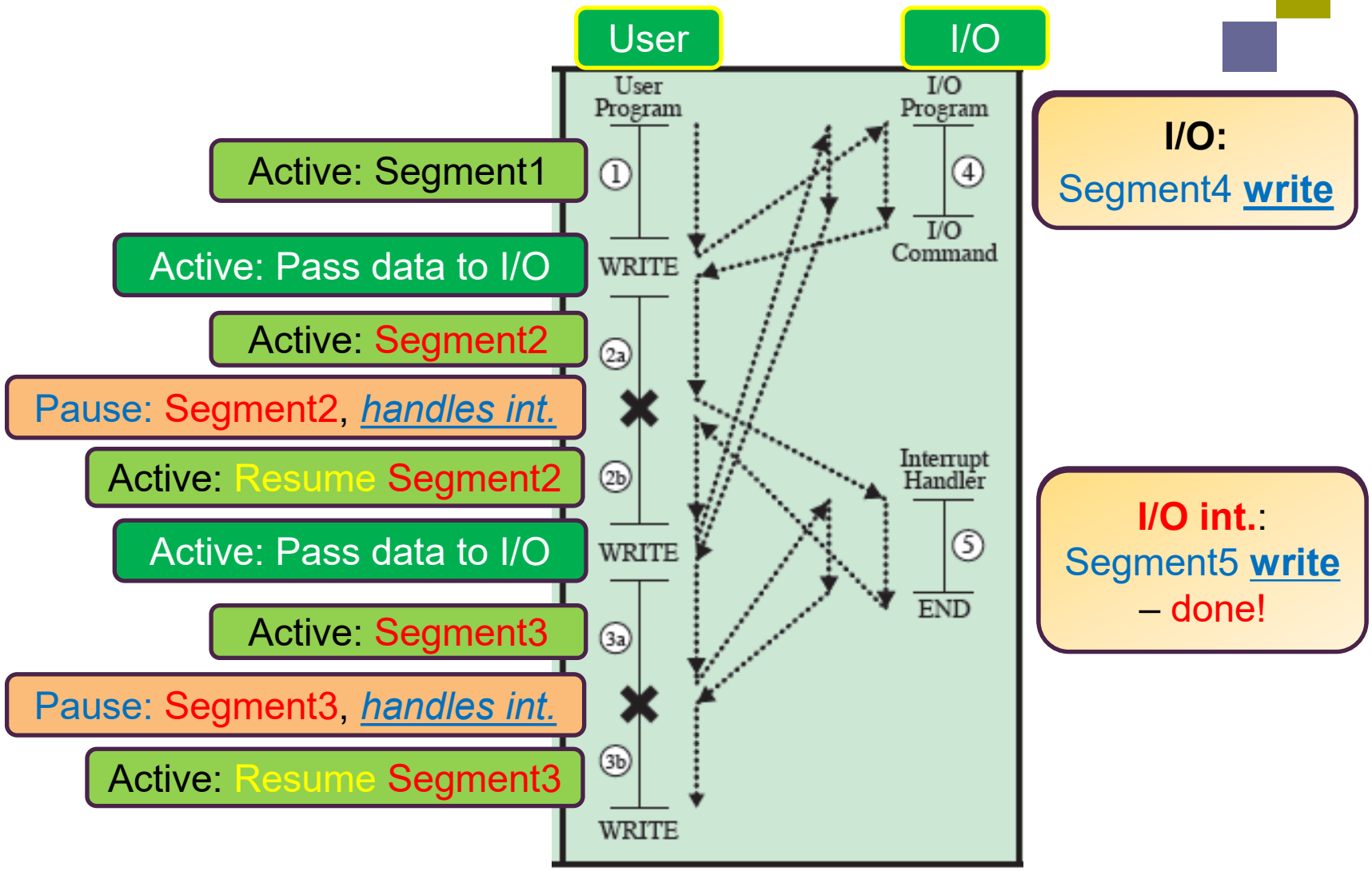
Example: **NO** Interrupts



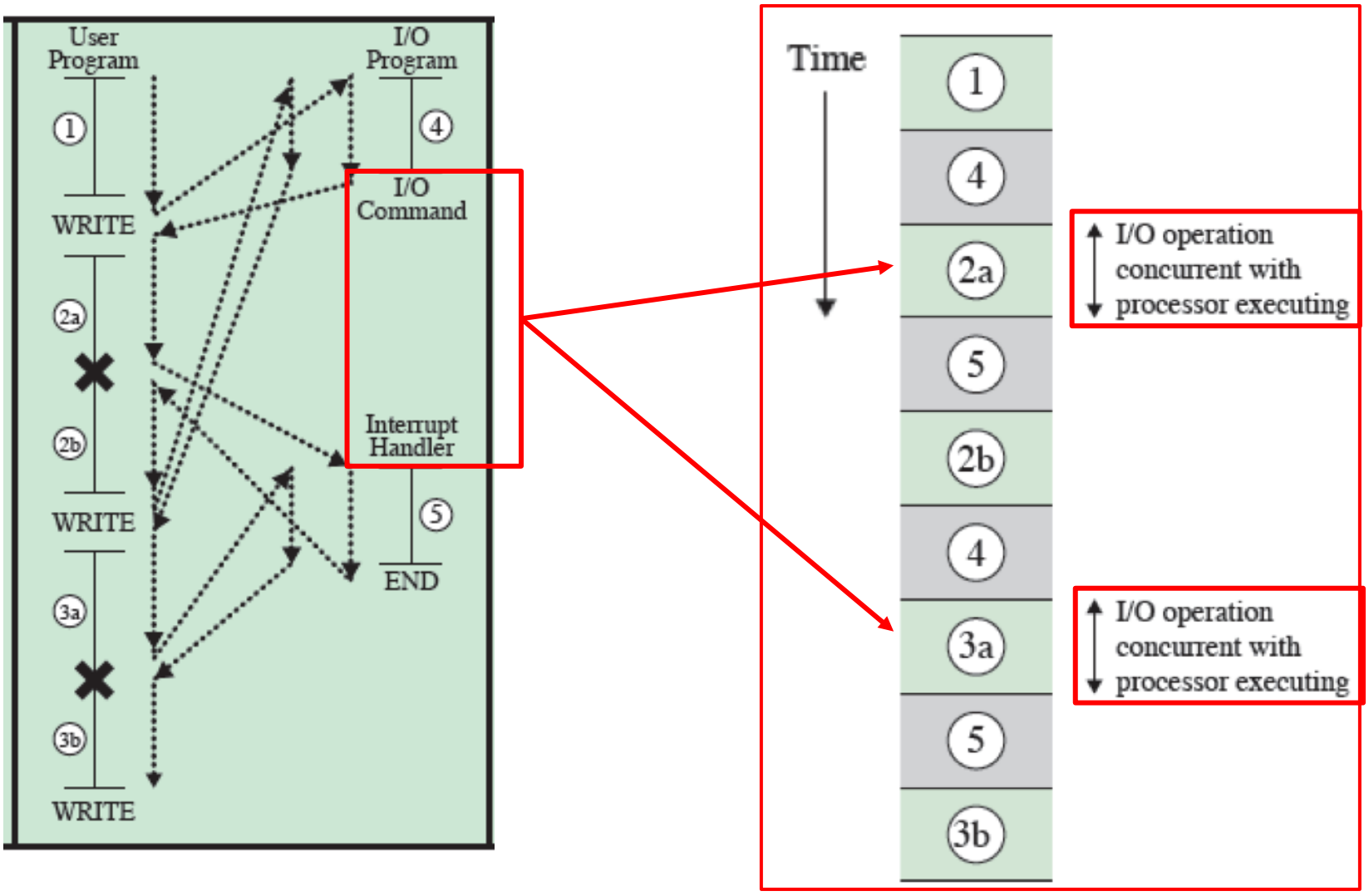
Timing Diagram: **NO** Interrupts



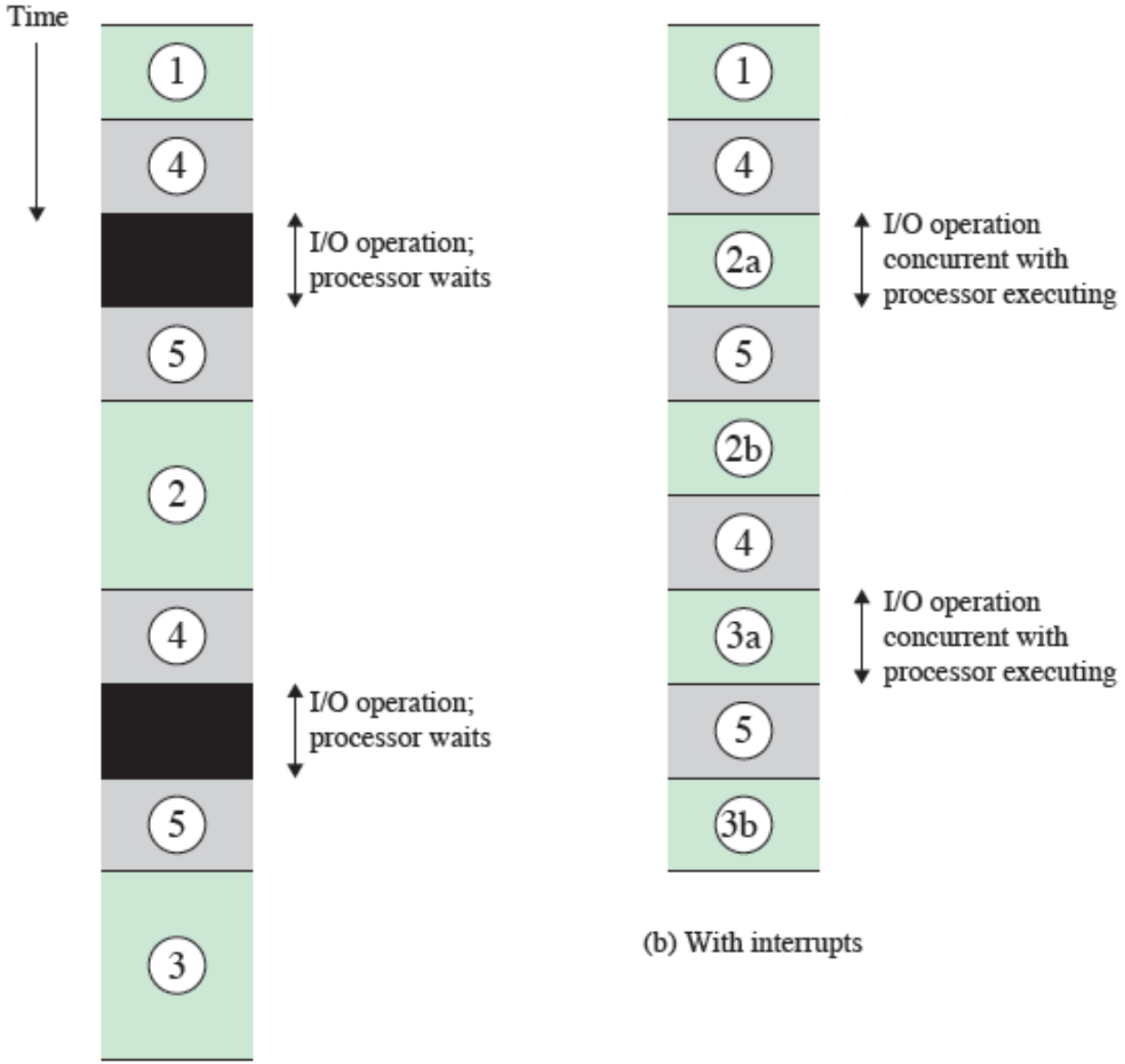
Example: **WITH** Interrupts



Timing Diagram: WITH Interrupts



Comparison: WITHOUT vs WITH Interrupts



(a) Without interrupts

(b) With interrupts

5

Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation, request service from the processor, or to signal a variety of error conditions.
Hardware failure	Generated by a failure such as power failure or memory parity error.

Figure: Class of interrupts.

5 Summary 1

- The major components of a computer system are its *control unit*, *registers*, *memory*, *ALU*, and *data path*.
- Distinguish between user-visible and control/status registers, and discuss the purposes of *registers* in each category.
- Computers run programs through iterative *fetch-decode-execute* cycles → Summarize the instruction cycle.
- All computers provide a way of interrupting the instruction cycle.

Module 5b

5.1 Processor Organization

5.2 Register Organization

5.3 Instruction Cycles

5.4 Instruction Pipelining

- Overview
- Pipeline Strategy
- Pipeline Performance
- Pipeline Hazards (Limitations)

5.5 Control Unit Operation

5.6 Microprogrammed Control

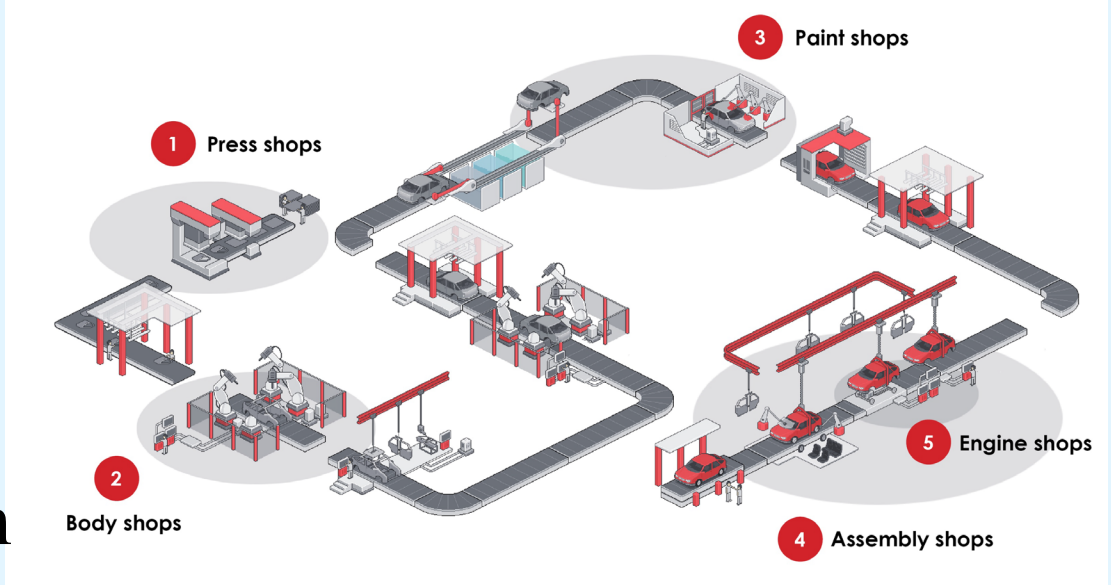
5.7 Summary

Pipeline

- Performance of a computer can be increased by increasing the performance of the CPU.
- This can be done by executing more than one tasks at a time.
- This procedure is referred to as pipelining.
- The concept of pipelining is to allow the processing of a new task even though the processing of previous task has not ended.

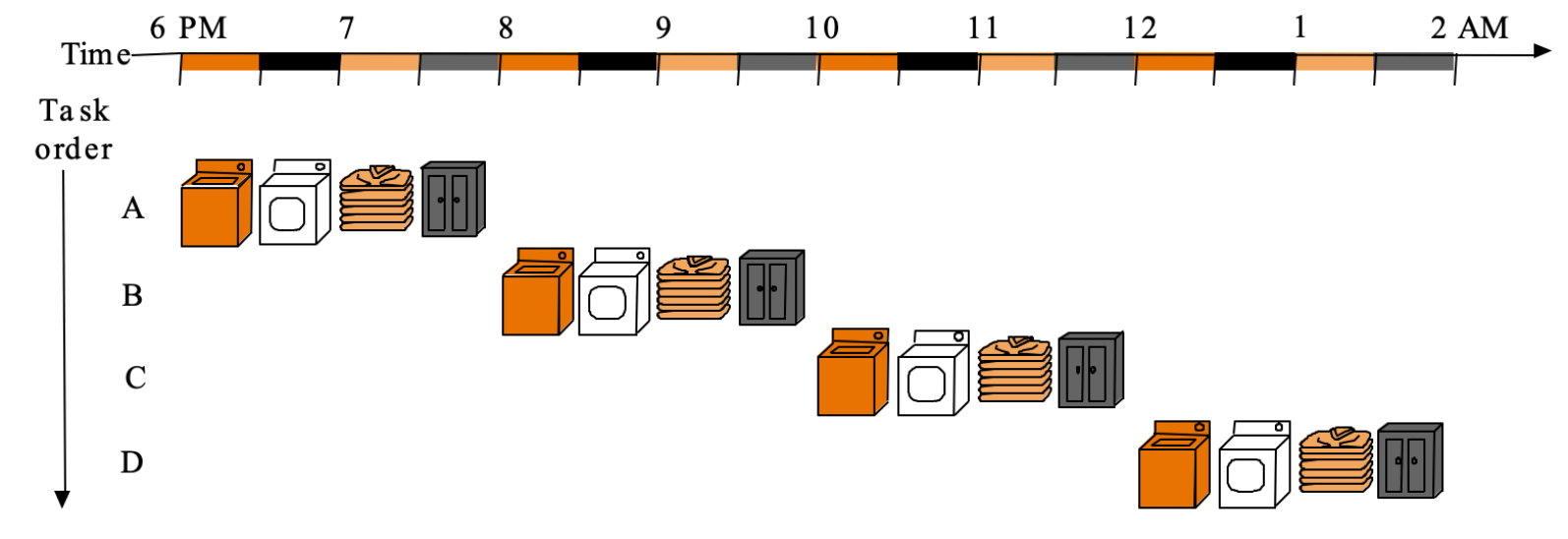
Analogy: Automobile Assembly Line

- Perform all tasks concurrently, but on different sequential segments
- For computer program execution, the approach is the instruction pipeline
- The result is temporal parallelism



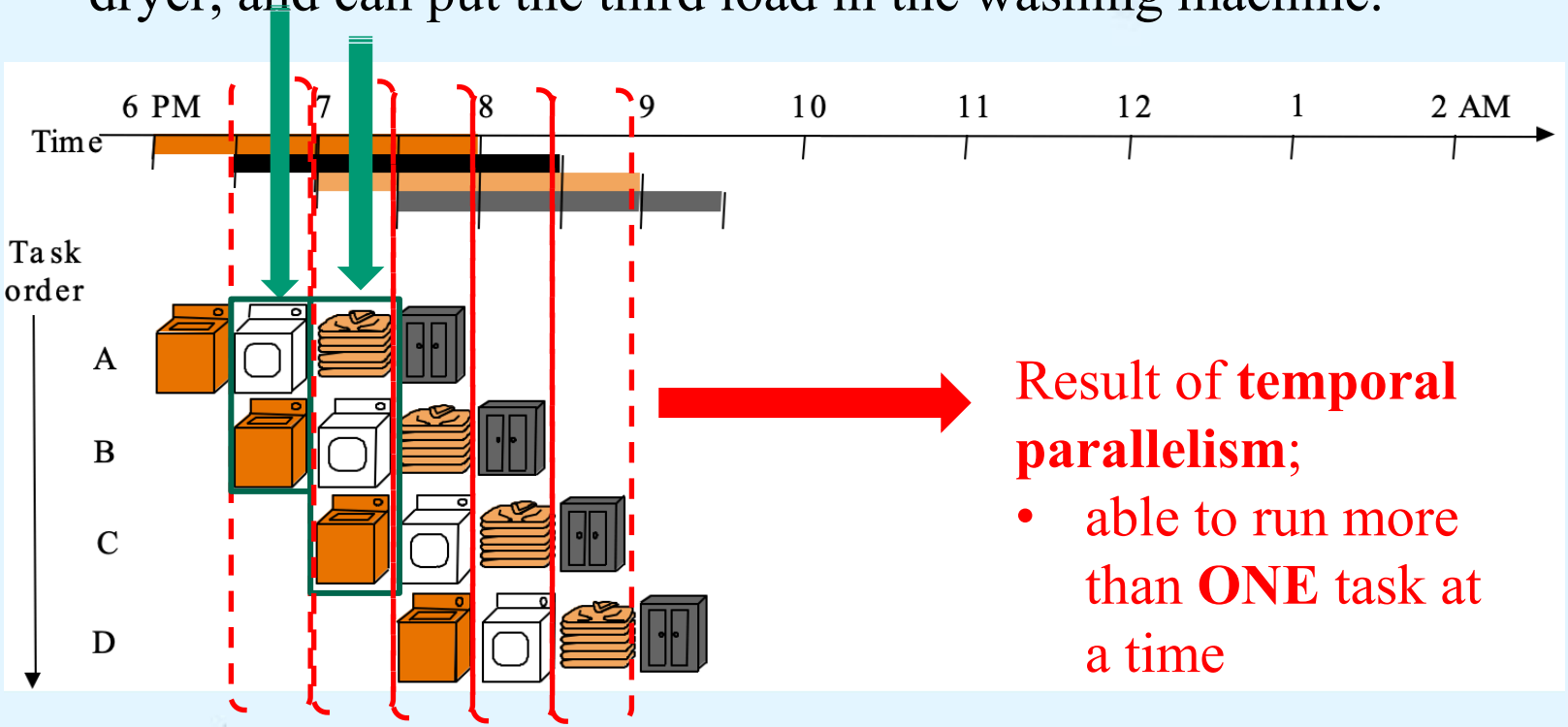
Analogy: Pipelined Laundry

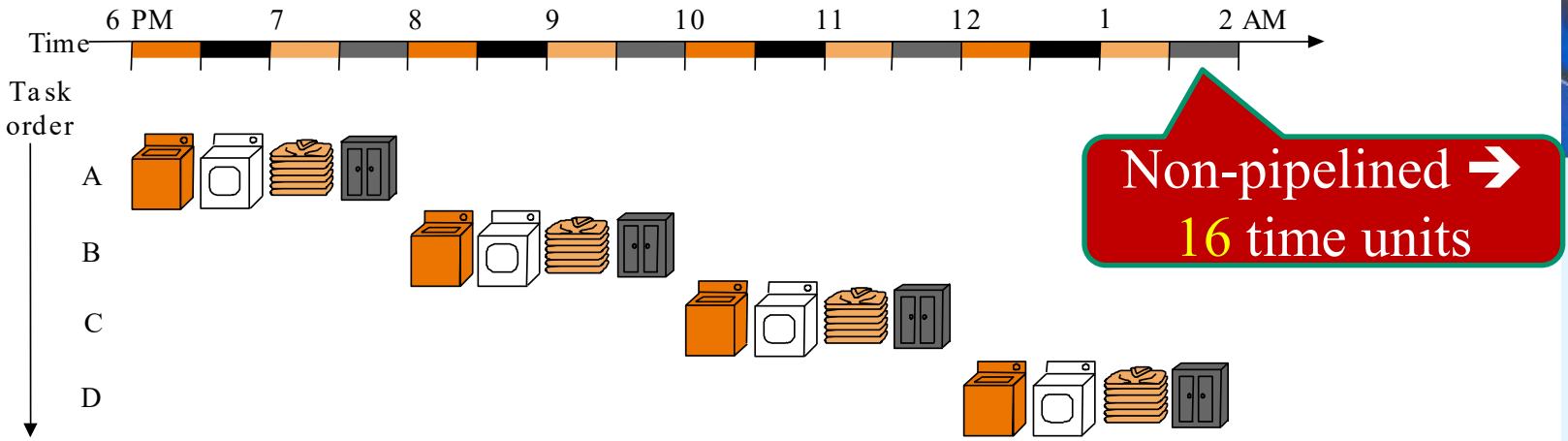
- Non-pipelined approach:
  1. run load of clothes through washer
  2. run load through dryer
  3. fold the clothes
  4. put the clothes into closet

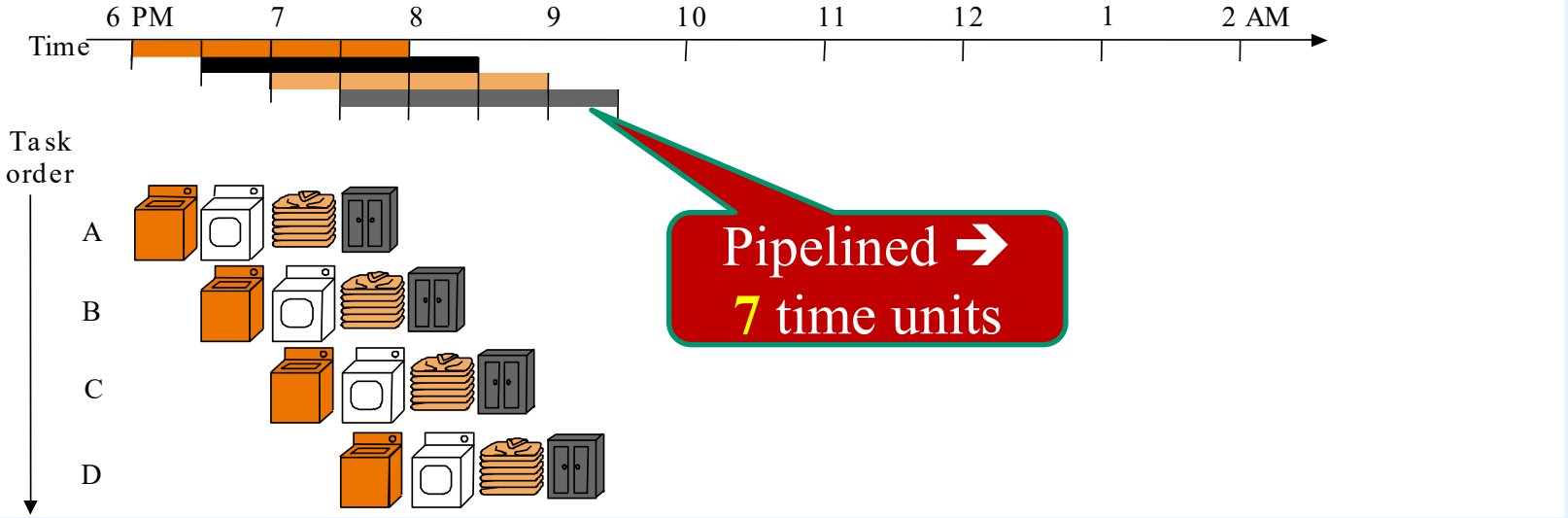


Analogy: Pipelined Laundry

- Pipeline approach:
  - While the first load is drying, put the second load in the washing machine.
  - When the first load is being folded and the second load is in the dryer, and can put the third load in the washing machine.




6 PM
7
8
9
10
11
12
1
2 AM
Time
Task order
A
B
C
D
Non-pipelined ➔
16 time units

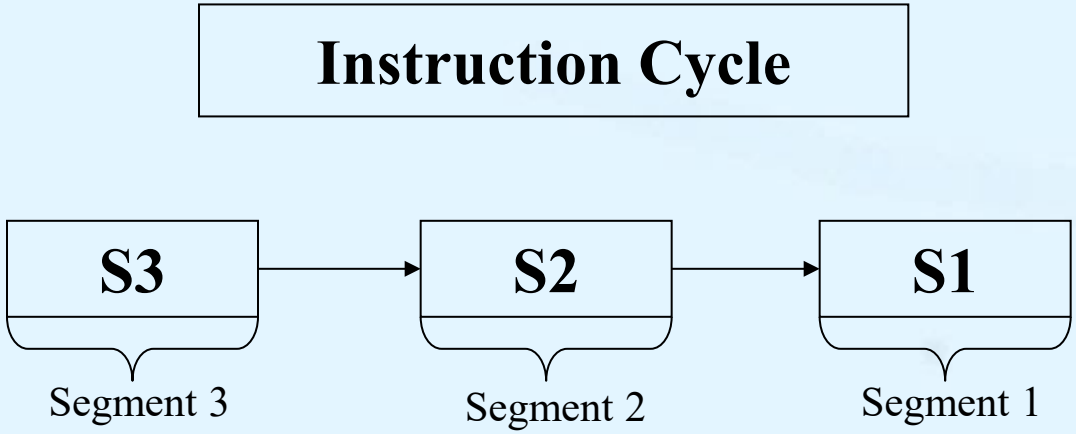

6 PM
7
8
9
10
11
12
1
2 AM
Time
Task order
A
B
C
D
Pipelined ➔
7 time units

Analogy: Pipelined Laundry

- For **4 loads**:
  - non-pipelined approach takes 16 units of time (**8 hours**).
  - pipelined approach takes 7 units of time (**3.5 hours**).

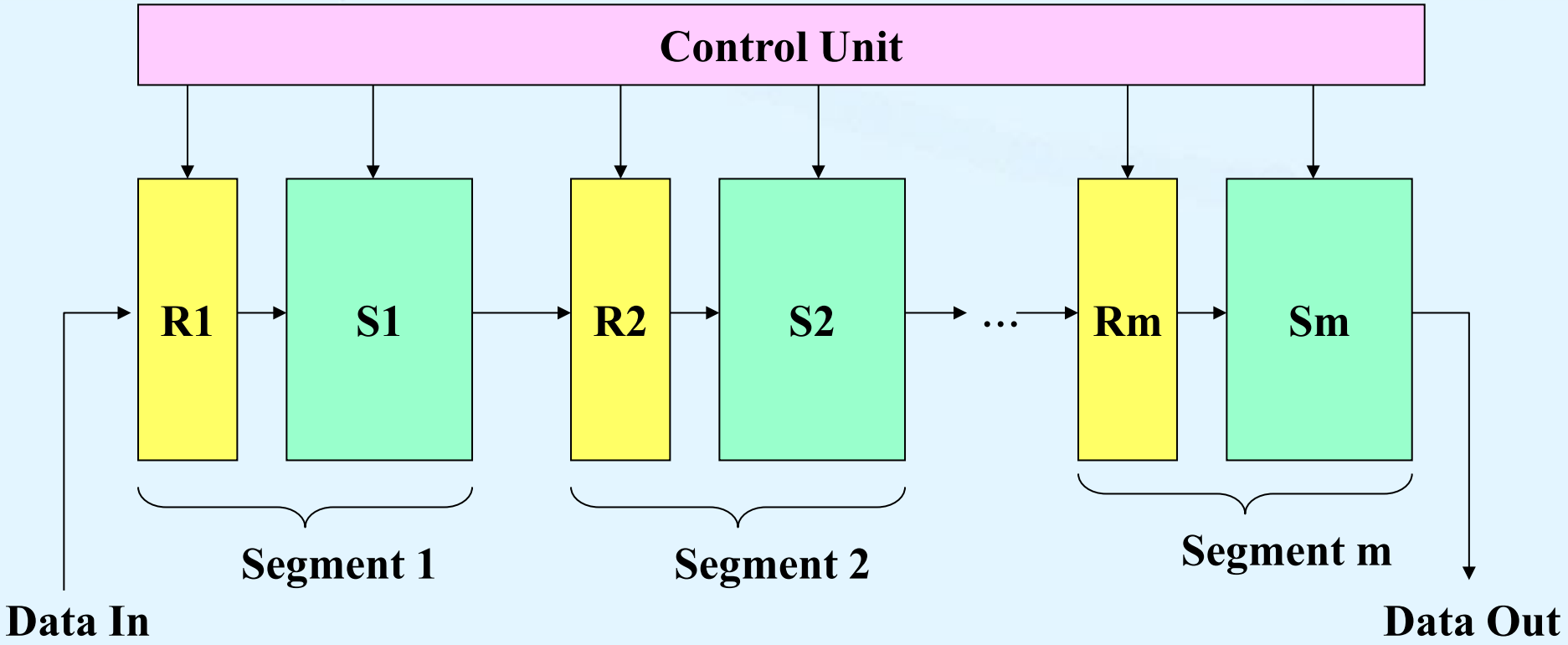
- For **816 loads**:
  - non-pipelined approach takes 3264 units of time.
    - 1632 hours OR **68 days**
  - pipelined approach takes 819 units of time.
    - 409.5 hours OR **17 days**

Instruction Pipeline



- A single instruction cycle process is divided into several small independent tasks (or segments).
- CPU break the instruction cycle into tasks (or segments) that can be performed in parallel
- With the use of pipeline , the average instruction processing time is reduced (i.e. referred to Cycle Per Instruction (CPI))

CPU Instruction Pipeline Structure



1. Instructions pass through all the segments in sequence
2. Each segment consists of a circuit that performs sub-operation
3. Segments are separated by registers, that hold the intermediate results between stages.

Implementation of Instruction Pipeline: Case 1

- Divide the instruction cycle into two stages
  - Instruction fetch (Fetch cycle)
  - Execute instruction (Execution cycle)

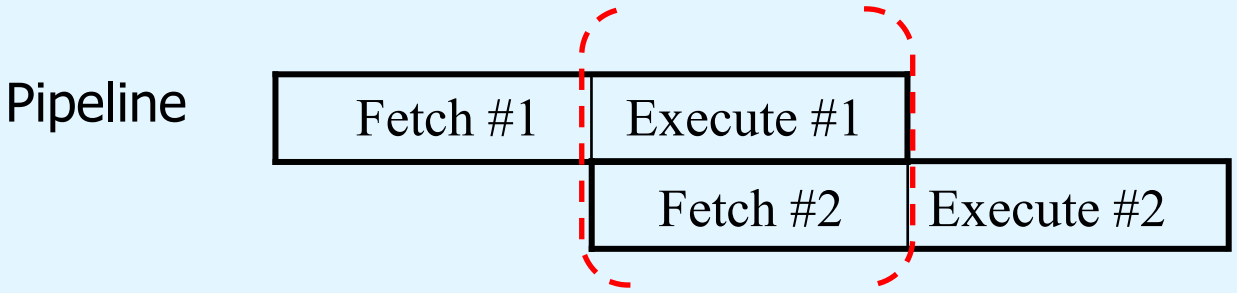
Sequential

Fetch #1	Execute #1	Fetch #2	Execute #2

- Instructions will be executed one-by-one in sequence
- The above instruction cycle (fetch-execute) clearly shows the sequence of operations that take place in order to execute a single instruction.
- A "good" design goal of any system is to have all of its components performing useful work all of the time – high efficiency.
- Following the instruction cycle in a sequential fashion does not permit this level of efficiency.

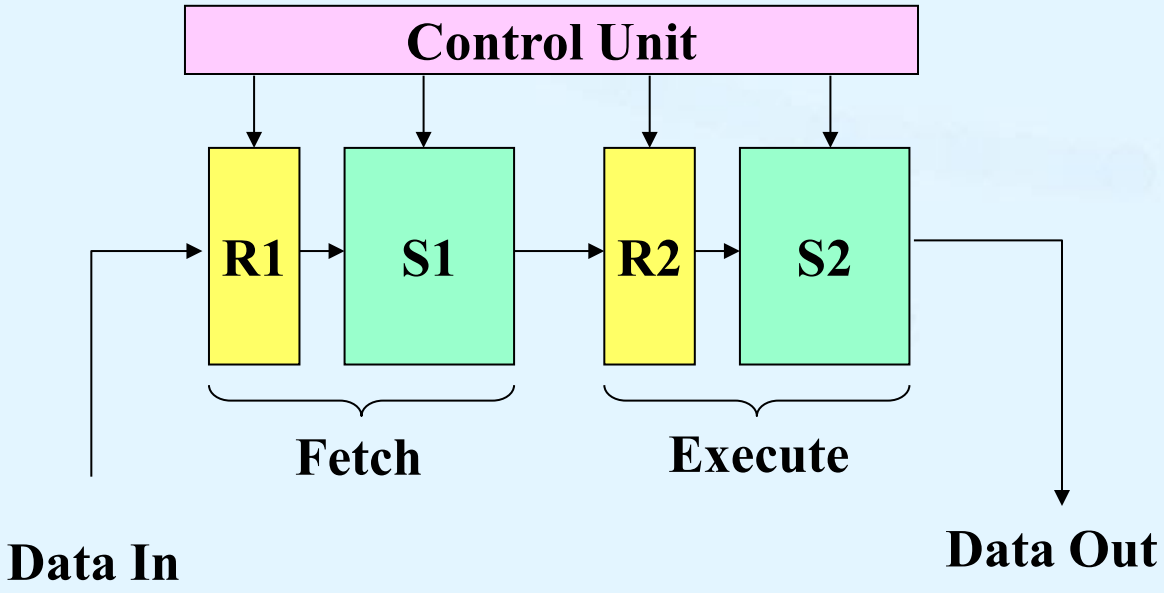
Implementation of Instruction Pipeline: Case 1

- Divide the instruction cycle into two stages
  - Instruction fetch (Fetch cycle)
  - Execute instruction (Execution cycle)

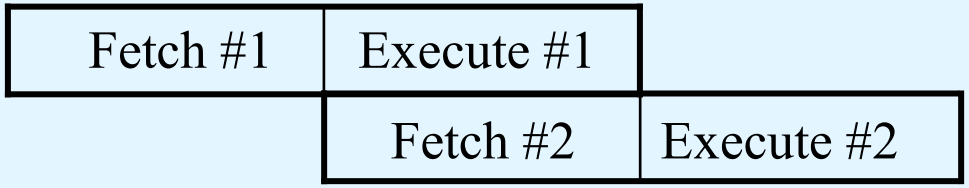


- While one instruction is in "execution", overlap with the 'fetch cycle' for the next instruction (*pre-fetching*)
  - Assumes the memory bus will be idle at some point during the execution phase.
- Reduces the time to fetch an instruction to zero (ideal situation).

Implementation of Instruction Pipeline: Case 1

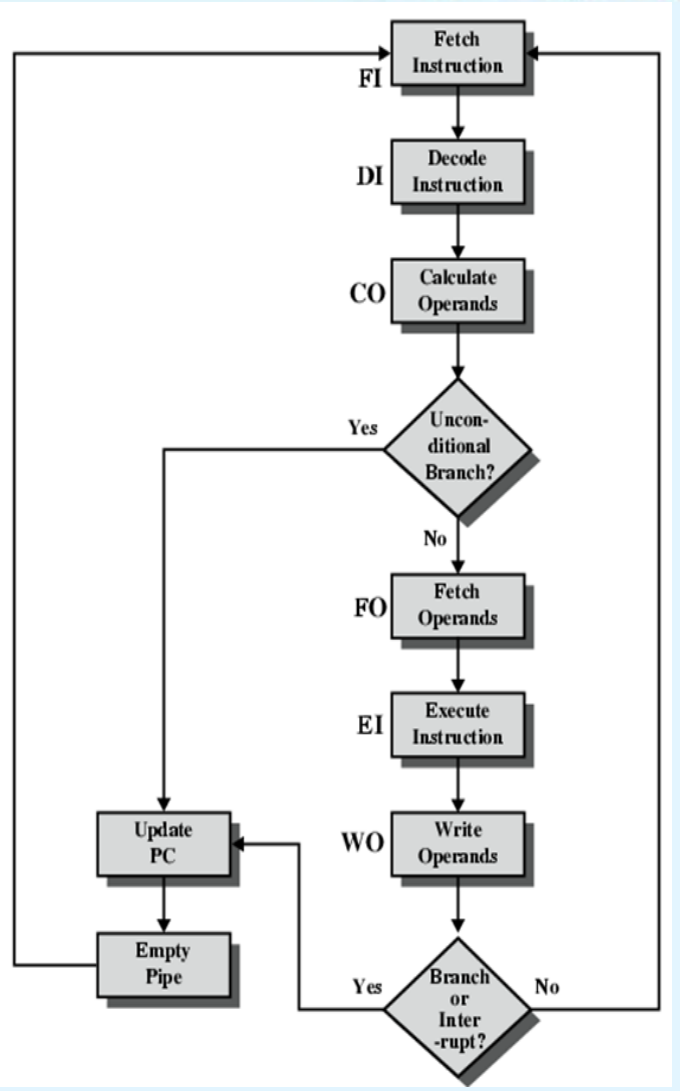


Pipeline



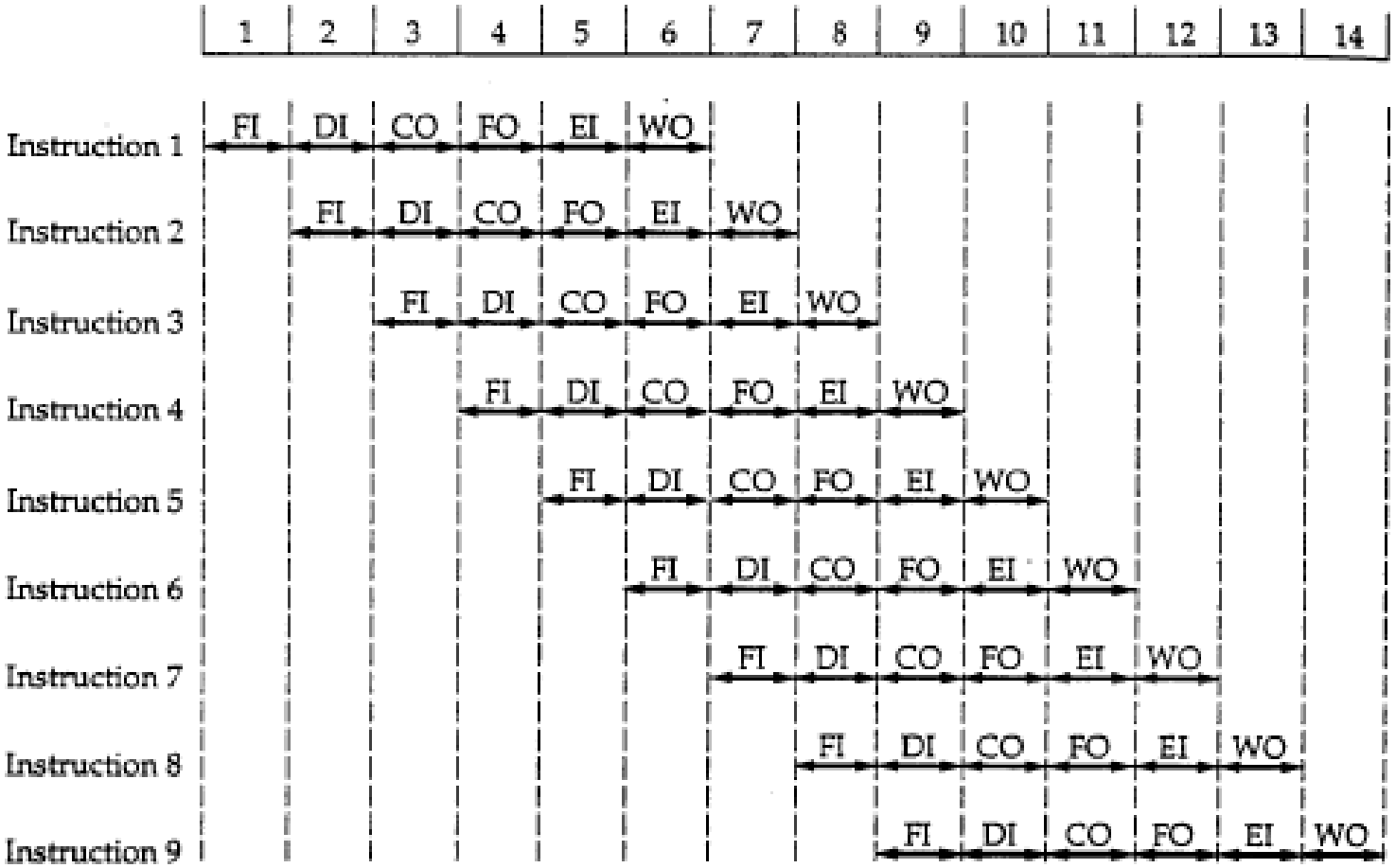
Implementation of Instruction Pipeline: Case 2

- Finer division of the instruction cycle with **more** stages – for better speedup
- Uses a **6-stages** pipeline:
  - Fetch instruction (FI)
  - Decode instruction (DI)
  - Calculate operands (CO)
  - Fetch operands (FO)
  - Execute instruction (EI)
  - Write (store) operand (WO)



Implementation of Instruction Pipeline: Case 2

Timing Diagram for Instruction Pipeline Operation



Pipeline
= 14 time units

vs

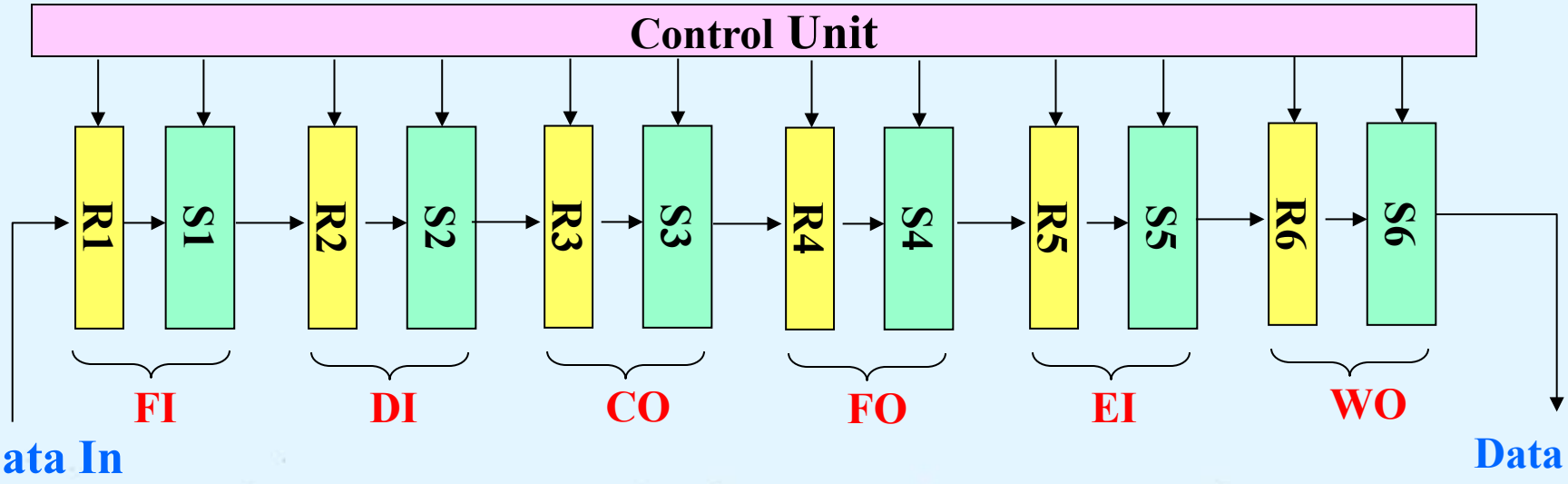
Non pipeline
= (9 x 6)
= 54 time units

Implementation of Instruction Pipeline: Case 2

To implement the instruction cycle with more stages in simpler form:

- Use *multiple functional units OR segments* to execute several instructions in parallel / concurrent

6-stages pipeline:
- Fetch instruction (FI)
- Decode instruction (DI)
- Calculate operands (CO)
- Fetch operands (FO)
- Execute instruction (EI)
- Write (store) operand (WO)



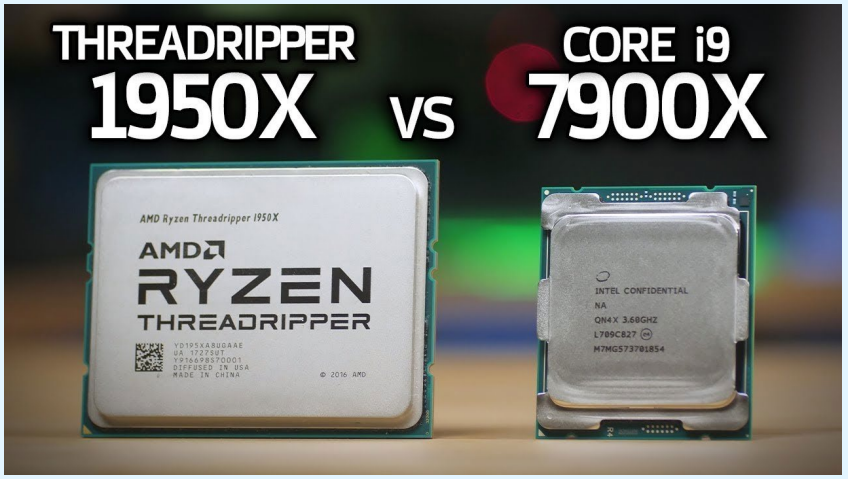
Multiple execution "functional units" in real CPUs

Implementation of instruction cycle with multiple pipeline stages in **real CPU** :

- Use of *multiple execution units (i.e. multiple ALUs)* to concurrently execute several instructions.







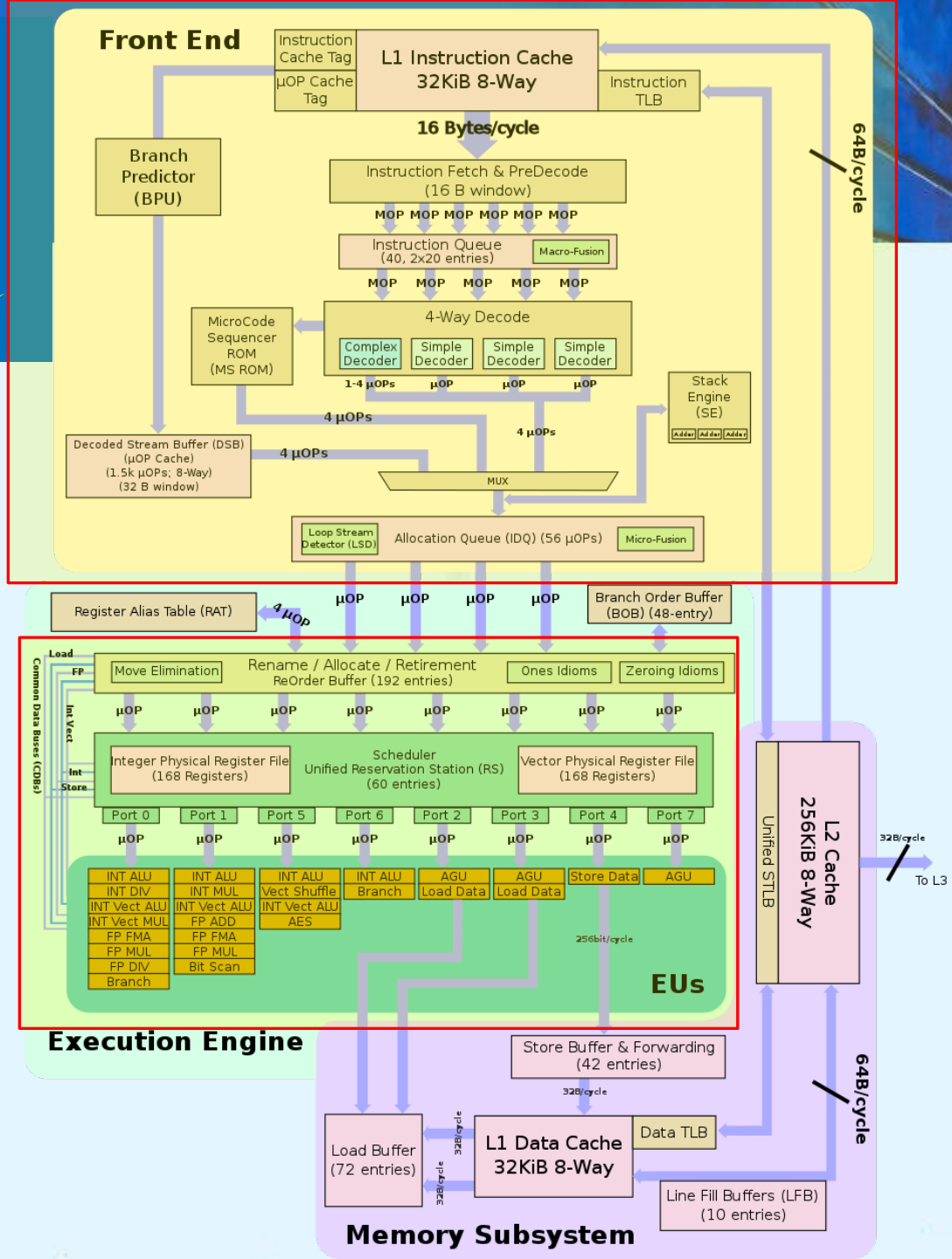
Multiple execution units in real CPUs

	AMD Istanbul	AMD Interlagos	Intel SandyBridge	Intel IvyBridge
System	Kraken (XT5)	Titan (XK7)	Eos (XC30)	XC30
Socket/Node	2	1	2	1 or 2
FPU/Socket	6	8	8	8+
ALU/Socket	6	16	8	8+
Cores/Socket	6	16	8	8+
Threads/Core	1	1	1 or 2	1 or 2
DP FP/FPU	4 /clock	8 /clock	8 /clock	16 /clock

Microarchitecture block diagram for real CPU – Intel Haswell.

Fetch and Decode cycle

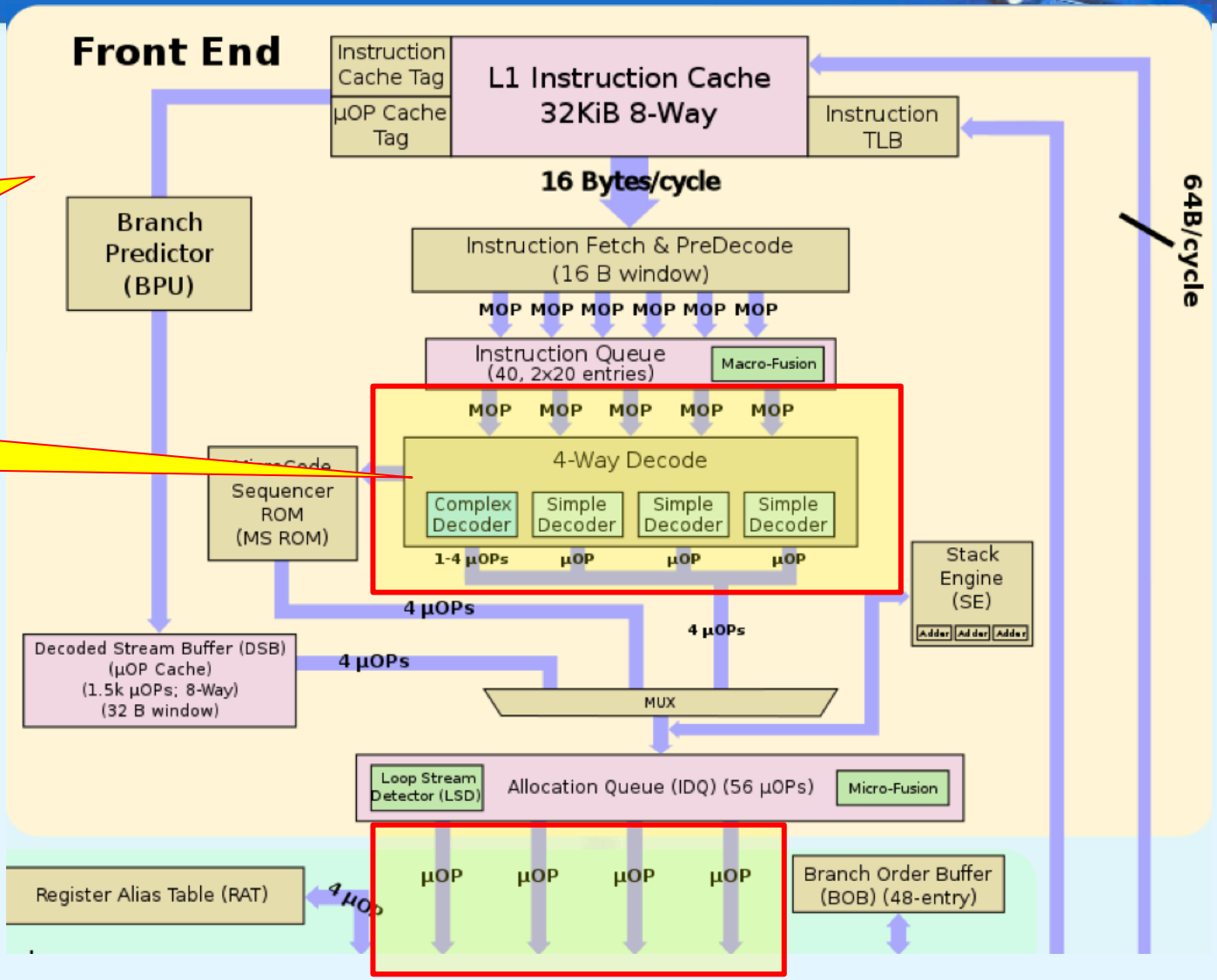
Execute cycle



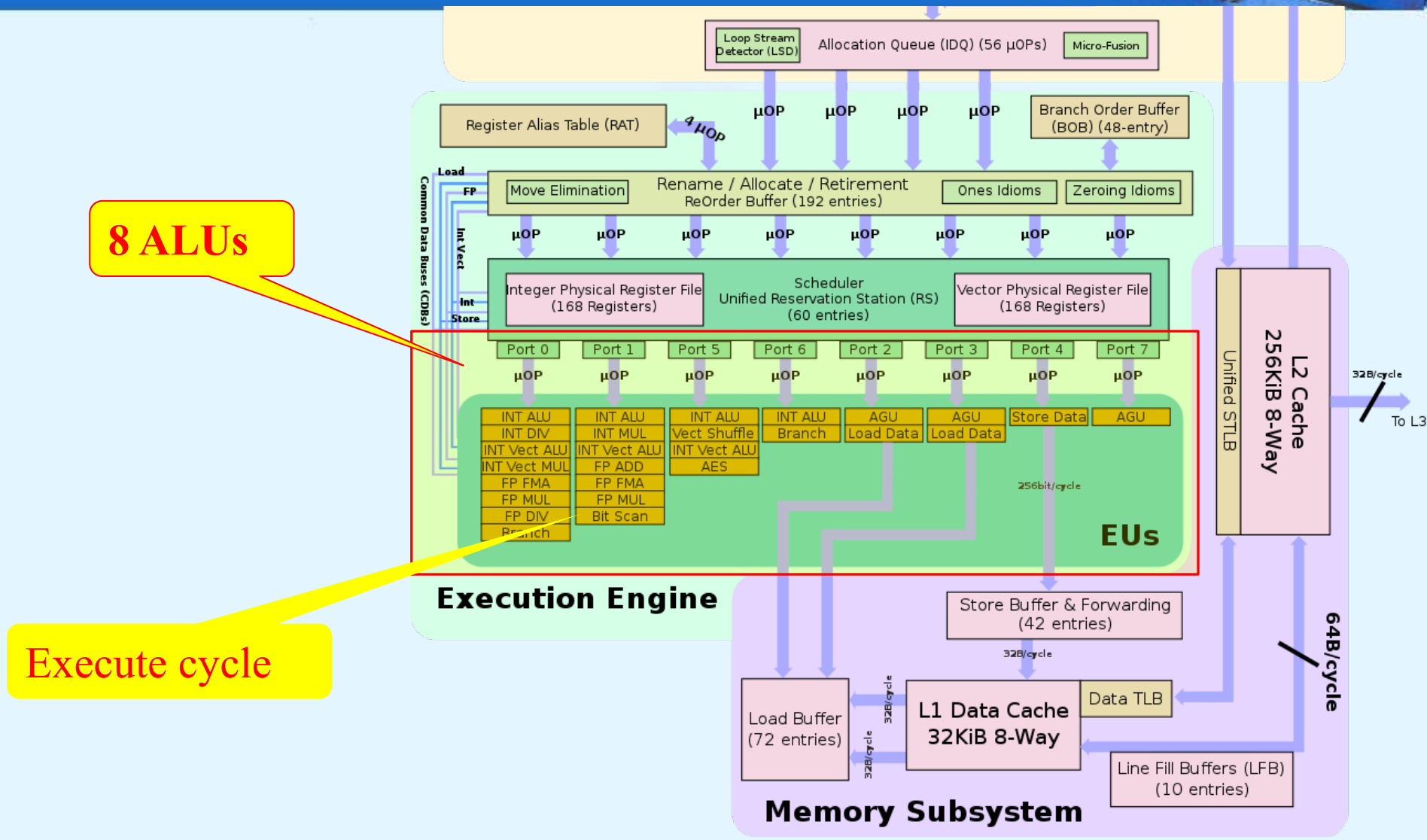
Multiple execution units in real CPUs – Intel Haswell, has **4-way decoder**

Fetch and decode cycle

4-way decoder



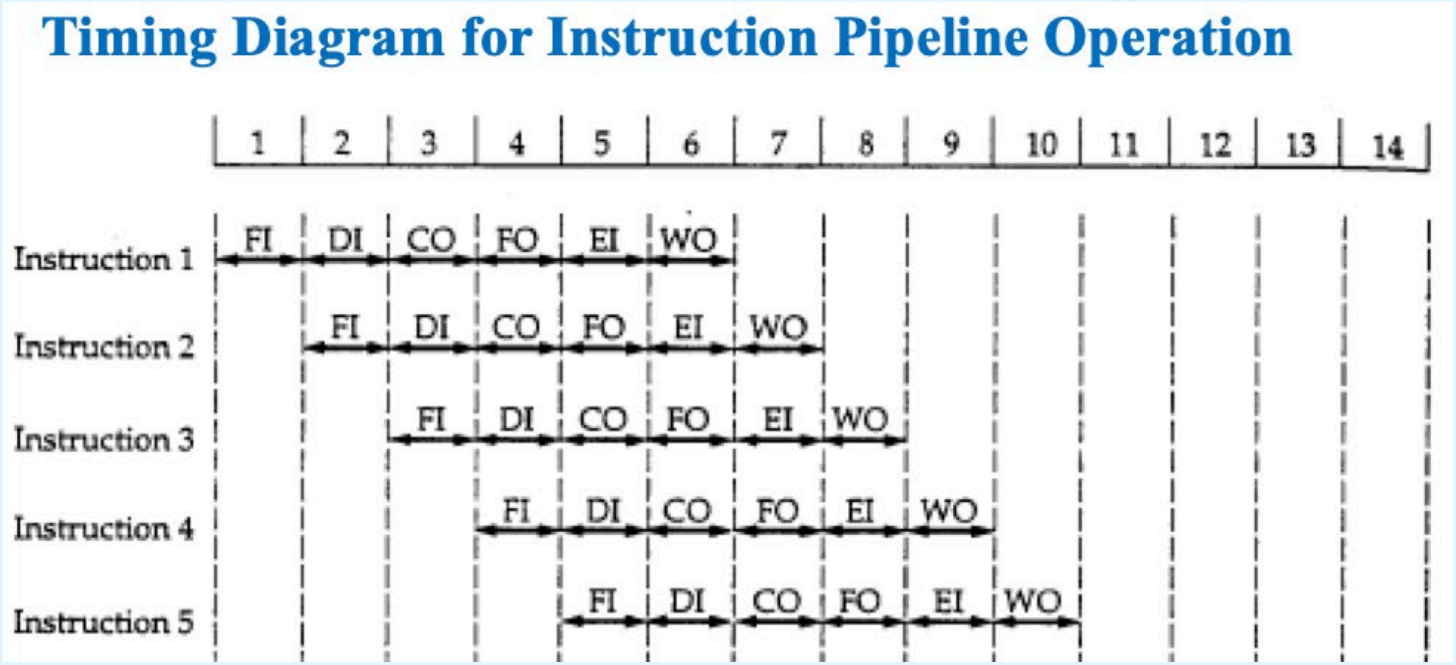
Multiple execution units in real CPUs – Intel Haswell, has **8 ALUs**



Pipeline Performance:

Pipeline performance

- *Cycle time* is the time needed to advance a set of instructions one stage through the pipeline
  - t_n is the *cycle time* for non-pipelining and t_p is the *cycle time* for pipelining
  - each column in timing diagram represents one cycle time



Pipeline performance

- An ideal pipeline divides a task into k independent sequential segments:
  - Each segment requires t_p time unit to complete,
  - The task itself then requires $k \times t_p$ time units to complete.
- For n iterations of the task, the *execution times* will be:
  - With no pipelining: $n \times t_n$ time units,
  - With pipelining: $k \times t_p + (n\text{-}1) \times t_p$ time units.
- *Degree of speedup* is thus:

$$S = \text{Exec. Time}_{\text{non-pipelining}} \div \text{Exec. Time}_{\text{with pipelining}}$$
$$= (n \times t_n) \div [k+(n\text{-}1)] \times t_p$$

- If n (*no. of tasks*) is too large from k, $n >> k$, the theoretical *max. speedup*:

$$S_{\max} = k$$

Example 1: Pipeline performance

Given a 4 segment pipeline whereby each segment has a delay time as follows:

- Segment 1 : 40 ns
- Segment 2 : 25 ns
- Segment 3 : 45 ns
- Segment 4 : 45 ns

The delay time for the interface register is 5 ns.

Calculate the:

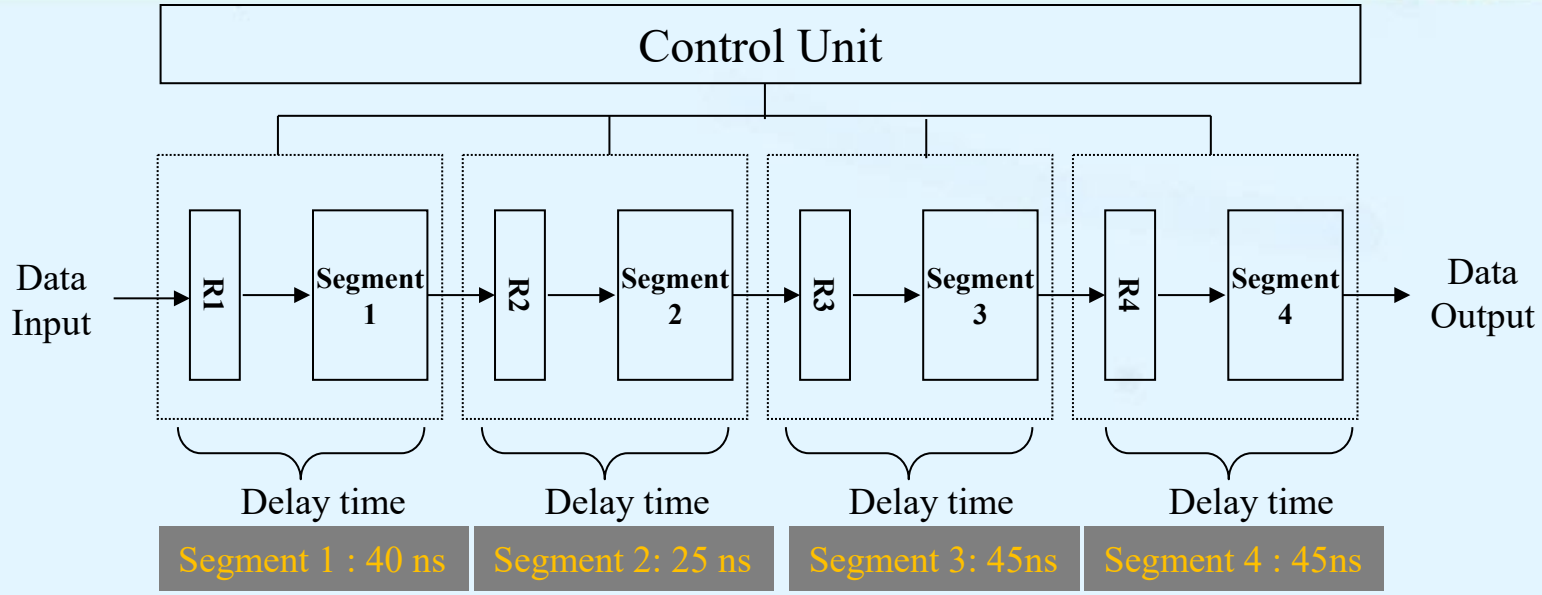
i) cycle time of the non-pipeline and pipeline,

ii) execution time for 100 tasks,

iii) real speedup, and

iv) maximum speedup.

Example 1: Pipeline performance



i. Cycle time:

t_n = (40 + 25 + 45 + 45 + 5) ns = 160 ns

t_p = the longest delay for execution + interface delay

= (45 + 5) ns = 50 ns

Example 1: Pipeline performance

ii. Execution time for **100 tasks**

For pipeline system;

$= k \times t_p + (n\text{-}1)\, t_p$

= ((4 * 50) + (99 * 50)) ns

= 5150 ns

For non-pipeline system;

$= n \times t_n$ = 100 * 160 ns = 16000 ns

iii. The real speedup for 100 task

Speed up = Execution time for non-pipeline ÷ Execution time for pipeline

= 16000 / 5150 = 3.1068

iv. Maximum speedup, $S_{max} = k = 4$

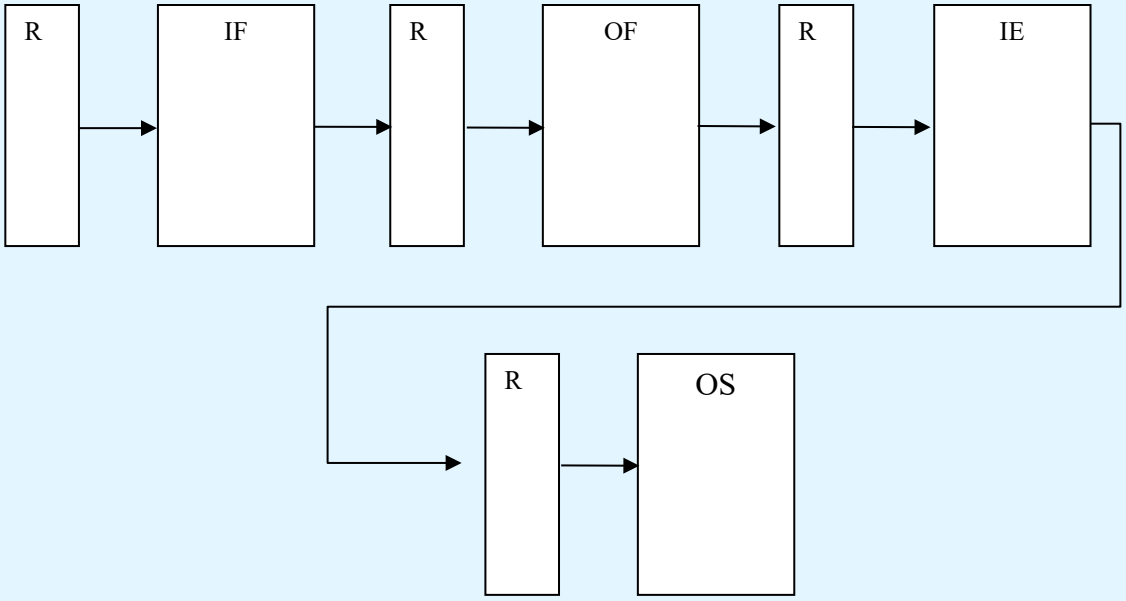
Example 2: Pipeline performance

There are 4 segments as follows:

The interface delay is **3 ns**.

The simple block diagram of the pipeline:

Segment Name	Segment Execution Time (ns)
Instruction Fetch & Decode (IF)	**52**
Operand Fetch (OF)	**40**
Instruction Execute (IE)	**30**
Operand Store (OS)	**40**



Draw the *space time diagram* and *calculate* the:

I. cycle time for the non-pipeline and pipeline,
II. execution time for 50 tasks,
III. real speedup, and
IV. maximum speedup.

Space Time Diagram

- Purpose of diagram – to show the events of a pipeline

	1	2	3	4	5	6	7	→ time	
S1	**P1**	**P2**	**P3**	**P4**					
S2		**P1**	**P2**	**P3**	**P4**				
S3			**P1**	**P2**	**P3**	**P4**			
S4				**P1**	**P2**	**P3**	**P4**		

S = segment; P = process;

Example 2: Pipeline performance

The space time diagram for the pipeline (limit to 5 processes):

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
IF	P1	P2	P3	P4	P5					
OF		P1	P2	P3	P4	P5				
IE			P1	P2	P3	P4	P5			
OS				P1	P2	P3	P4	P5		

i. Cycle time:

t_n = (52 + 40+ 30 + 40 + 3) ns = <u>165 ns</u>

t_p = the longest delay for execution + interface delay
= (52+ 3) ns = <u>55 ns</u>

ii. Execution time
For pipeline system, execution time for 50 tasks
= $k \times t_p + (n\text{-}1)\, t_p$
= ((4 * 55) + (49 * 55)) ns
= 220+2695ns = <u>2915 ns</u>

For non-pipeline system, the total execution time for 50 tasks
= $n \times t_n$ = 50 * 165ns = <u>8250 ns</u>

iii. The real speedup for 50 task
Speed up = Execution time for non-pipeline / Execution time for pipeline
= 8250/ 2915
= <u>2.83</u>

iv. Maximum speedup, $S_{max} = k$ = <u>4</u>

Segment Name	Segment Execution Time (ns)
Instruction Fetch & Decode (IF)	52
Operand Fetch (OF)	40
Instruction Execute (IE)	30
Operand Store (OS)	40

Pipeline Limitations:

Pipeline Limitations

- 3 major issues that affect pipeline performance:

1. *Resource conflict* (or resource hazard or structural hazard)

2. *Data dependency* (or data hazard)

3. *Branch difficulties* (or control hazard)

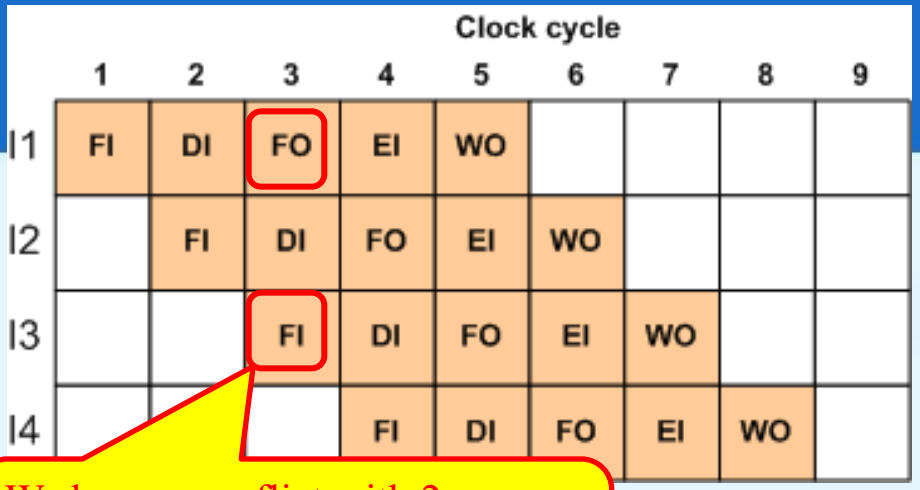
Pipeline Limitations:

Resource Conflict

Pipeline Limitation: 1. Resource Conflict

- Occurs when two segments need to access memory at the same time
- Can be solved by implementing modular memory.

Pipeline Limitation: 1. Resource Conflict



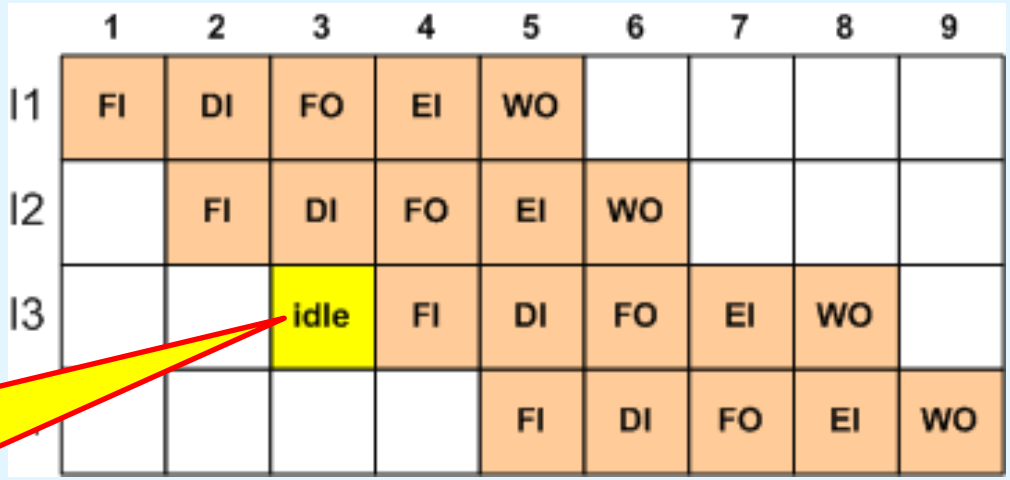
We have a conflict with 2 instruction need the same resource (i.e. memory) at T_3

5-staged pipeline, ideal case

Assume that memory has a single port ➔ so data reads and writes can only happen 1 at a time.

Assume that the source operand for I1 is in memory.

Solution



The FI stage for I3 must be *delayed* OR **idle** for 1 clock cycle before beginning

Pipeline Limitation: 1. Resource Conflict

- Conflict of resources introduce '*pipeline delay*'
- A delay in any stage can cause pipeline stalls (pipeline stop working),

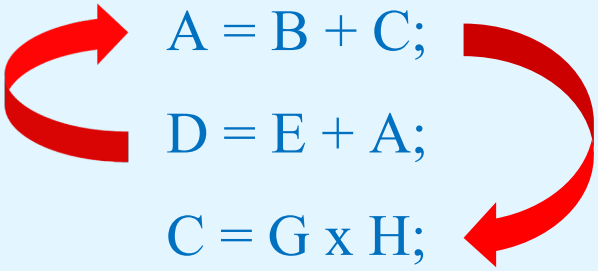
	1	2	3	4	5	6	7	8	9
I1	FI	DI	FO	EI	WO				
I2		FI	DI	FO	EI	WO			
I3			idle	FI	DI	FO	EI	WO	
I4					FI	DI	FO	EI	WO

Pipeline Limitations:

Data Dependency

Pipeline Limitation: 2. Data dependency

- Conflict arises when an instruction depends on the result of previous instruction, but this result is not yet available
- Pipelining must ensure that computed results are the same as if computation was performed in strict sequential order.
- With multiple stages pipelining, two instructions in the pipeline may have data dependencies
- Data dependency is shown in the following portion of a program:



Solution to Data Dependency

- Hardware interlocks
  - An interlock is a circuit that detects instructions with data dependency and inserts required delays to resolve conflicts
- Operand forwarding
  - Allowing the result of ALU to be used by other ALU operations in the next instruction cycle.
- Delayed load (NOP)
  - Compiler is used to detect conflict and reorder instructions to delay loading of conflicting data.
  - Using NOP instruction.

Example 4: Data Dependency

I1: SUB AX, BX ; (AX) ← (AX) – (BX)

I2: ADD AX, CX ; (AX) ← (AX) + (CX)

	1	2	3	4	5	6	7	
Fetch Instruction	I1	I2	I3	I4				
Decode Instruction		I1	I2	I3	I4			
Execute Instruction			I1	I2	I3	I4		
Store Result				I1	I2	I3	I4	

Example 4: Solution – Delay Load (NOP)

I1: SUB AX, BX ; (AX) ← (AX) – (BX)

NOP

I2: ADD AX, CX ; (AX) ← (AX) + (CX)

	1	2	3	4	5	6	7	
Fetch Instruction	**I1**	**NOP**	**I2**	**I3**	**I4**			
Decode Instruction		**I1**	**NOP**	**I2**	**I3**	**I4**		
Execute Instruction			**I1**	**NOP**	**I2**	**I3**	**I4**	
Store Result				**I1**	**NOP**	**I2**	**I3**	**I4**

Example 5: Data Dependency:

I1: ADD AH, BH
I2: SUB AH, 2
I3: CMP AH, 0

Clock	1	2	3	4	5	6	7	8	9	10
IF	I1	I2	I3							
OF		I1	I2	I3						
IE			I1	I2	I3					
OS				I1	I2	I3				

Example 5: Solution to Data Dependency: Delay Load (NOP)

I1: ADD AH, BH
I2: SUB AH, 2
I3: CMP AH, 0

Clock	1	2	3	4	5	6	7
IF	I1	I2	I3				
OF		I1	I2	I3			
IE			I1	I2	I3		
OS				I1	I2	I3	

Clock	1	2	3	4	5	6	7	8	9	10
IF	I1	I2	I3							
OF		I1	I2	I3						
IE			I1	I2	I3					
OS				I1	I2	I3				

Data dependency
Both are about to access the same register **AH**.

Example 5: Solution to Data Dependency: Delay Load (NOP)

I1: ADD AH, BH
I2: SUB AH, 2
I3: CMP AH, 0

Clock	1	2	3	4	5	6	7	8	9	10
IF	I1	I2	I3							
OF		I1	I2	I3						
IE			I1	I2	I3					
OS				I1	I2	I3				

Clock	1	2	3	4	5	6	7	8	9	10
IF	I1	I2	I3							
OF		I1	I2	I3						
IE			I1	I2	I3					
OS				I1	I2	I3				

I1: ADD AH, BH
NOP
I2: SUB AH, 2
NOP
I3: CMP AH, 0

Using NOP to solve the problem – Delayed Load.

Clock	1	2	3	4	5	6	7	8	9	10
IF	I1	NOP	I2	NOP	I3					
OF		I1	NOP	I2	NOP	I3				
IE			I1	NOP	I2	NOP	I3			
OS				I1	NOP	I2	NOP	I3		

Pipeline Limitations:

Branching

Pipeline Limitation: 3. Branching

- Problems arise from **branch** instructions that change the PC value
- i.e. instructions that change the address of subsequence instructions

Pipeline Limitation: 3. Branching

- For the pipeline to have the desired operational speedup, we must "feed it" with long strings of instructions.
- However, 15-20% of instructions in an assembly-level code are (conditional) branches.
- Of these, 60-70% take the branch to a target address.
- Impact of the branch is that pipeline never really operates at its full capacity – limiting the performance improvement that is derived from the pipeline

Branching: Illustration

Time →

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO	EI	WO					
Instruction 5					FI	DI	CO	FO	EI	WO				
Instruction 6						FI	DI	CO	FO	EI	WO			
Instruction 7							FI	DI	CO	FO	EI	WO		
Instruction 8								FI	DI	CO	FO	EI	WO	
Instruction 9									FI	DI	CO	FO	EI	WO

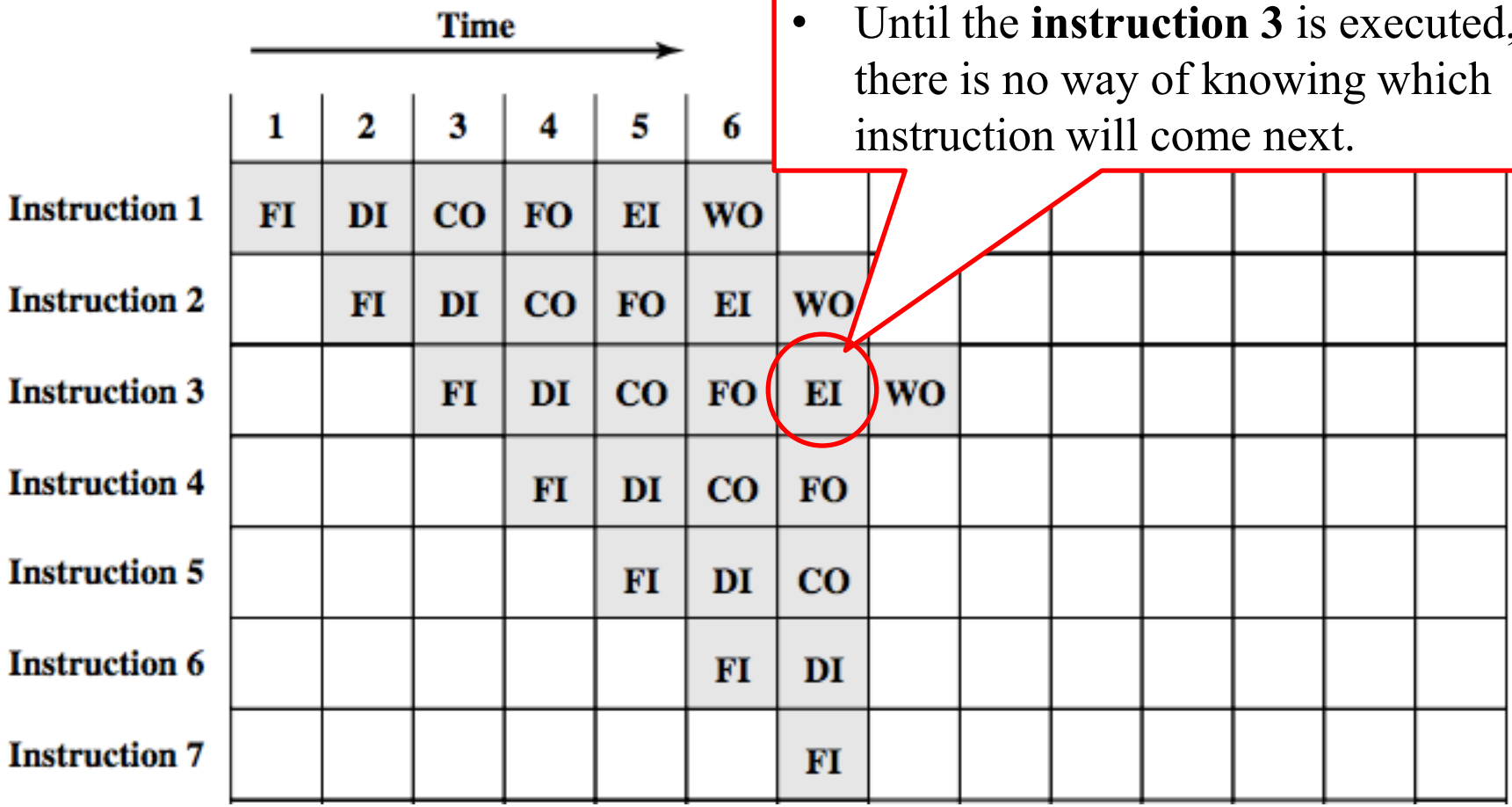
Branching: Illustration

Assume that **instruction 3** is a **conditional branch** to **instruction 15**.

Time →

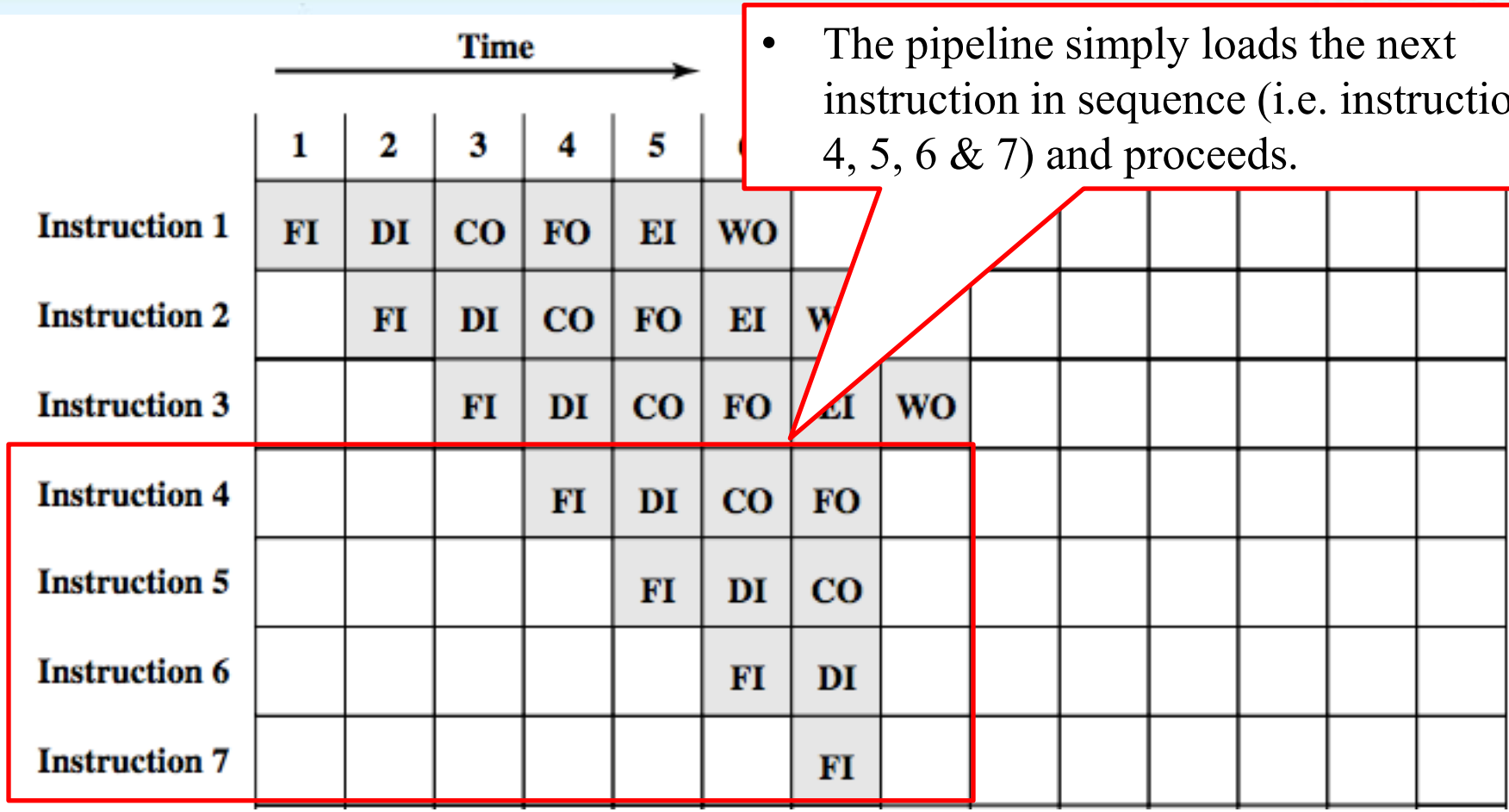
	1	2	3	4	5	6	7	8					
Instruction 1	FI	DI	CO	FO	EI	WO							
Instruction 2		FI	DI	CO	FO	EI	WO						
Instruction 3			FI	DI	CO	FO	EI	WO					
Instruction 4				FI	DI	CO	FO	EI	WO				
Instruction 5					FI	DI	CO	FO	EI	WO			
Instruction 6						FI	DI	CO	FO	EI	WO		
Instruction 7							FI	DI	CO	FO	EI	WO	
Instruction 8								FI	DI	CO	FO	EI	WO
Instruction 9									FI	DI	CO	FO	EI	WO

Branching: Illustration



- Until the **instruction 3** is executed, there is no way of knowing which instruction will come next.

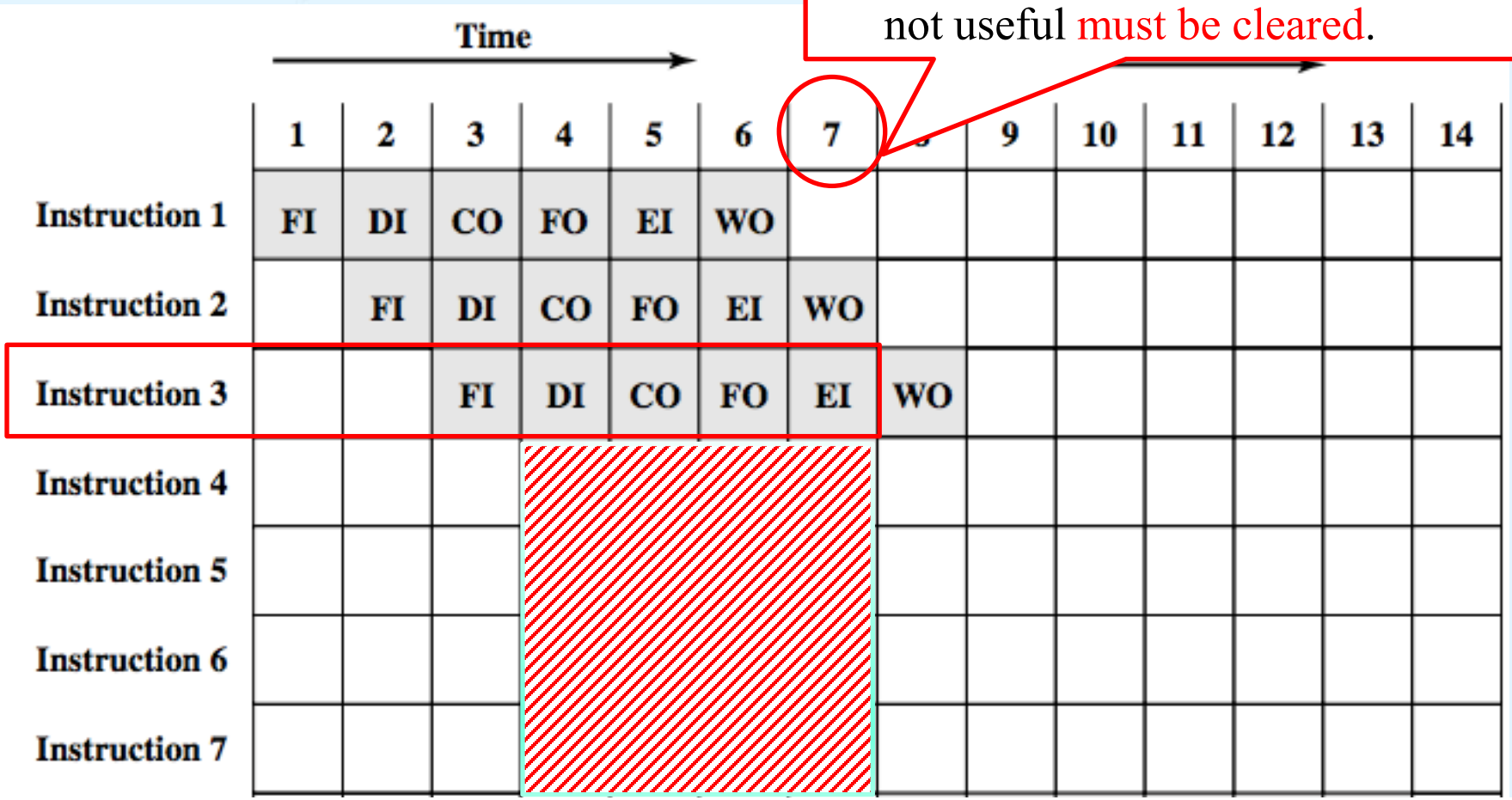
Branching: Illustration



- The pipeline simply loads the next instruction in sequence (i.e. instruction 4, 5, 6 & 7) and proceeds.

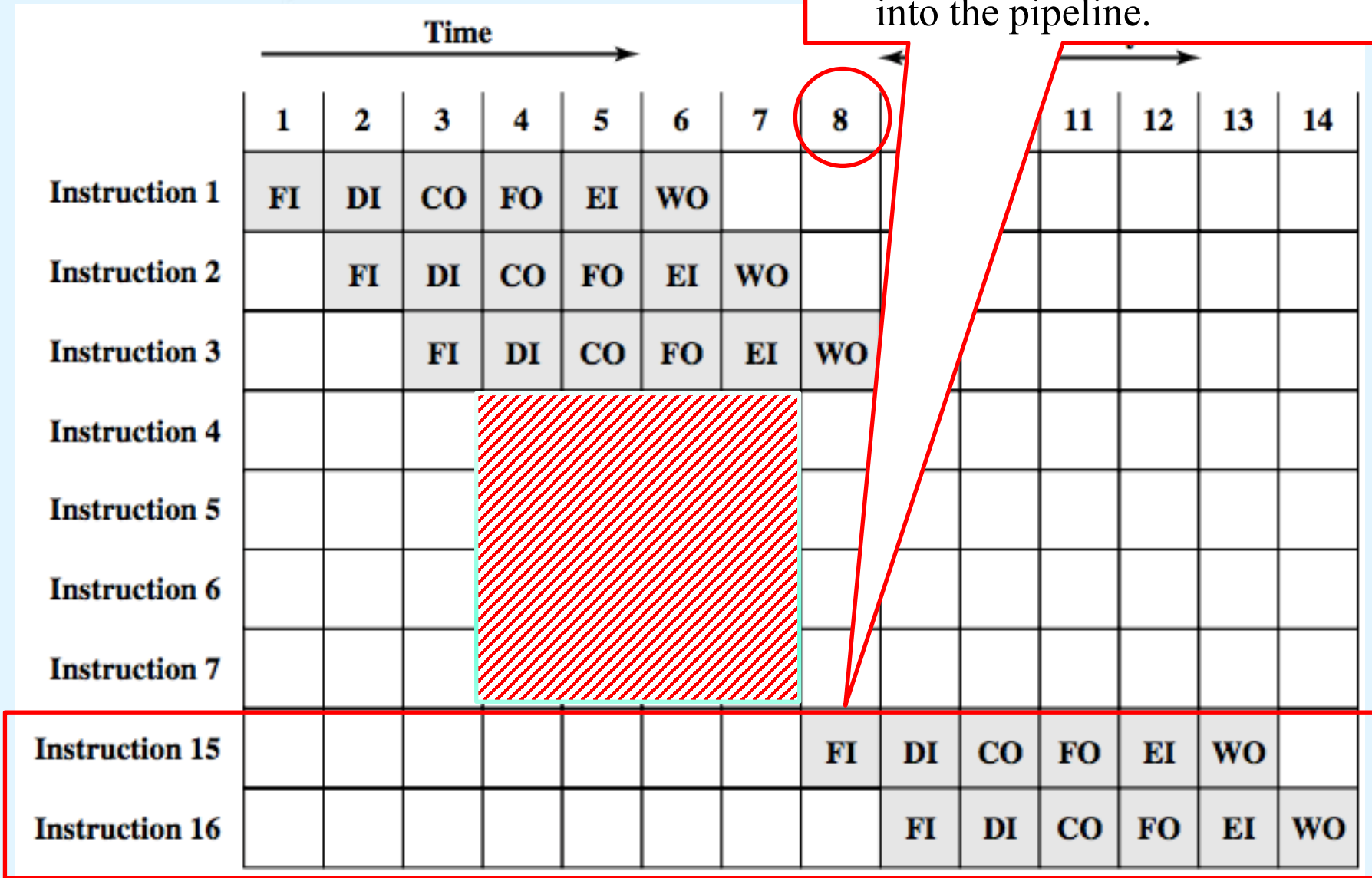
Branching: I

- until the end of time unit 7, the instructions in the pipeline that are not useful must be cleared.



Branching: Illu

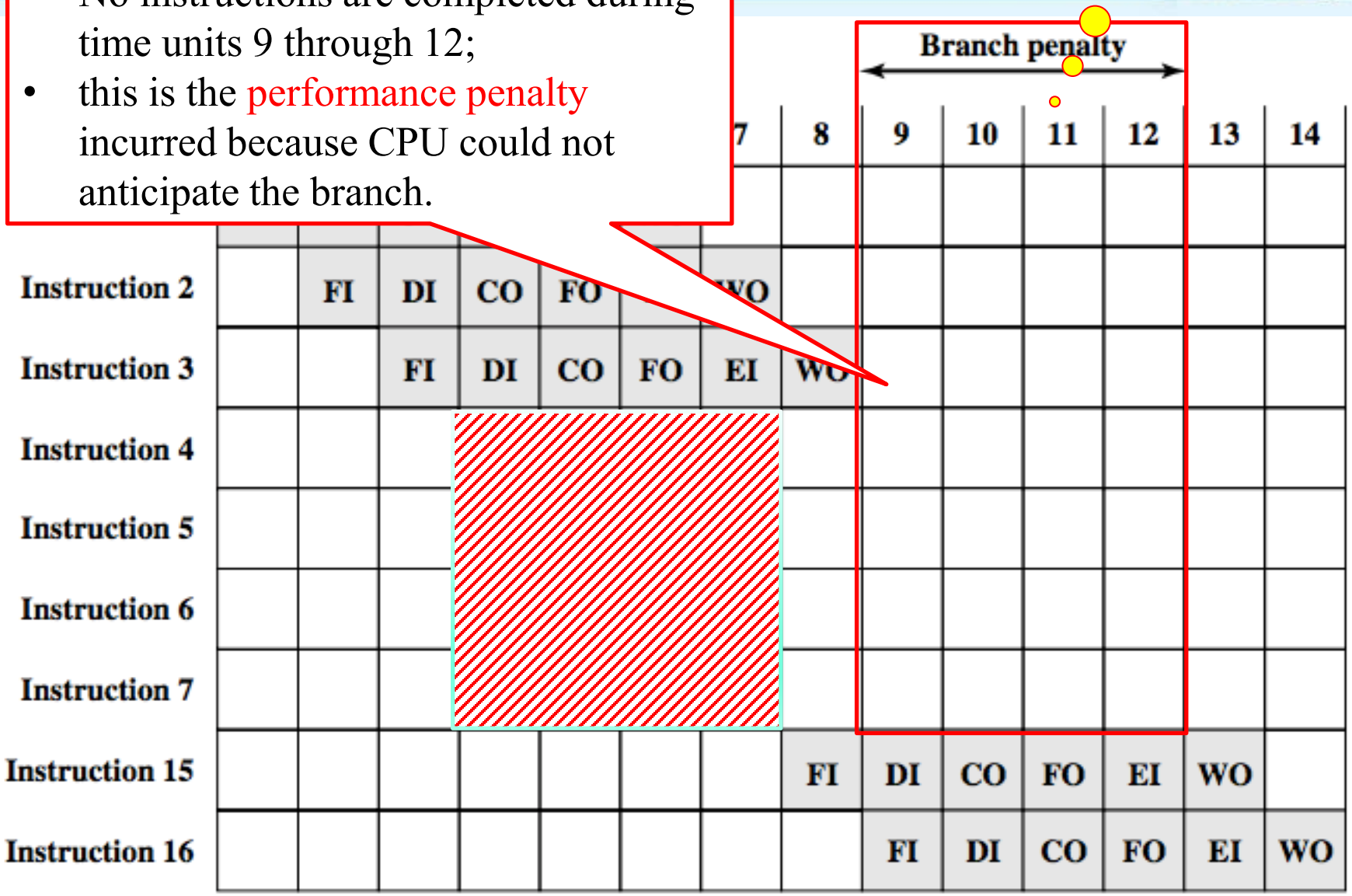
- Then at time unit 8, instruction 15 and instruction 16 will be fed into the pipeline.



Branching: Illustra

4 clock cycles are wasted without full execution of instructions 4,5,6 & 7

- No instructions are completed during time units 9 through 12;
- this is the performance penalty incurred because CPU could not anticipate the branch.



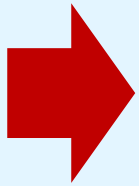
Solution for Branching

1. Delayed branch (use of NOP instruction)
2. Re-arranging the instructions

Example 6: Solution for Branching: Delay Branch – Using NOP

- When the compiler detect a branch, it will automatically insert several NOP so that there is no interruptions in the pipeline.
- Example:

```
        Mov          Mov
        Inc          Inc
        Add          Add
        Jmp L1       Jmp L1
        I_X          NOP
        I_X+1        NOP
        I_X+2        NOP
L1:     Sub     L1:  Sub
```



How many **NOP** to insert
= No segment – 1
= k – 1
= 4 – 1 = 3

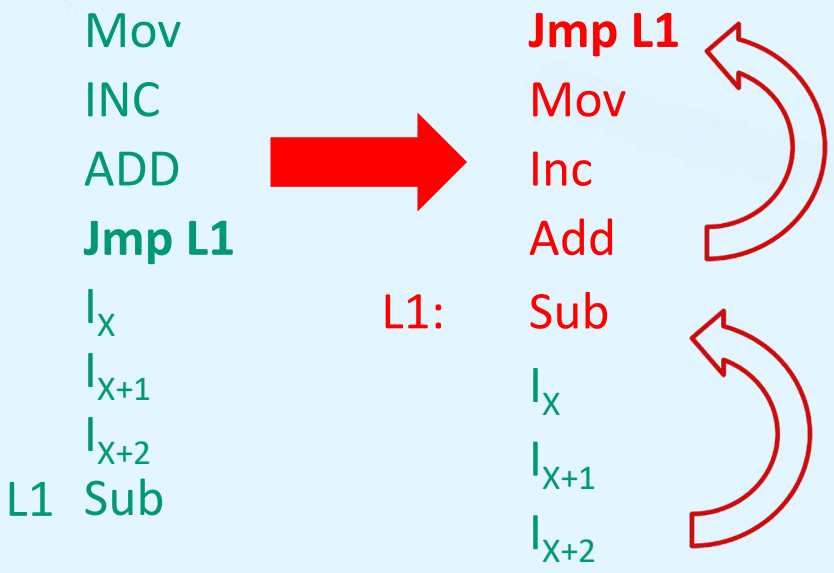
Example 6: Solution for Branching: Delay Branch – Using NOP

- Illustrate with space-time diagram:

Fetch Instruction	Mov	Inc	Add	Jmp	**NOP**	**NOP**	**NOP**	Sub	I_X	I_{X+1}	I_{X+2}
Decode Instruction		Mov	Inc	Add	Jmp	**NOP**	**NOP**	**NOP**	Sub	I_X	I_{X+1}
Execute Instruction			Mov	Inc	Add	Jmp	**NOP**	**NOP**	**NOP**	Sub	I_X
Store Result				Mov	Inc	Add	Jmp	**NOP**	**NOP**	**NOP**	Sub

- Wasted 3 pipeline clock cycles

Example 7: Solution for Branching: Rearranging Instructions



How to arrange, **moving instruction up**
= No segment – 1
= k – 1
= 4 – 1 = 3
(move up 3)

Fetch Instruction	Jmp	Mov	Inc	Add	Sub	I_X	I_{X+1}	I_{X+2}	
Decode Instruction		Jmp	Mov	Inc	Add	Sub	I_X	I_{X+1}	I_{X+2}
Execute Instruction			Jmp	Mov	Inc	Add	Sub	I_X	I_{X+1}
Store Result				Jmp	Mov	Inc	Add	Sub	I_X

Example 8: Solution for Branching

```
I1              MOV AX,BX
I2              ADD BX,CX
I3              INC CX
I4              JMP LOSSY
I5              MOV DX,1
I6              INC BX
:
I11     LOSSY:  DEC AX
```

Example 8: Solution for Branching

```
I1              MOV AX,BX
I2              ADD BX,CX
I3              INC CX
I4              JMP LOSSY
I5              MOV DX,1
I6              INC BX
:
I11     LOSSY:  DEC AX
```

How many delay NOP to insert
= No segment – 1
= k – 1
= 4 – 1 = 3

, i.e. will create 3 *NOP* instruction cycle before targeted label

Example 8: Solution for Branching

```
I1              MOV AX,BX
I2              ADD BX,CX
I3              INC CX
I4              JMP LOSSY
I5              MOV DX,1
I6              INC BX
:
I11   LOSSY:    DEC AX
```

How many delay branch NOP to insert
= No segment – 1
= k – 1
= 4 – 1 = 3

, i.e. will create 3 *NOP* instruction cycle before targeted label

	1	2	3	4	5	6	7	8	9	10	11
Fetch Inst.(FI)	I1	I2	I3	**I4**	I5	I6		**I11**			
Decode Inst.(DI)		I1	I2	I3	**I4**	I5			**I11**		
Execute Instr.(EI)			I1	I2	I3	**I4**				**I11**	
Store Result				I1	I2	I3	**I4**				**I11**

Example 8: Solution for Branching: Delay Branch (using NOP)

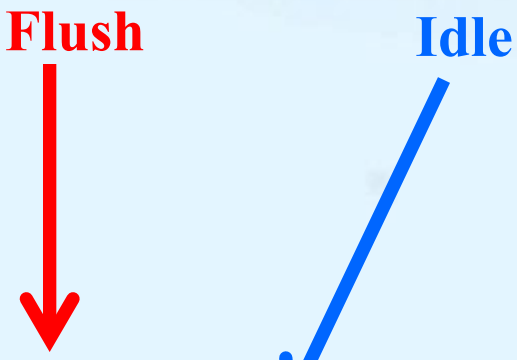
```
I1              MOV AX,BX
I2              ADD BX,CX
I3              INC CX
I4              JMP LOSSY
I5              MOV DX,1
I6              INC BX
:
I11     LOSSY:  DEC AX
```



	1	2	3	4	5	6	7	8	9	10	11
Fetch Inst.(FI)	I1	I2	I3	I4	I5	I6		I11			
Decode Inst.(DI)		I1	I2	I3	I4	I5	I6		I11		
Execute Instr.(EI)			I1	I2	I3	I4	I5	I6		I11	
Store Result				I1	I2	I3	I4	I5	I6		I11

Example 8: Solution for Branching: Delay Branch (using NOP)

```
I1              MOV AX,BX
I2              ADD BX,CX
I3              INC CX
I4              JMP LOSSY
I5              MOV DX,1
I6              INC BX
:
I11     LOSSY:  DEC AX
```



	1	2	3	4	5	6	7	8	9	10	11
Fetch Inst.(FI)	I1	I2	I3	I4	I5	I6		I11			
Decode Inst.(DI)		I1	I2	I3	I4	I5	I6		I11		
Execute Instr.(EI)			I1	I2	I3	I4	I5	I6		I11	
Store Result				I1	I2	I3	I4	I5	I6		I11

Example 8: Solution for Branching: Delay Branch (using NOP)

```
I1          MOV AX,BX
I2          ADD BX,CX
I3          INC CX
I4          JMP LOSSY
I5          MOV DX,1
I6          INC BX
:
I11   LOSSY:  DEC AX
```

Using Delay Branch (NOP):

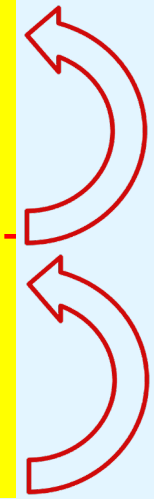
→ **the problem of branching can be solved**

→ **3 wasted pipeline cycles**

	1	2	3	4	5	6	7	8	9	10	11
Fetch Inst.(FI)	I1	I2	I3	I4	NOP	NOP	NOP	I11			
Decode Inst.(DI)		I1	I2	I3	I4	NOP	NOP	NOP	I11		
Execute Instr.(EI)			I1	I2	I3	I4	NOP	NOP	NOP	I11	
Store Result				I1	I2	I3	I4	NOP	NOP	NOP	I11

Example 8: Solution for Branching: Rearranging Instructions

```
I4              JMP LOSSY
I1              MOV AX,BX
I2              ADD BX,CX
I3              INC CX
- - - - - - - - - - - - - - -
I5              MOV DX,1
I6              INC BX
:
I11     LOSSY:  DEC AX
```



How to arrange, move up 3 times
= No segment – 1
= k – 1
= 4 – 1 = 3

	1	2	3	4	5	6	7	8	9	10	11
Fetch Inst.(FI)	I4	I1	I2	I3	I11						
Decode Inst.(DI)		I4	I1	I2	I3	I11					
Execute Instr.(EI)			I4	I1	I2	I3	I11				
Store Result				I4	I1	I2	I3	I11			

Summary

- Instruction pipelining is a powerful technique for enhancing performance but requires careful design to achieve optimum results with reasonable complexity.
- Discussed the principle behind instruction pipelining and how it works in practice.
- Discussed the 3 forms of pipeline limitations:
  - resource conflict,
  - data dependency,
  - branch difficulty.



** Finish **

5b - Pipeline

Module 5c:
The Control Unit



CONTROL UNIT OPERATION
MICROPROGRAMMED CONTROL

REFERENCES:
STALLINGS – COMPUTER ORGABIZATION AND ARCHITECTURE
MANO - COMPUTER ORGABIZATION AND ARCHITECTURE

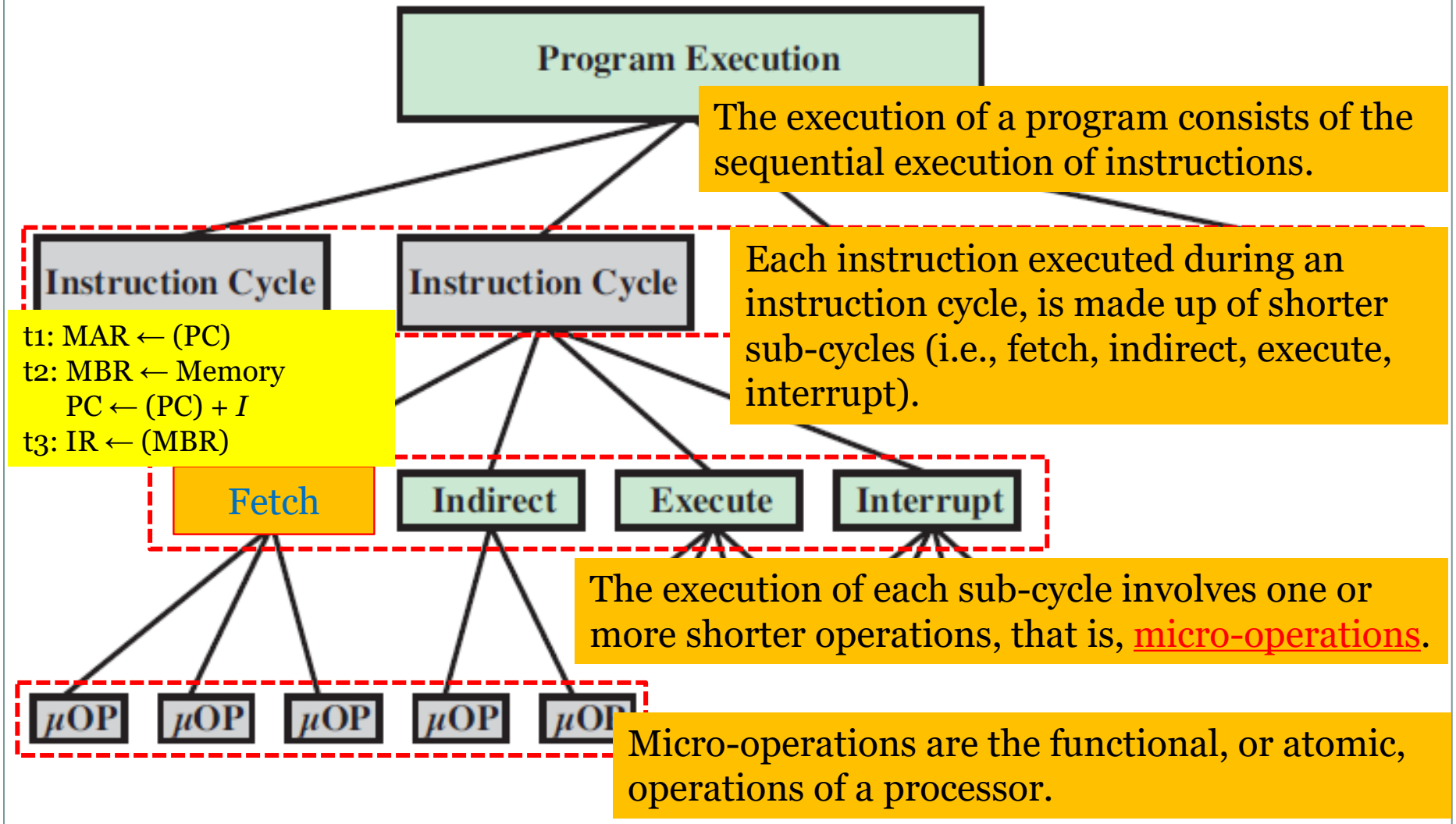
Micro-operations

MICRO-OPERATIONS

- The operation of a computer, in executing a program, consists of a sequence of instruction cycles, with one machine instruction per cycle.
- Each instruction cycle is made up of a number of smaller units.
  - One subdivision that we found convenient is *fetch*, *indirect*, *execute*, *and interrupt*, with only fetch and execute cycles always occurring.
- Each of the smaller cycles involves a series of steps, each of which involves the processor registers.

Micro-Operation

MICRO-OPERATIONS



The execution of a program consists of the sequential execution of instructions.

Each instruction executed during an instruction cycle, is made up of shorter sub-cycles (i.e., fetch, indirect, execute, interrupt).

The execution of each sub-cycle involves one or more shorter operations, that is, micro-operations.

Micro-operations are the functional, or atomic, operations of a processor.

Control of the Processor

- The basis for the design and implementation of the control unit are based on the definition of a functional requirements.
- Three-step process leads to define these requirements:

1) Define the **basic elements** of the processor.
2) Describe the **micro-operations** that the processor performs.
3) Determine the **functions** that the **control unit** must perform.

1. Define the basic elements of the processor

- The basic functional elements of the processor are the following:
  - ALU – the functional essence of the computer.
  - Registers - used to store data (also status) internal to the processor
  - Internal data paths - used to move data between registers and between register and ALU
  - External data paths - External data paths *link* registers to memory and I/O modules, often by means of a system bus.
  - Control unit – causes operations to happen within the processor.

2. Describe the micro-operations

- All micro-operations fall into one of the following categories:
  - Transfer data from one register to another.
  - Transfer data from a register to an external interface (e.g., system bus).
  - Transfer data from an external interface to a register.
  - Perform an arithmetic or logic operation, using registers for input and output.

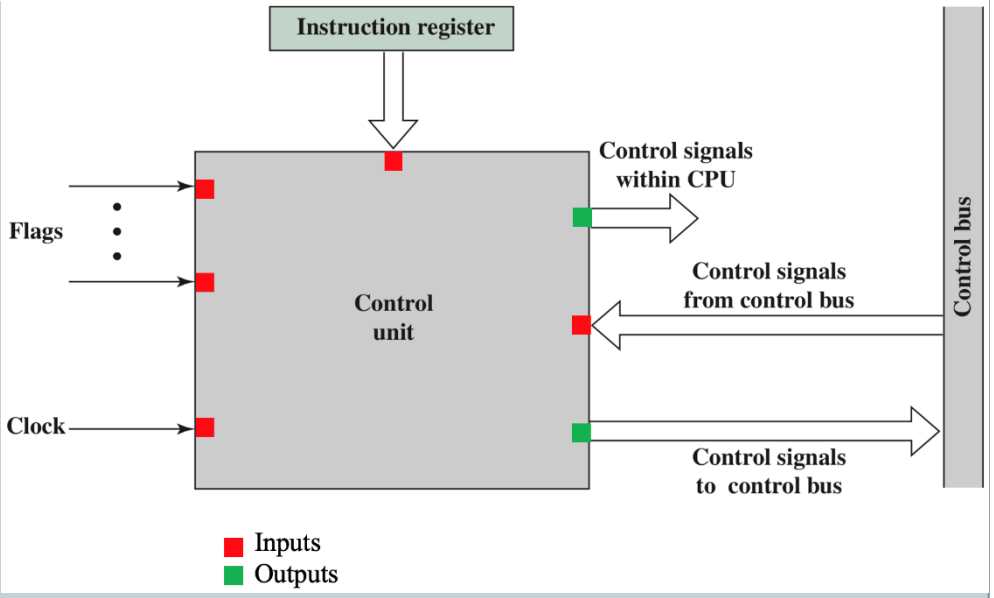
3. Determine the Control Unit functions

- The control unit performs two basic tasks:
  - **Sequencing**:
    - Go through a sequence of micro-operations, based on the program being executed.
  - **Execution**:
    - Execute each micro-operations.

- These 3 functional requirements for the control unit can be achieved through the use of **control signals**.

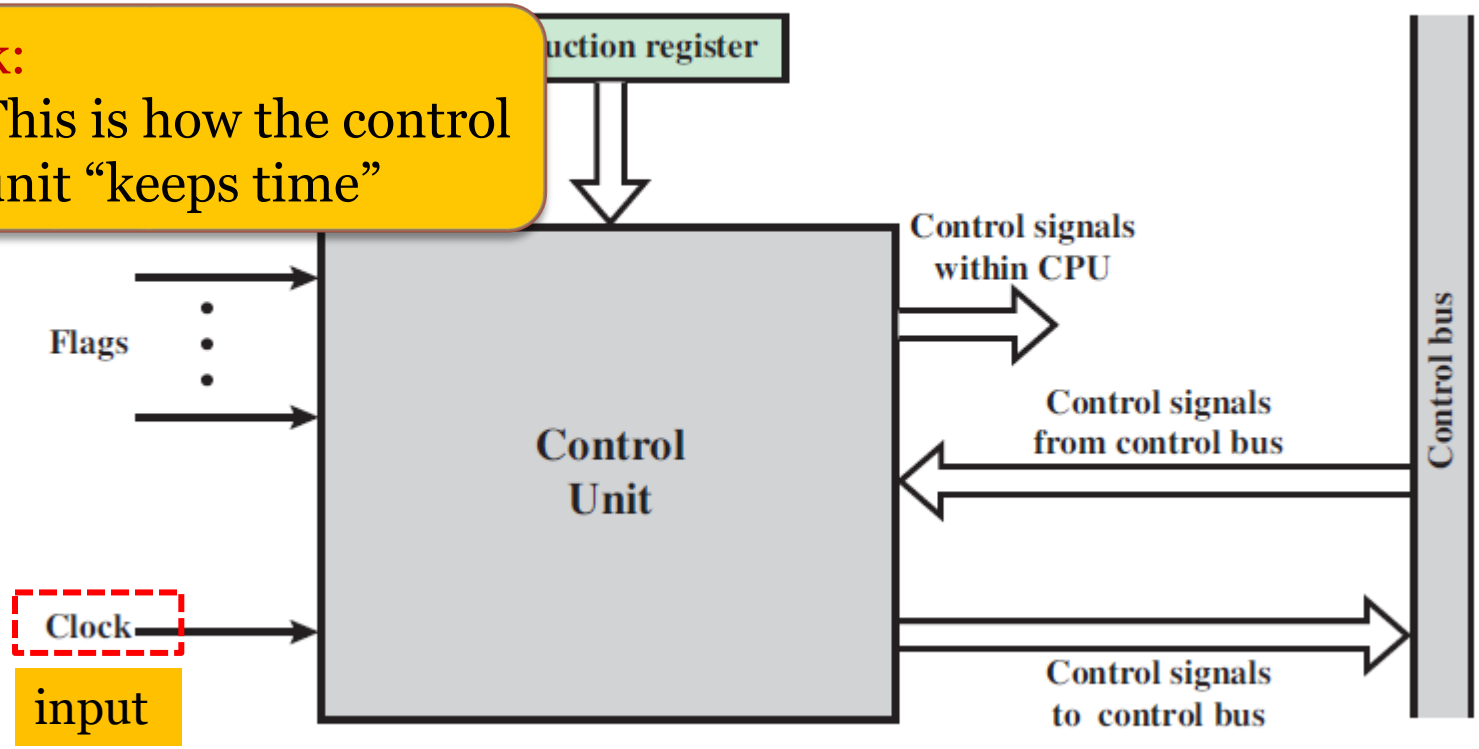
Control Signals

- For the control unit to perform its function, it must have
  - inputs that allow it to determine the state of the system
  - outputs that allow it to control the behavior of the system.



The inputs to CU

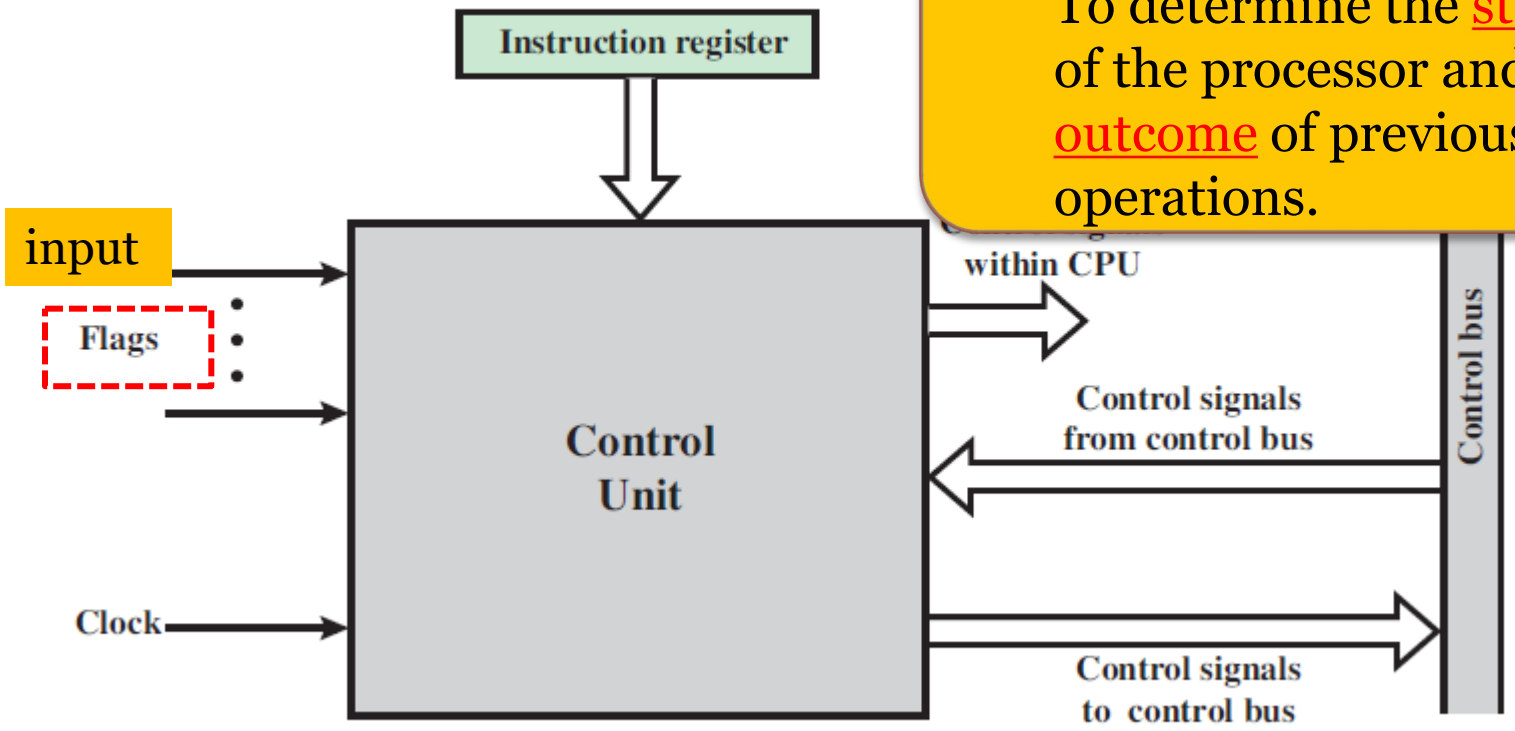
Clock:
This is how the control unit "keeps time"



The inputs to CU

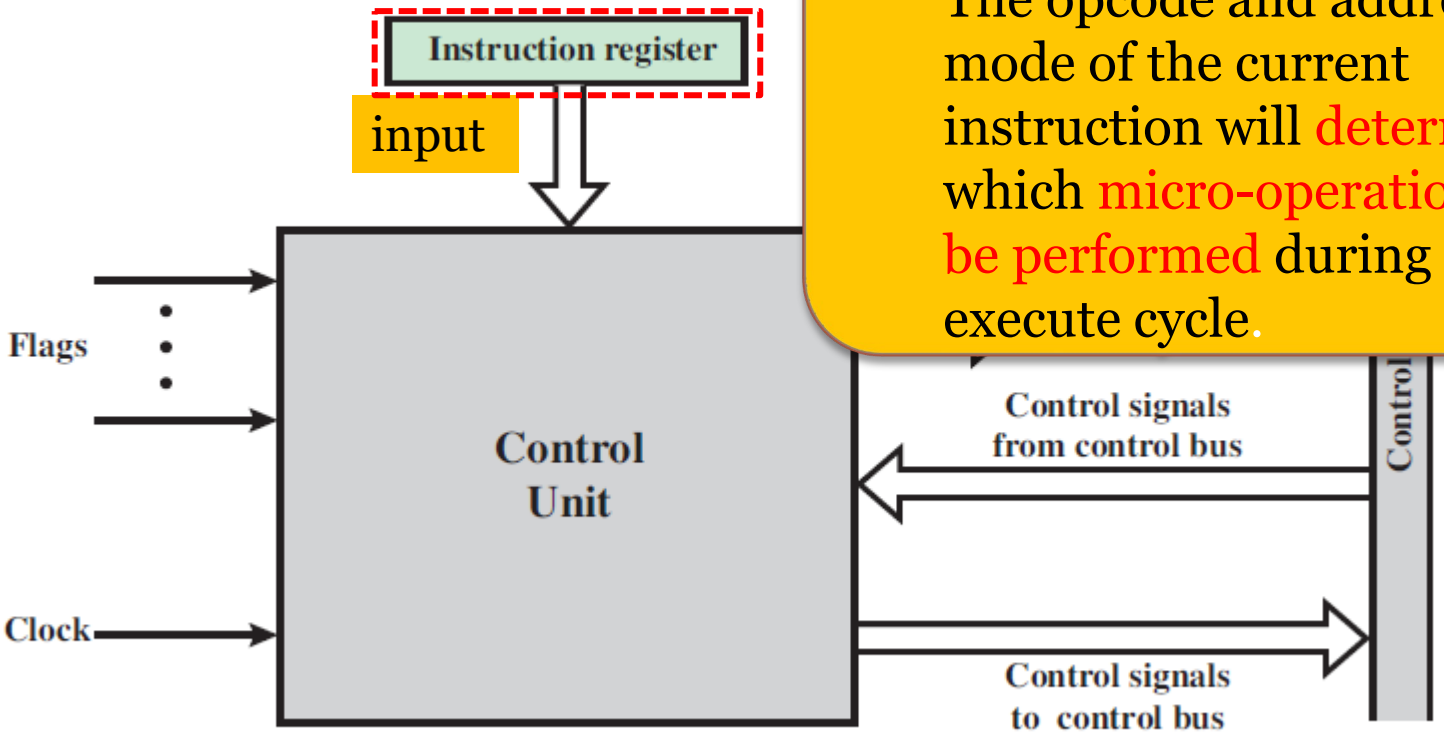
Flags:

To determine the status of the processor and the outcome of previous ALU operations.



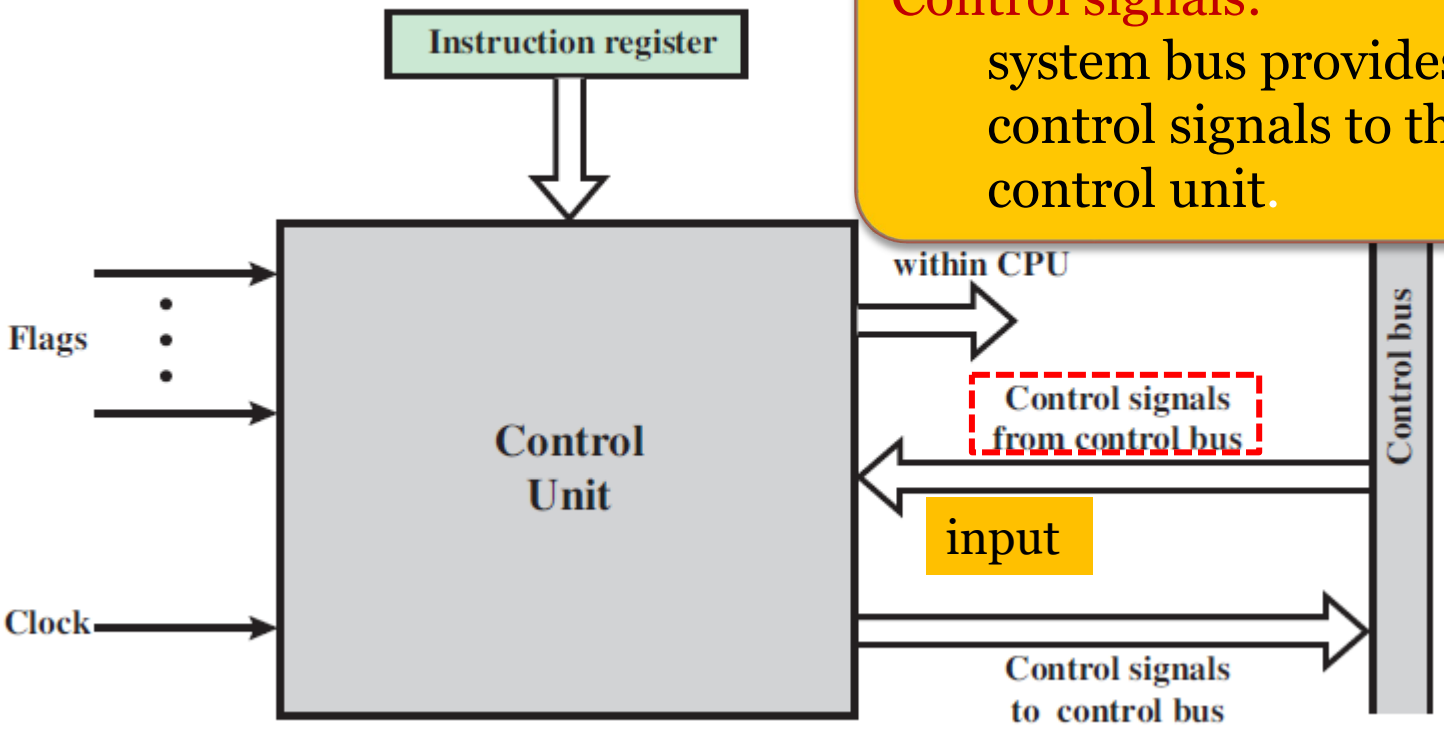
The inputs to CU

Instruction register:
The opcode and addressing mode of the current instruction will determine which micro-operations to be performed during the execute cycle.

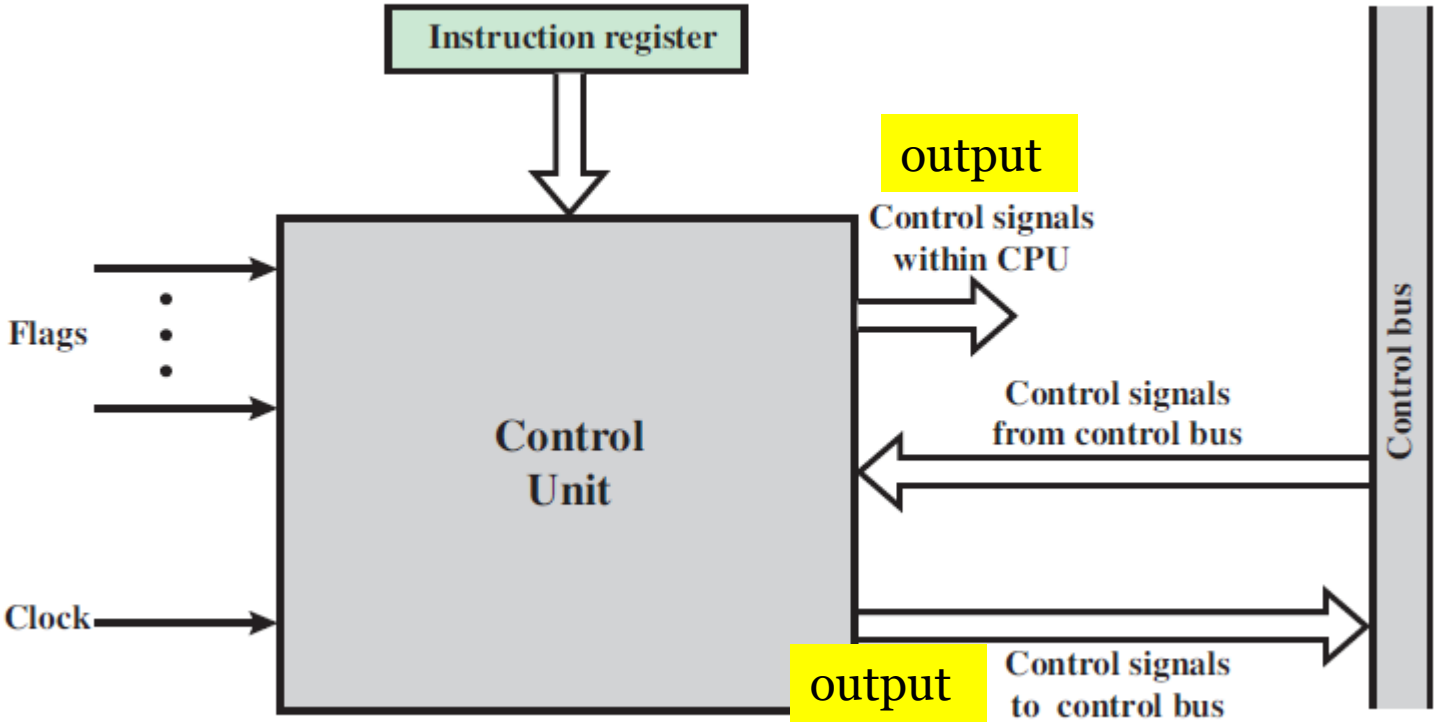


The inputs to CU

Control signals:
system bus provides control signals to the control unit.



The outputs of CU



The outputs of CU

- Control signals within the processor:
  - those that cause data to be moved from one register to another, and
  - those that activate specific ALU functions.
- Control signals to control bus:
  - control signals to memory, and
  - control signals to the I/O modules.

Implementation of Control Unit

Implementation of Control Unit



- A wide variety of techniques have been used. Most of these fall into one of two categories:
  - Hardwired implementation
  - Microprogrammed implementation

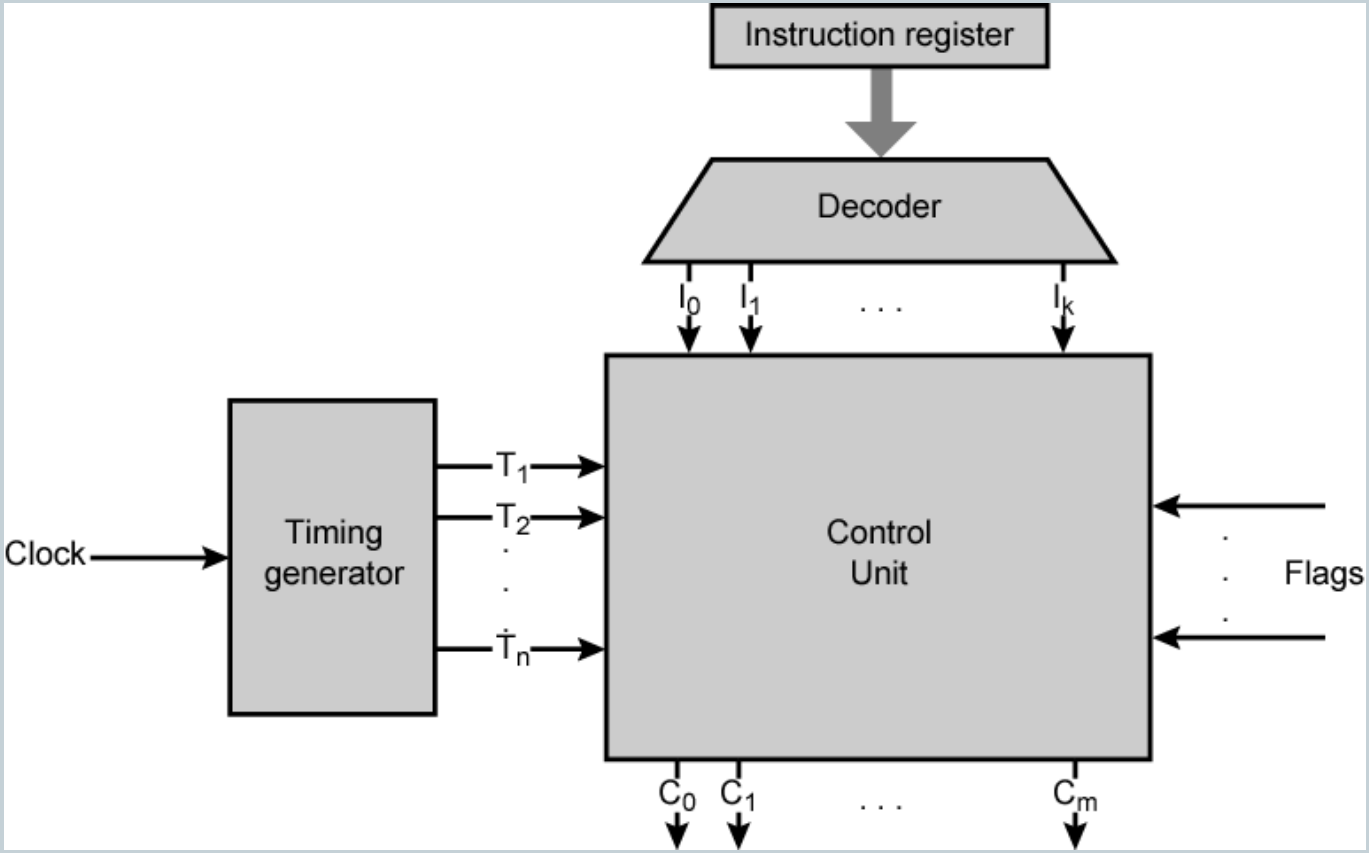
Implementation of Control Unit



- Hardwired implementation
  - Sequential circuits
  - The control logic is implemented with gates, flip flops, decoders, and other digital circuit components

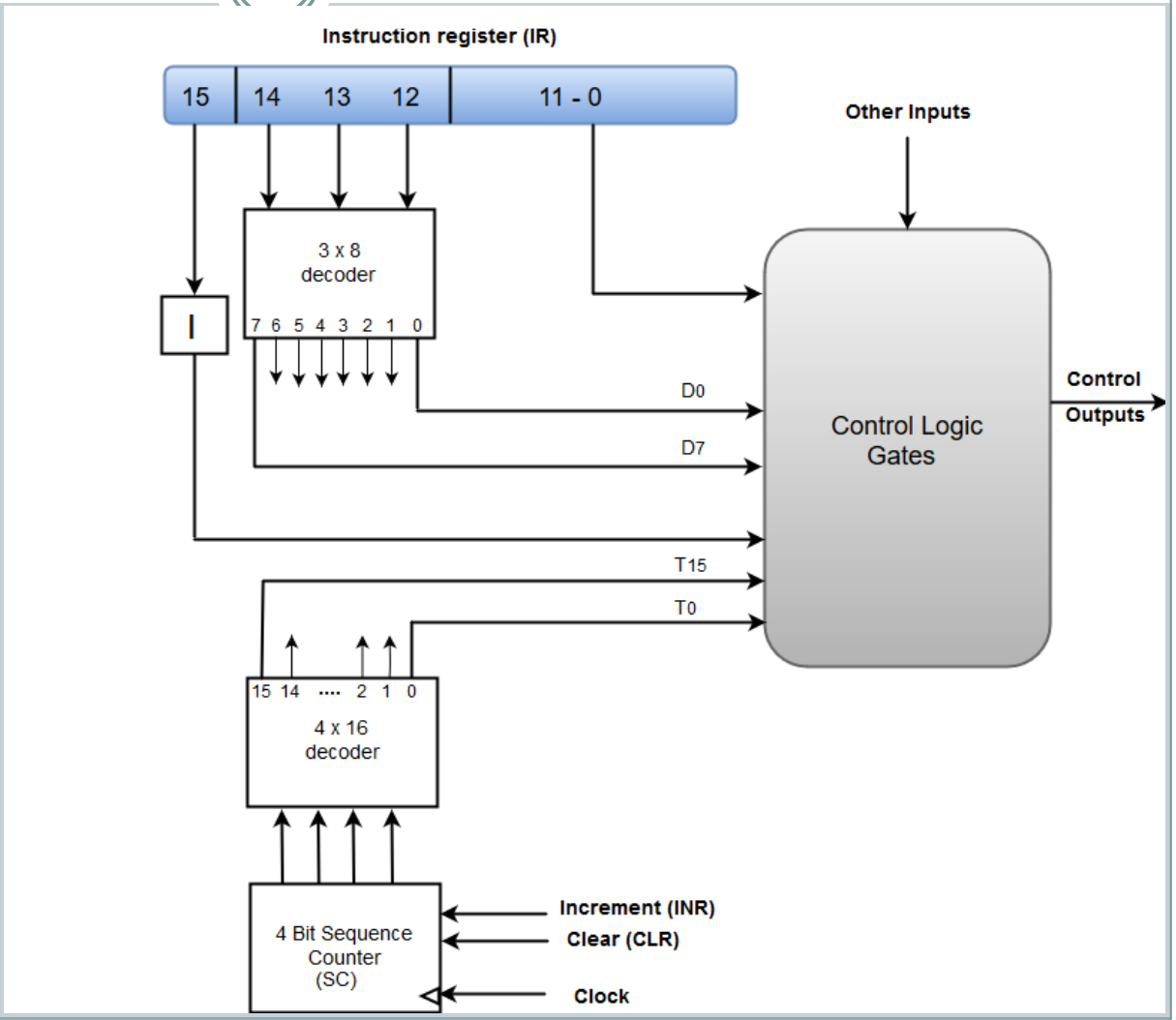
Implementation of Control Unit

- Hardwired implementation



Implementation of Control Unit

- Hardwired implementation
- 'Decoder' example #1
  - 3 inputs, 8 outputs



Implementation of Control Unit

- Hardwired implementation
- 'Decoder' example #1
  - 3 inputs, 8 outputs
- Logic circuit for this decoder

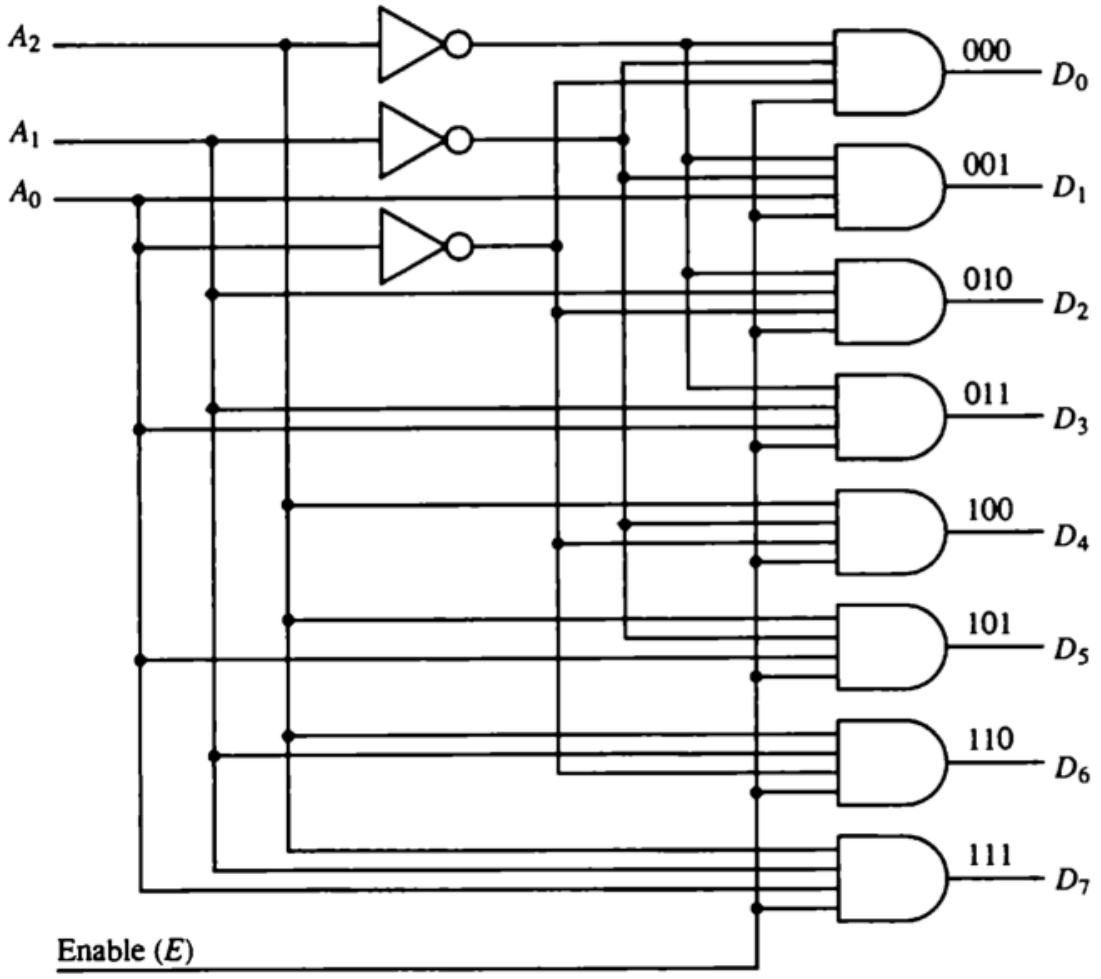


Figure 2-1 3-to-8-line decoder.

Implementation of Control Unit

- Hardwired; 'Decoder' example #1: Truth Table

TABLE 2-1 Truth Table for 3-to-8-Line Decoder

Enable	Inputs			Outputs							
E	A_2	A_1	A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	×	×	×	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

Implementation of Control Unit

- Hardwired implementation
- 'Decoder' example #2
  - 4 inputs, 16 outputs
- Truth table

Table 19.3 A Decoder With Four Inputs and Sixteen Outputs

I1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
I2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
I3	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
I4	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
O1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
O2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
O3	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
O4	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
O5	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
O6	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
O7	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
O8	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
O9	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
O10	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
O11	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
O12	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
O13	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
O14	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
O15	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
O16	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Implementation of Control Unit



- Hardwired implementation, problems;
  - Difficult to design and test
    - Complex sequencing & micro-operation logic
  - Inflexible design
  - Difficult to add new instructions
- Advantage:
  - Fast operation, small (requires less components)
- Disadvantage:
  - Needs wiring change (if the design has to be modified)

Implementation of Control Unit



- End of
  - … Hardwired implementation

Implementation of Control Unit



MICROPROGRAMMED

Micro-programmed Control Unit

- An alternative to a hardwired control unit is a microprogrammed control unit, in which the logic of the control unit is specified by a microprogram (firmware).
- A microprogram consists of a sequence of microinstructions
  - These are very simple instructions that specify micro-operations.
  - Microinstructions are encoded in microprogramming language.
- The microinstructions are stored in a **control memory**
  - the control memory is programmed to initiate the required sequence of micro-operations

Micro-programmed Control Unit

- A micro-programmed control unit is a logic circuit that is capable of
  - (1) sequencing through microinstructions and
  - (2) generating control signals to execute each microinstruction.
- The control signals generated by a microinstruction are used to cause
  - register transfers and
  - ALU operations.

Micro-programmed Control Unit

- Advantage:
  - Systematic design
  - Any required changes can be done by updating the micro-program in control memory.
- Disadvantage:
  - Slow operation
  - Requires more components

Example Implementation of Control Unit

	Micro-operations	Active Control Signals
Fetch:	t_1: MAR ← (PC)	C_2
	t_2: MBR ← Memory PC ← (PC) + 1	C_5, C_R
	t_3: IR ← (MBR)	C_4
Indirect:	t_1: MAR ← (IR(Address))	C_8
	t_2: MBR ← Memory	C_5, C_R
	t_3: IR(Address) ← (MBR(Address))	C_4
Interrupt:	t_1: MBR ← (PC)	C_1
	t_2: MAR ← Save-address PC ← Routine-address	
	t_3: Memory ← (MBR)	C_{12}, C_W

C_R = Read control signal to system bus.
C_W = Write control signal to system bus.

Example Implementation of Control Unit

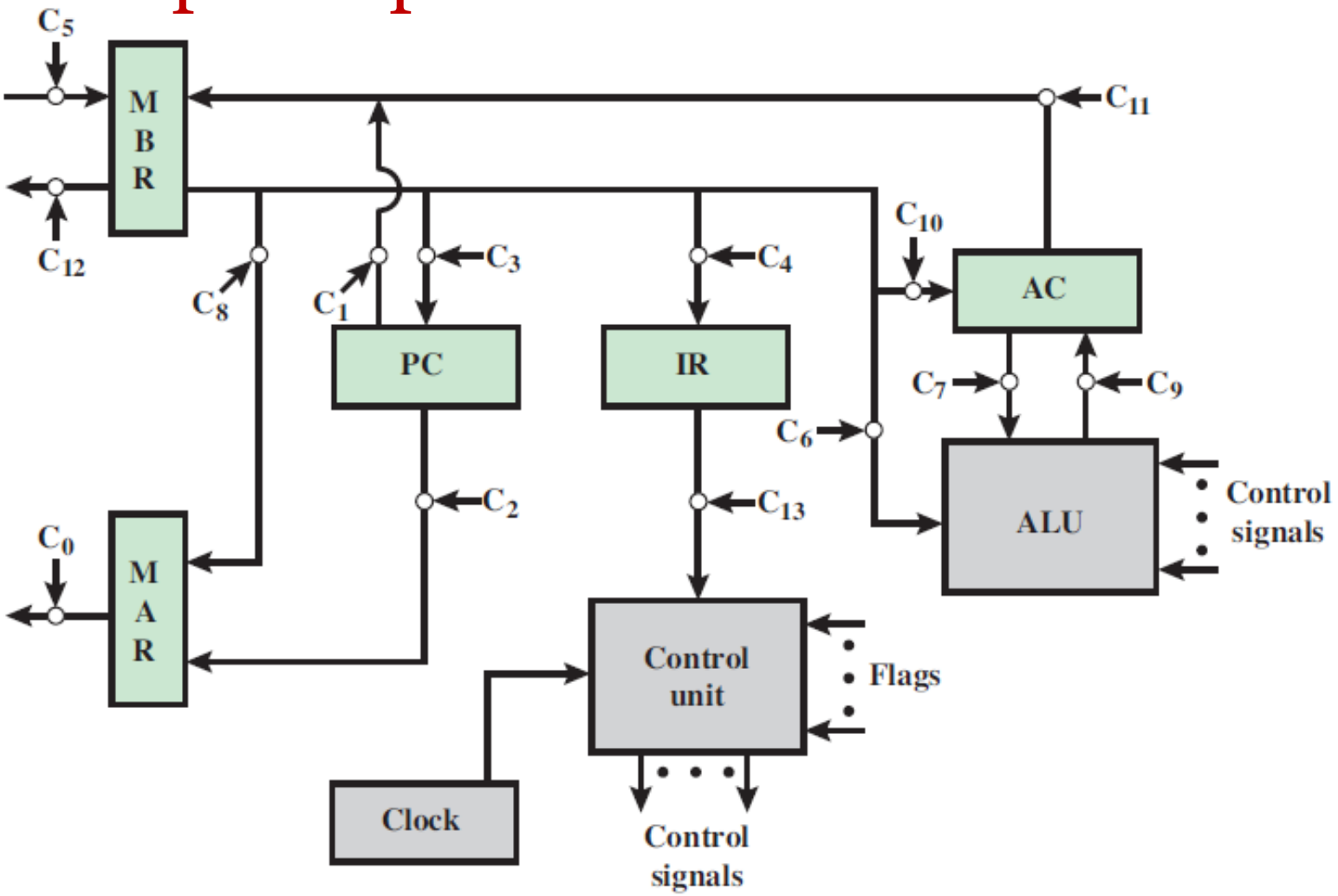
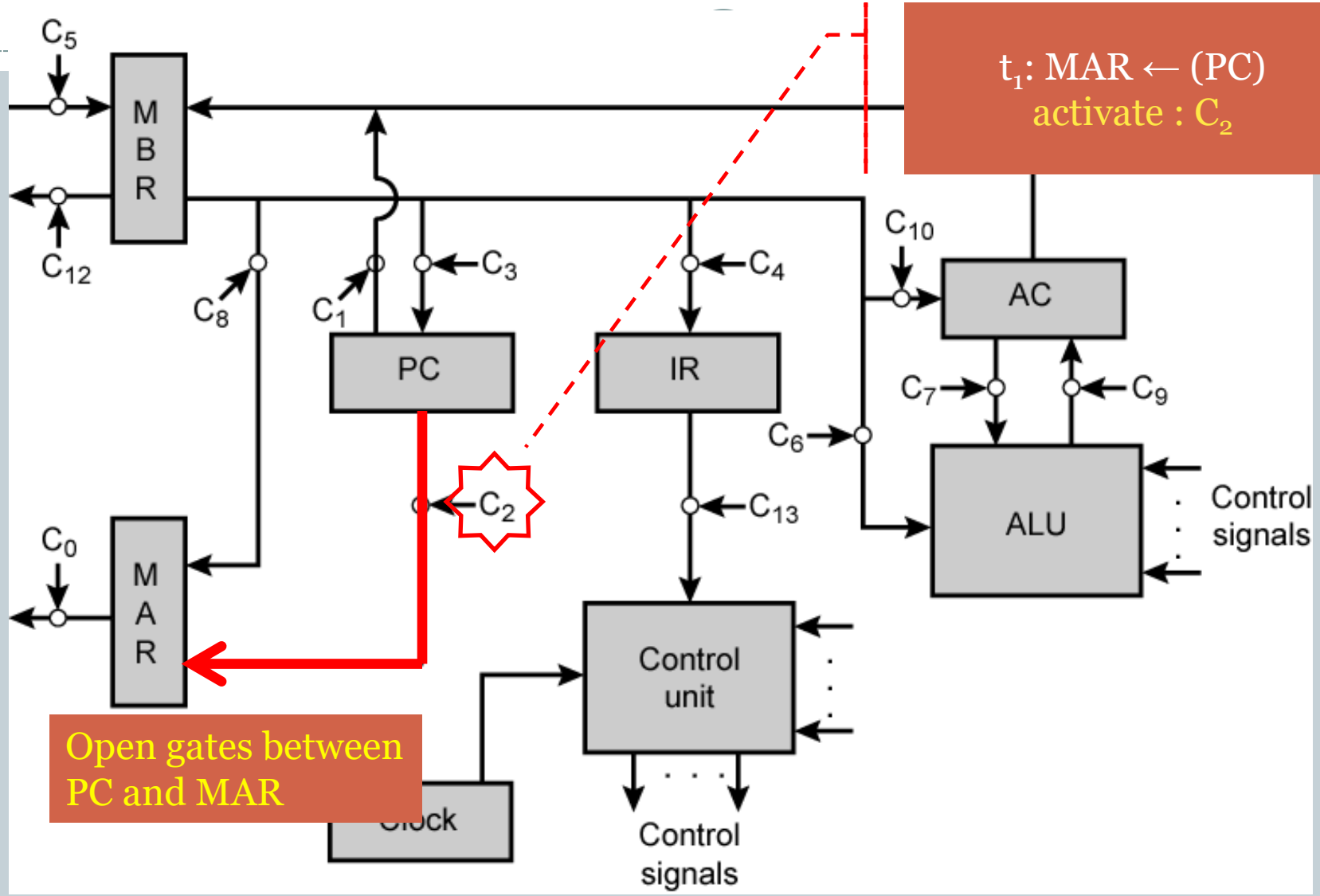
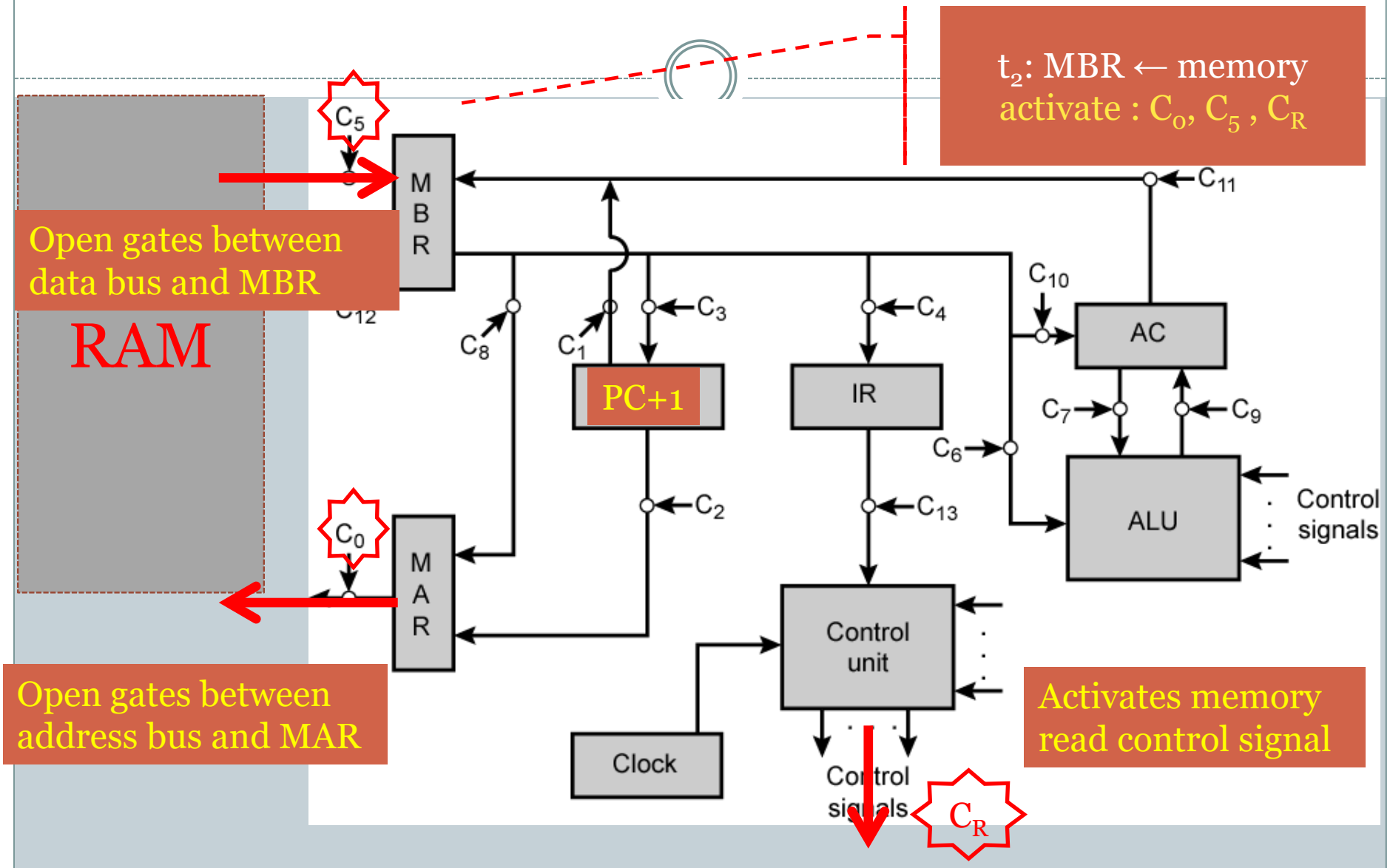


Figure 19.5 Data Paths and Control Signals

Example Control Signal Sequence - fetch



Example Control Signal Sequence - fetch

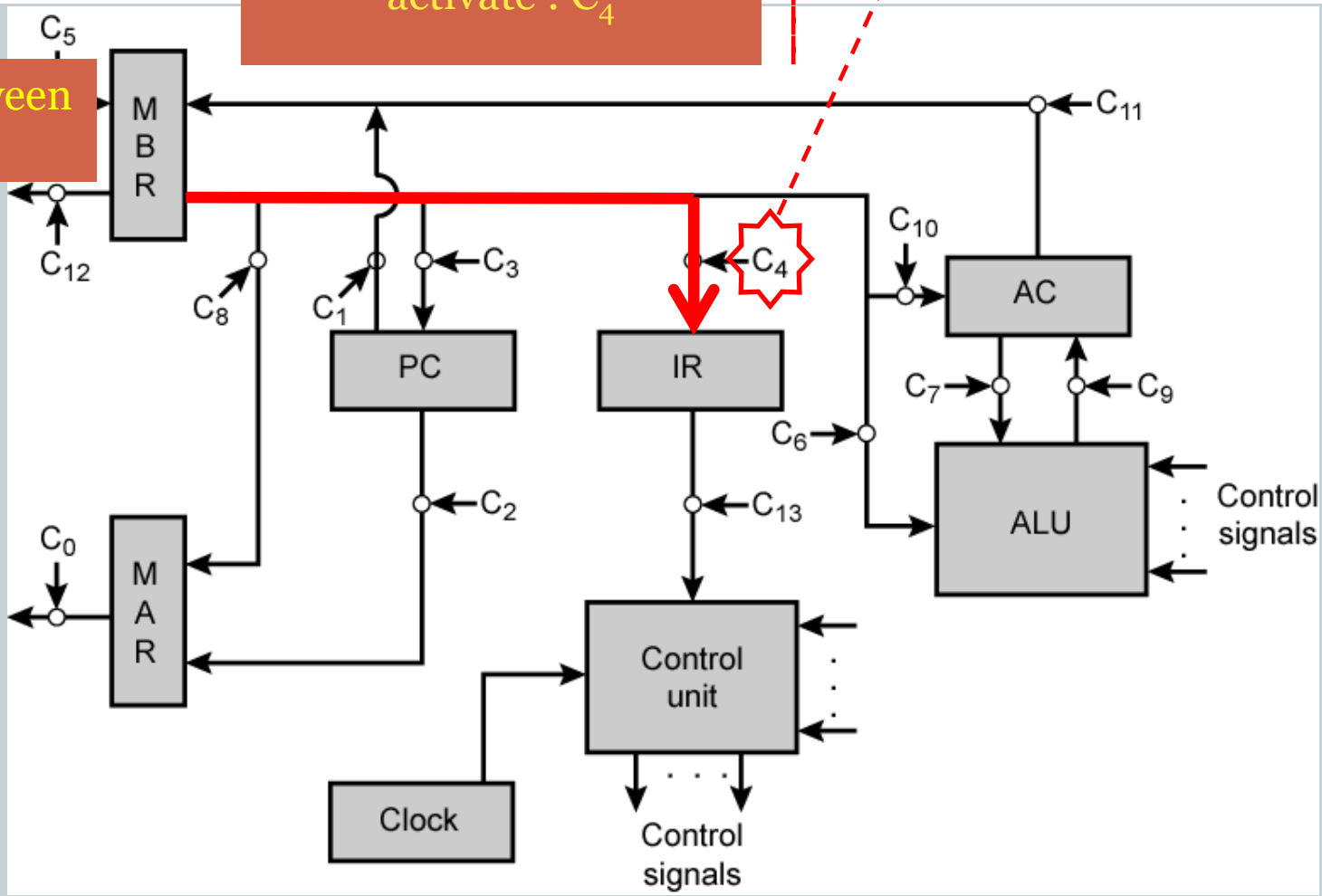


Example Control Signal Sequence - fetch

t_3: IR ← MBR
activate : C_4

open gates between MBR and IR

RAM



Example Control Signal Sequence - fetch

- MAR <- (PC)
  - Control unit activates signal to open gates between PC and MAR
- MBR <- memory
  - Open gates between MAR and address bus
  - Memory read control signal
  - Open gates between data bus and MBR
  - Increment PC by 1
- IR <- (MBR)
  - Control unit activates signal to open gates between IR and MBR

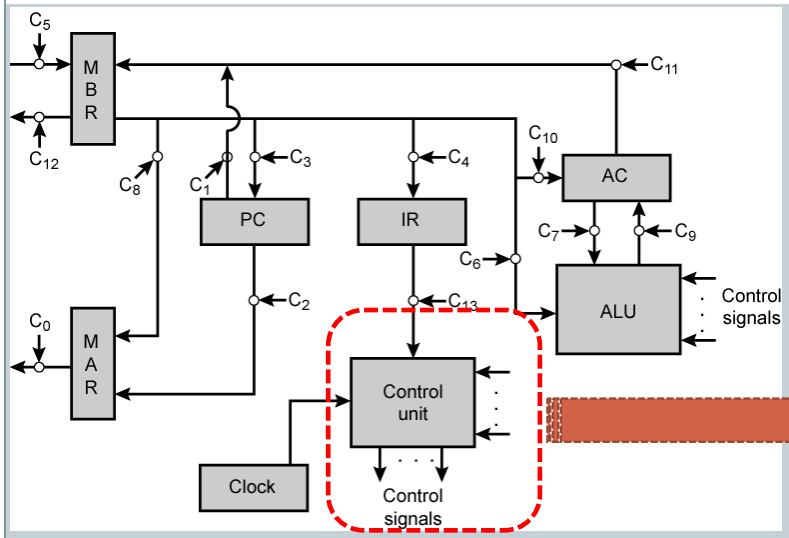
Micro-operations	Active Control Signals
t_1: MAR ← (PC)	C_2
t_2: MBR ← Memory PC ← (PC) + 1	C_5, C_R
t_3: IR ← (MBR)	C_4

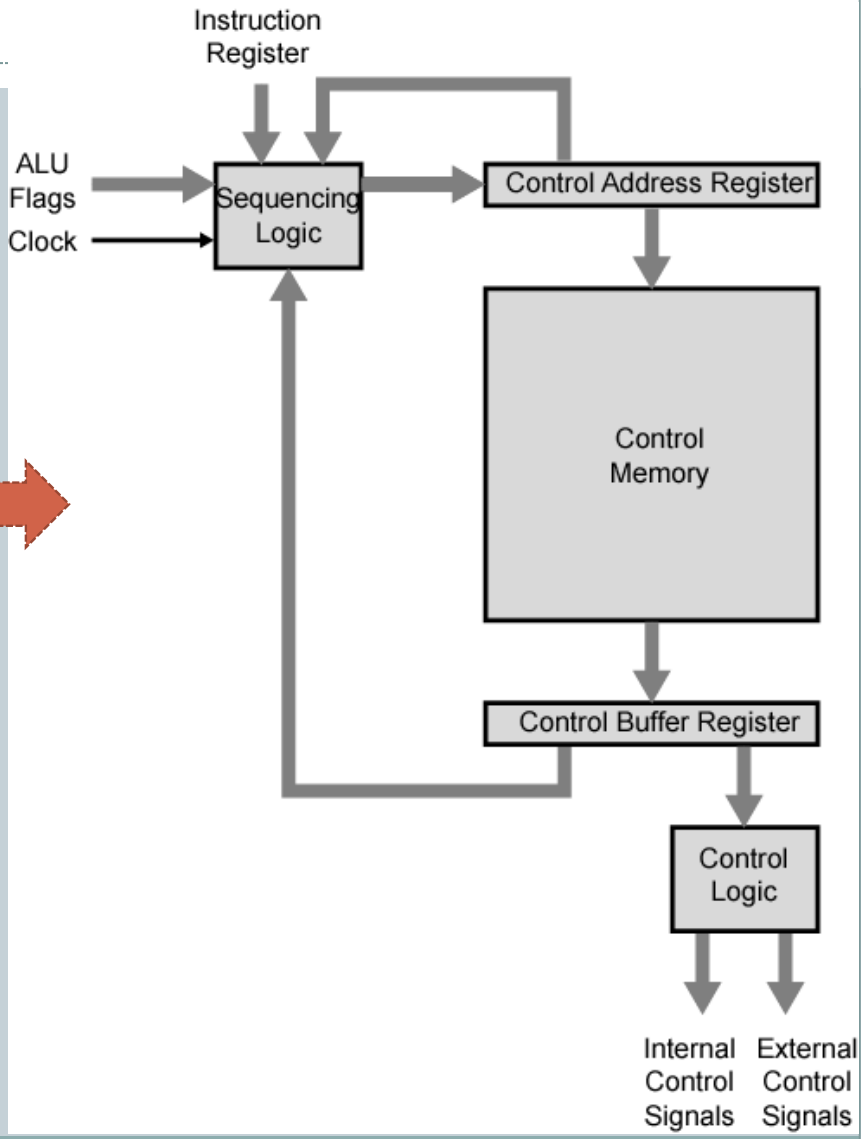
Microprogrammed Control



CONTROL UNIT ORGANIZATION

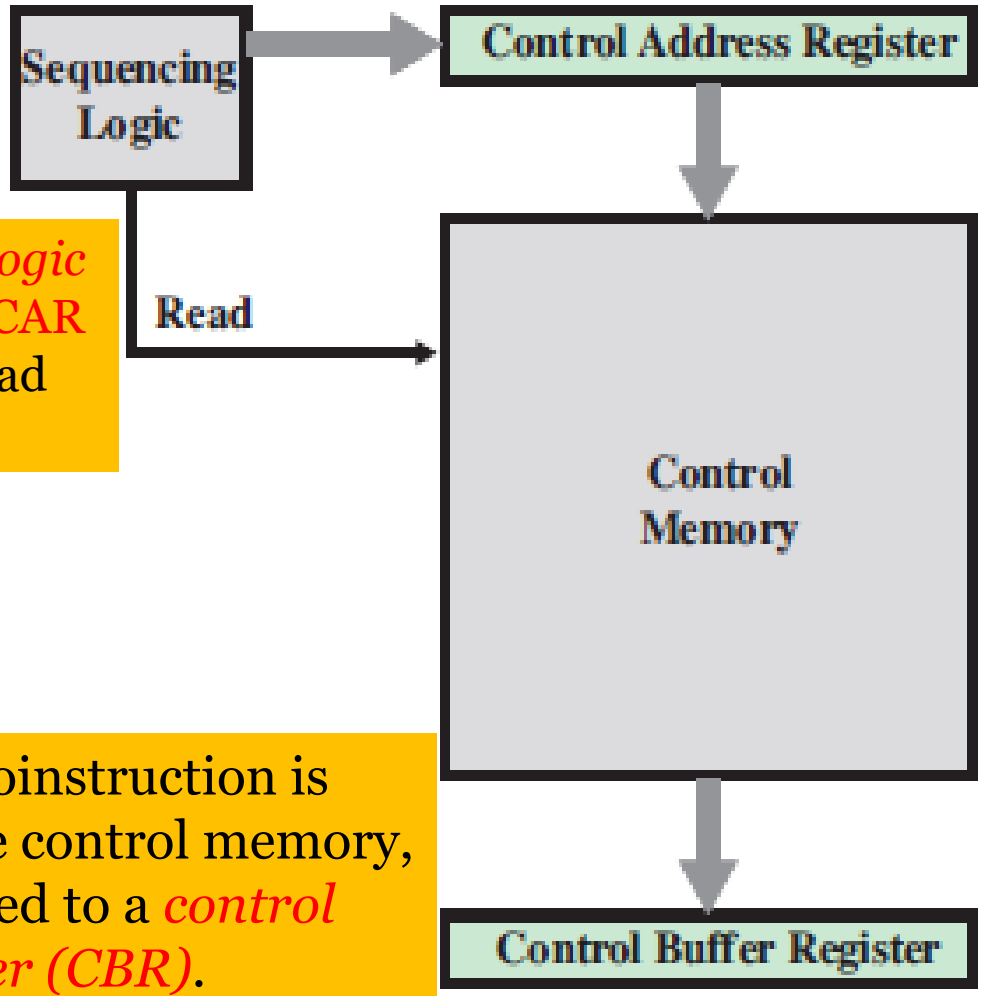
Control Unit Organization





- To discuss how control unit functions

Micro-programmed Control Unit



The *control address register (CAR)* contains the address of the next microinstruction to be read.

A *sequencing logic* unit loads the CAR and issues a read command.

The set of microinstructions are stored in the *control memory*.

When a microinstruction is read from the control memory, it is transferred to a *control buffer register (CBR)*.

The figure shows how the micro instructions could be arranged in a **control memory.**

The micro instructions in each routine are to be executed sequentially.

Each routine ends with a jump instruction indicating where to go next.

A special execute cycle routine for machine instructions which depend on opcode

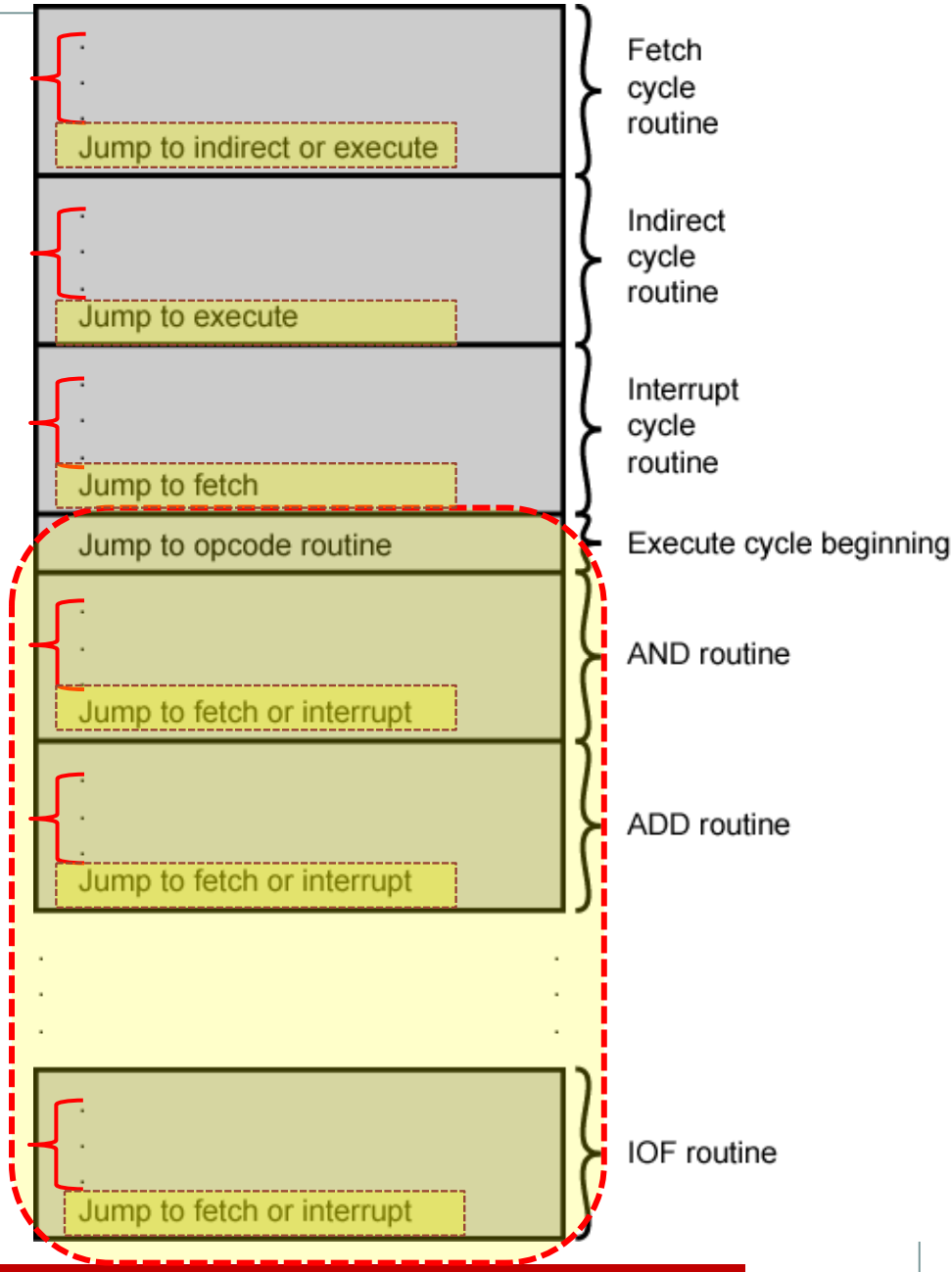
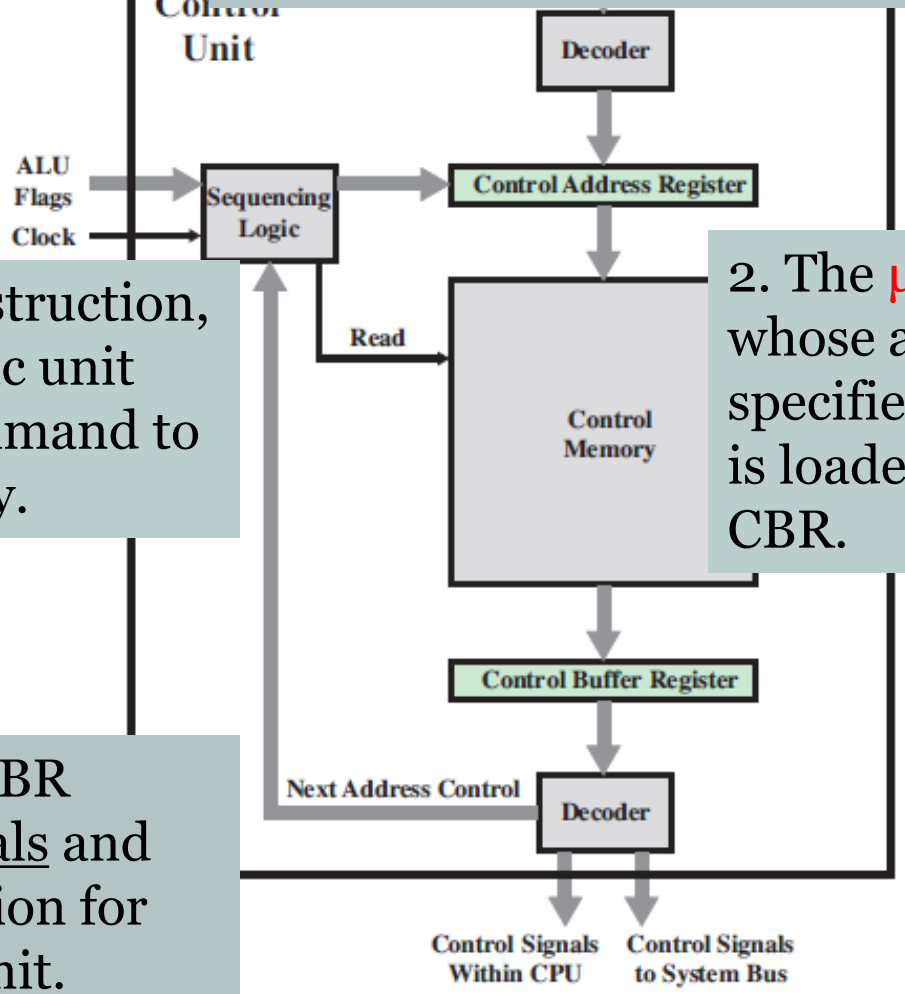


Figure: Control Memory contents

All this happens during one clock pulse.

4. The sequencing logic unit loads a new address into the CAR based on the next-address information from the CBR and the ALU flags.



1. To execute an instruction, the sequencing logic unit issues a READ command to the control memory.

2. The μ-instruction whose address is specified in the CAR is loaded into the CBR.

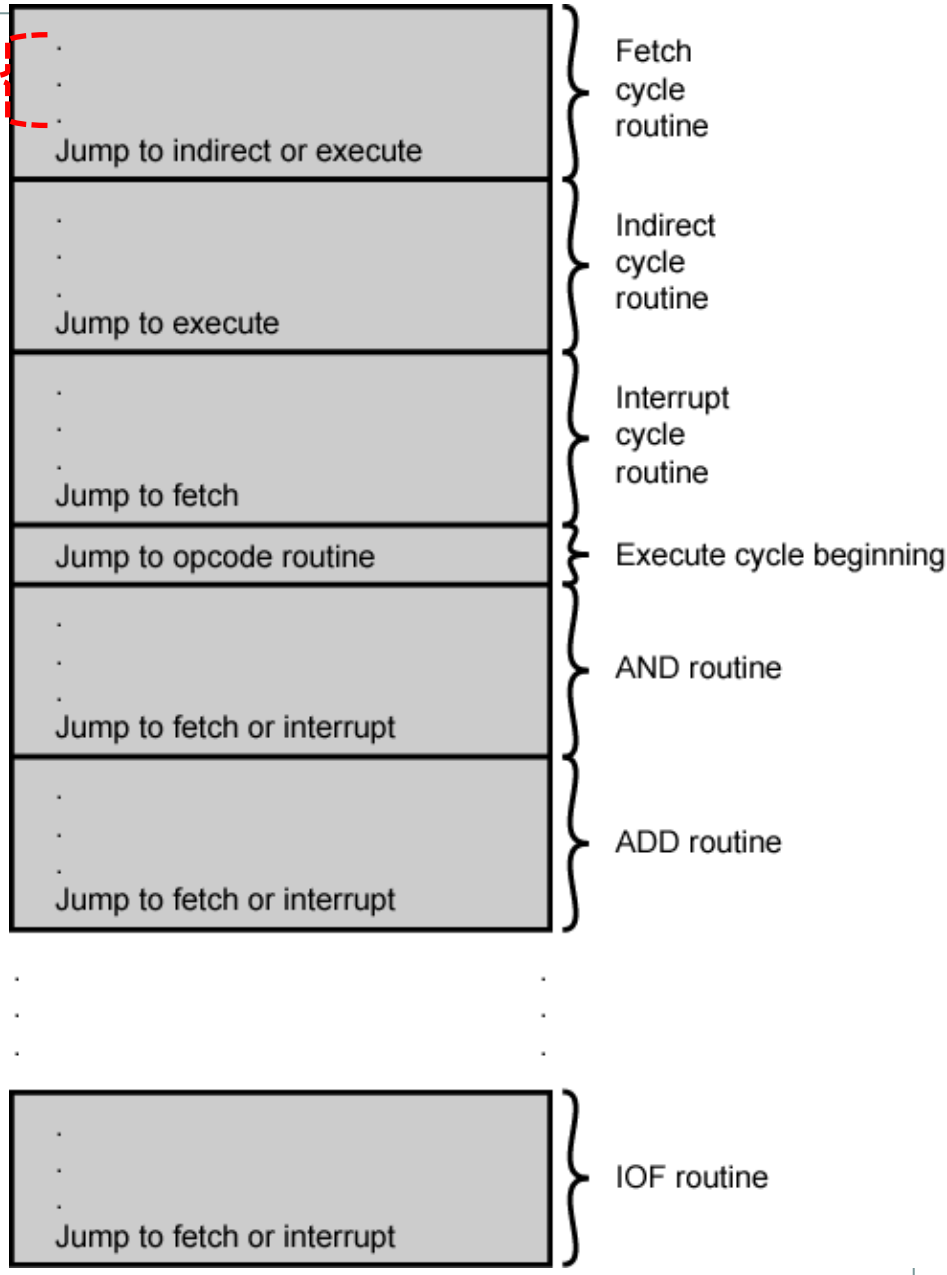
3. The content of the CBR generates control signals and next-address information for the sequencing logic unit.

Next Address Decision (@step 3)



- At the execution of each μ- instruction, the sequencing logic unit loads a new address into the control address register (CAR).
- Depending on the value of the ALU flags and the control buffer register, one of these three decisions is made:
  - Get the next instruction
  - Jump to a new routine based on a jump microinstruction
  - Jump to a opcode routine

1. Get the next instruction: Add 1 to the CAR

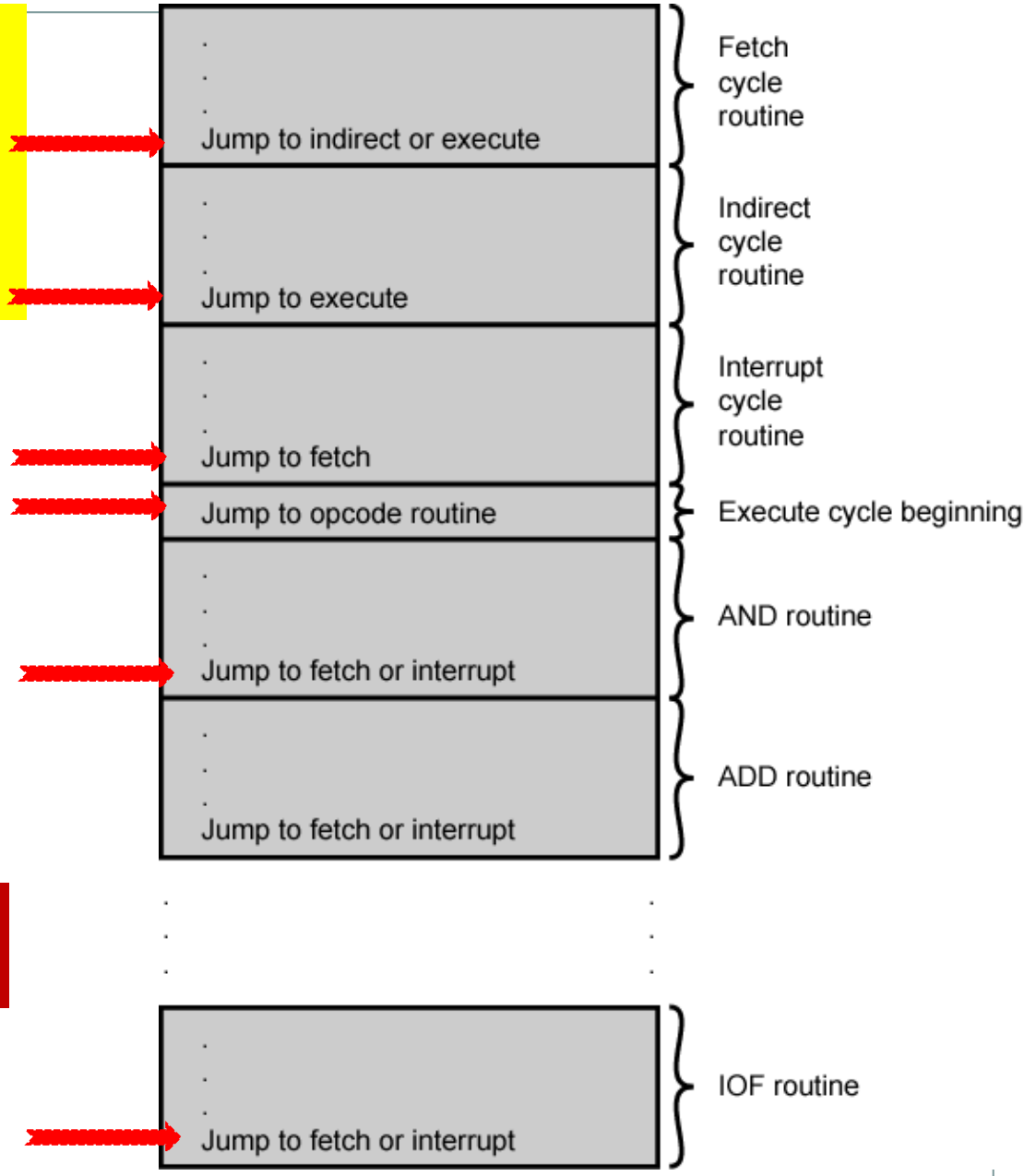


CBR - control buffer register
CAR - control address register

Control Memory

2. Jump to a new routine based on a jump microinstruction:

Load the address field of the CBR into the CAR.

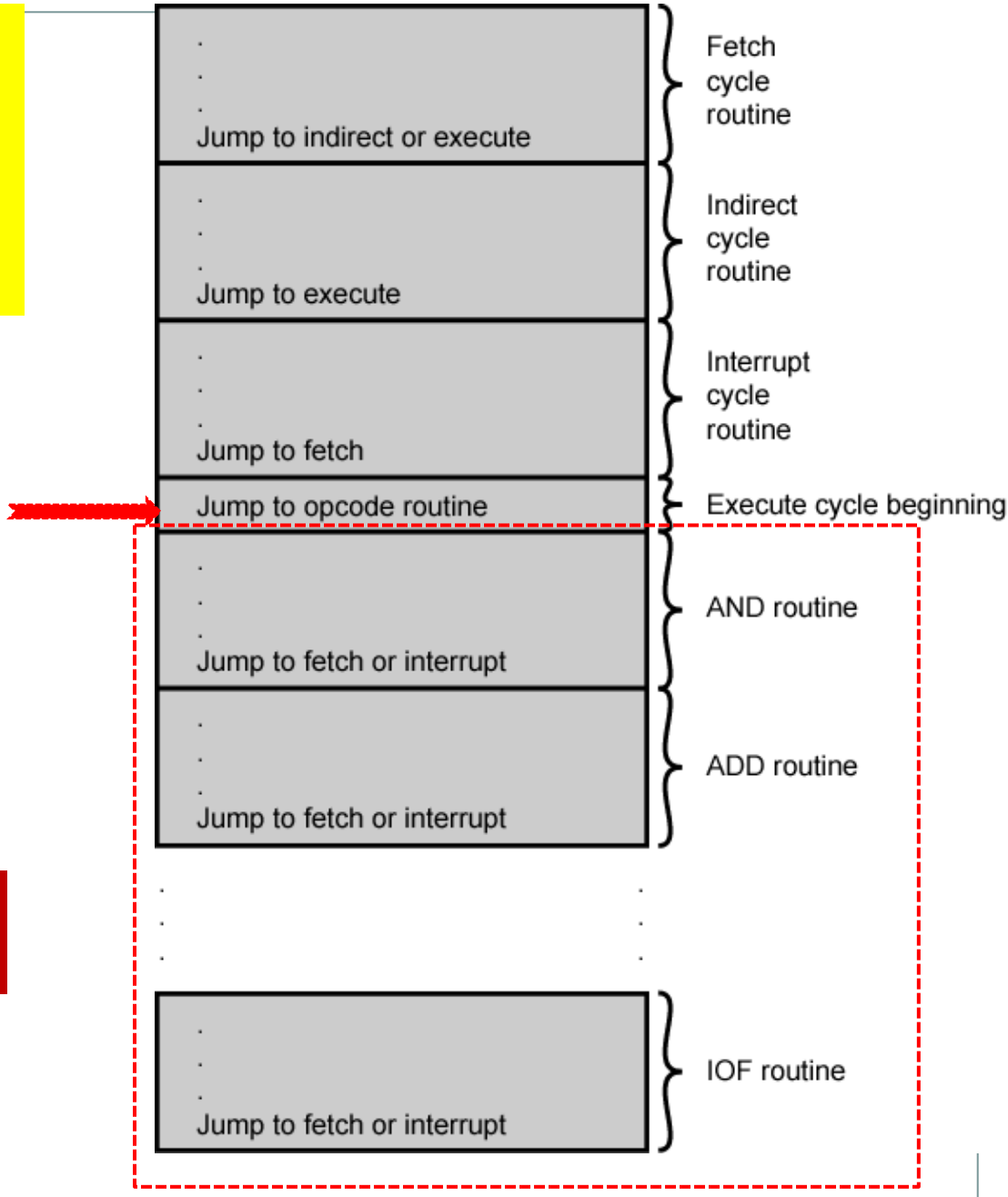


CBR - control buffer register
CAR - control address register

Control Memory

3. Jump to a machine instruction (opcode) routine:

Load the CAR based on the opcode in the IR.

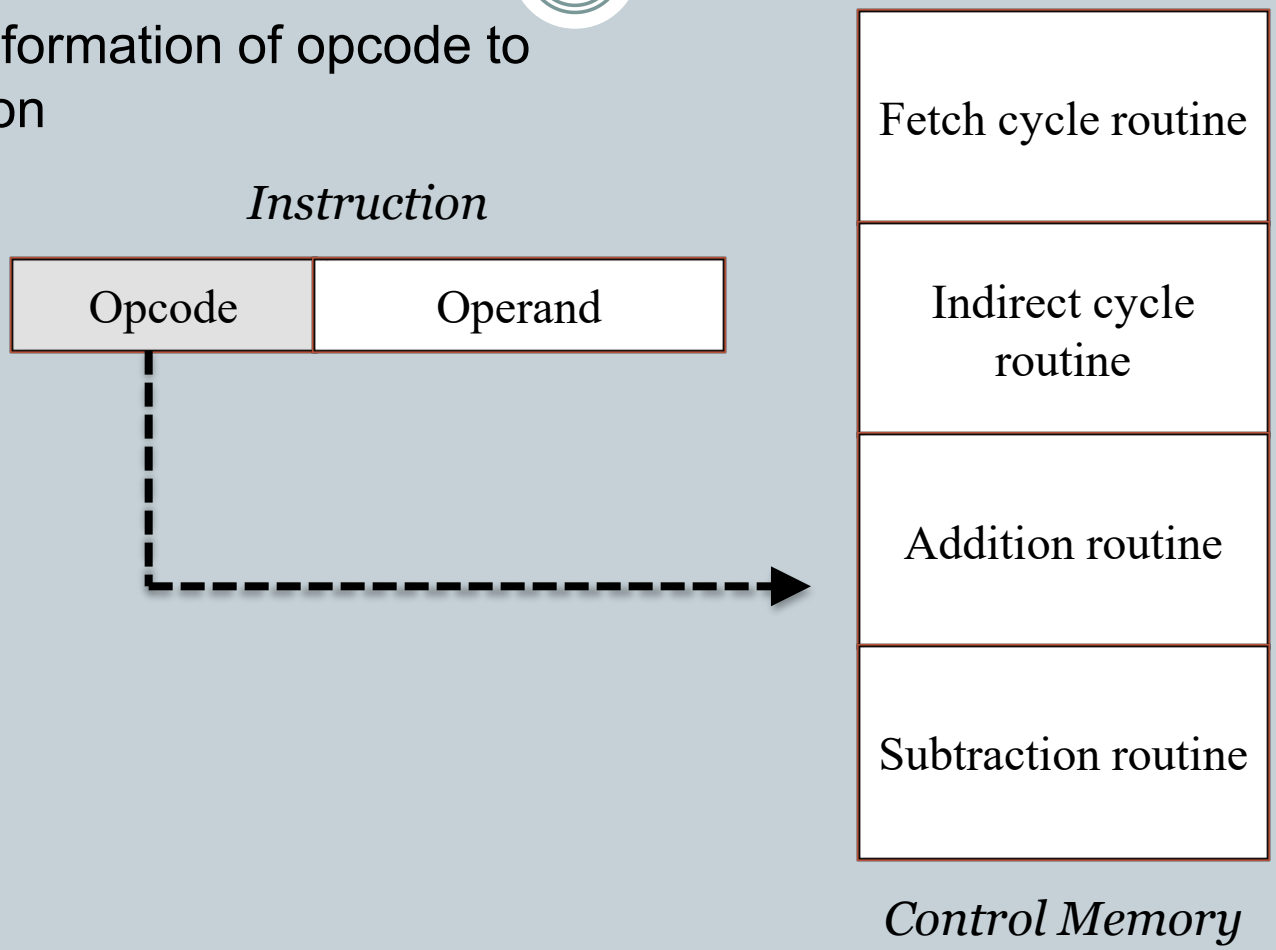


CBR - control buffer register
CAR - control address register

Control Memory

Transformation of Microprogram

Figure: Transformation of opcode to microinstruction



Transformation of Microprogram

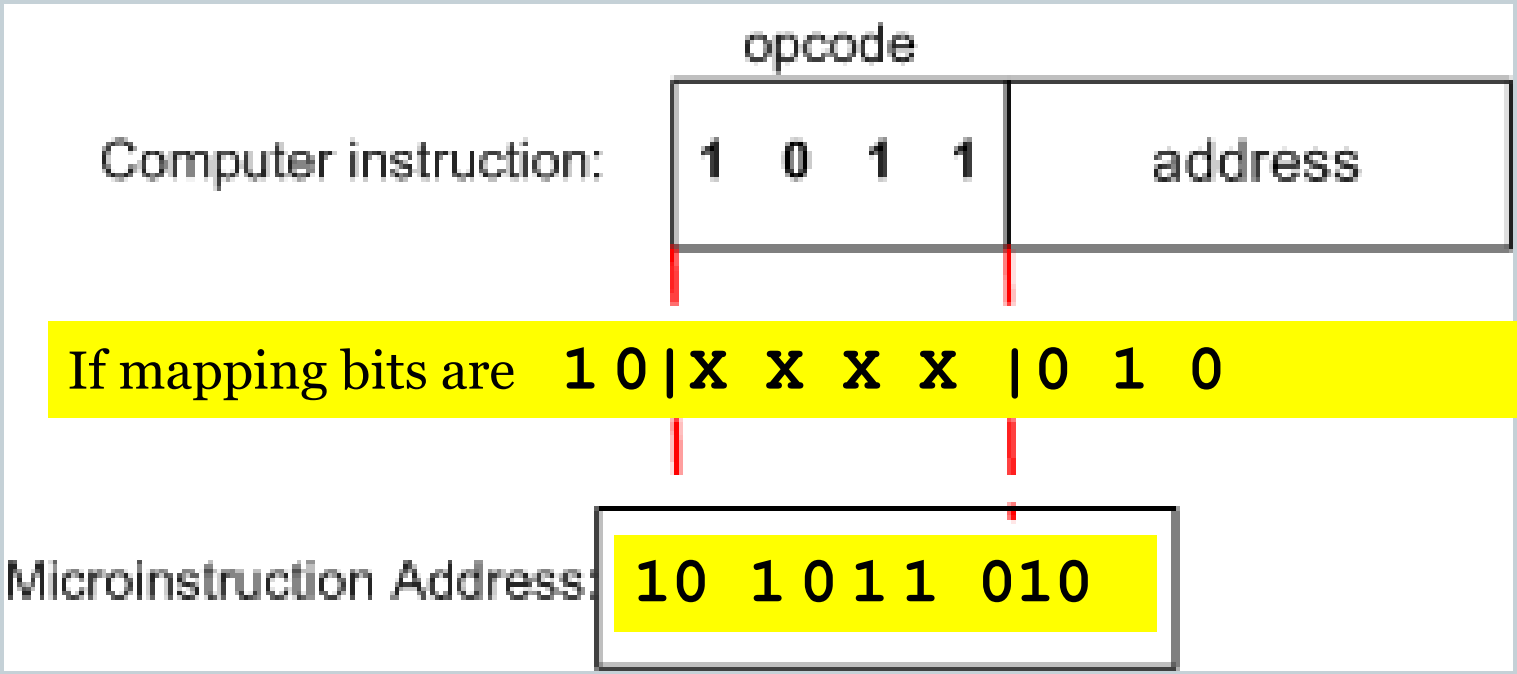
- Involve mapping of control memory address and opcode bits
- There are 2 methods:
  - Direct Mapping
  - Using ROM to specify mapping function

Direct Mapping

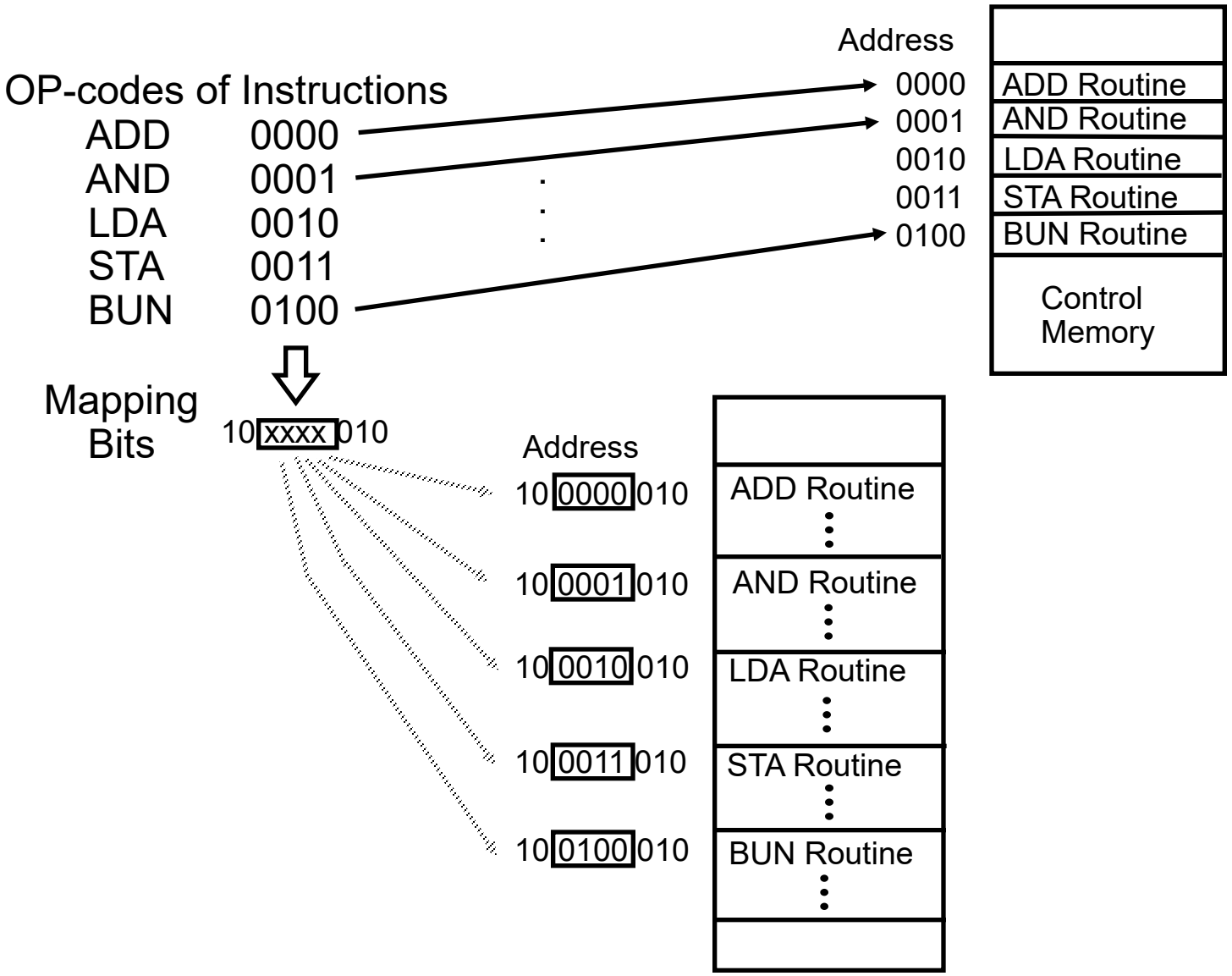


- We assume:
  - An instruction with 4 bit opcode
    - can specify up to 16 distinct machine instructions
  - A control memory with 9 bit address
    - can specify up to 512 micro instructions

Direct Mapping

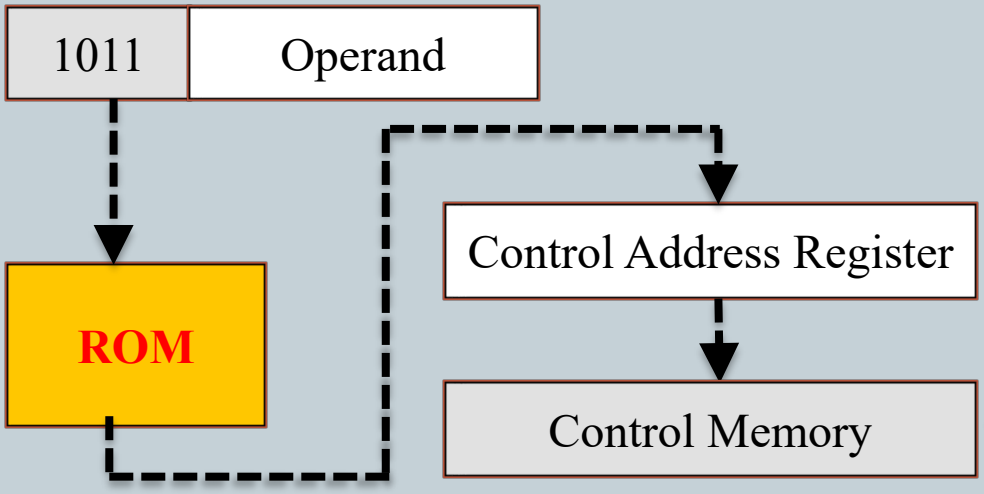


Direct Mapping Example



Using ROM to specify mapping function

- We can extend the concept of direct mapping with more general rules by using a ROM to specify the mapping function.
- The bits of the instruction specify the address of a mapping ROM.
- The contents of the mapping ROM gives the bits for the control address register.



The microprogram routine can be anywhere in the control memory → provides flexibility for adding instructions for control memory as need arise.

* Mano, Morris M. (1993). *Computer System Architecture* (3rd Edition). p.219.

Microprogrammed Control



MICROINSTRUCTION FORMATS

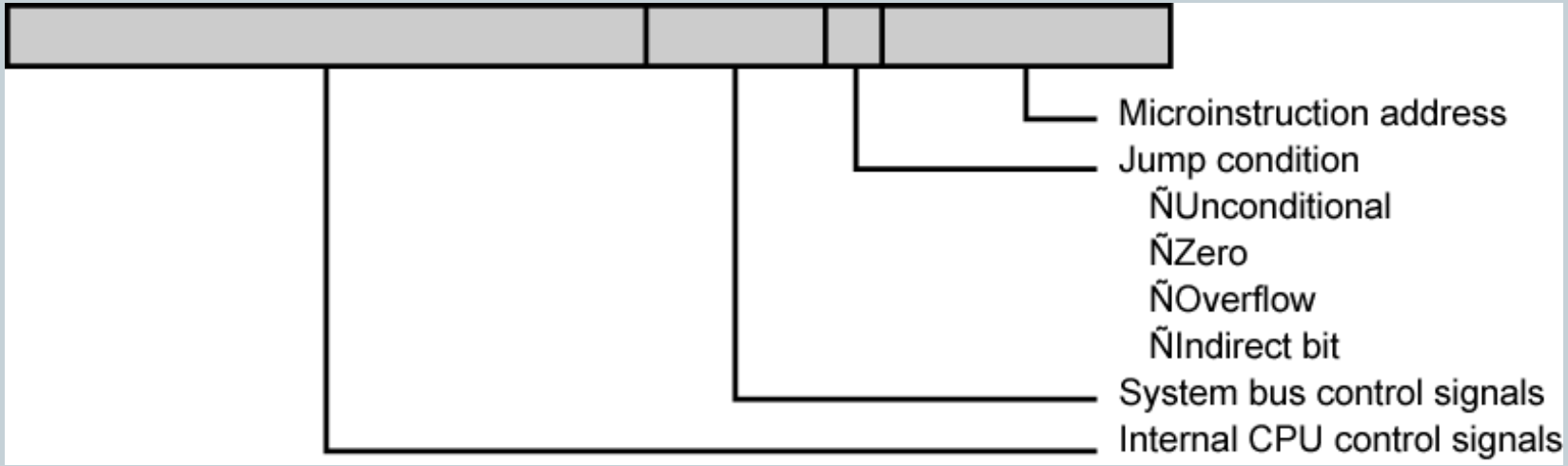
Microinstruction formats



- Microinstruction formats are typically horizontal or vertical

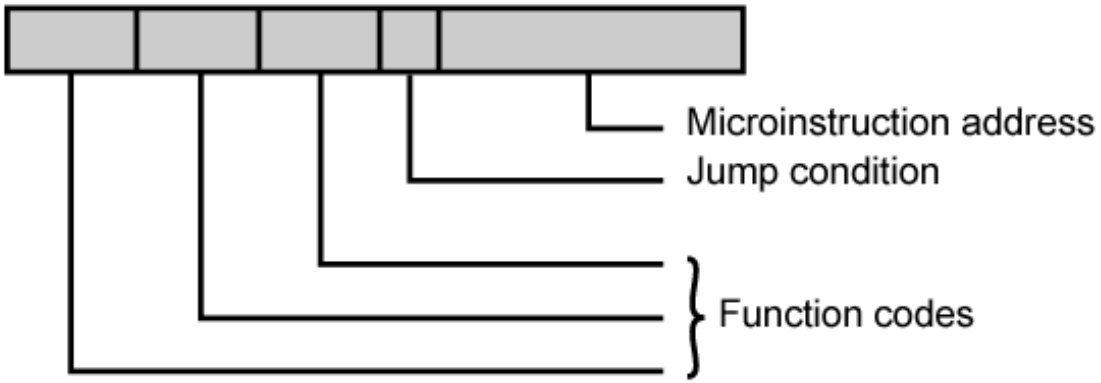
Microinstruction formats

- *Horizontal* micro-programming
  - Each micro-instruction specifies *many* different micro-operations to *be performed in parallel*



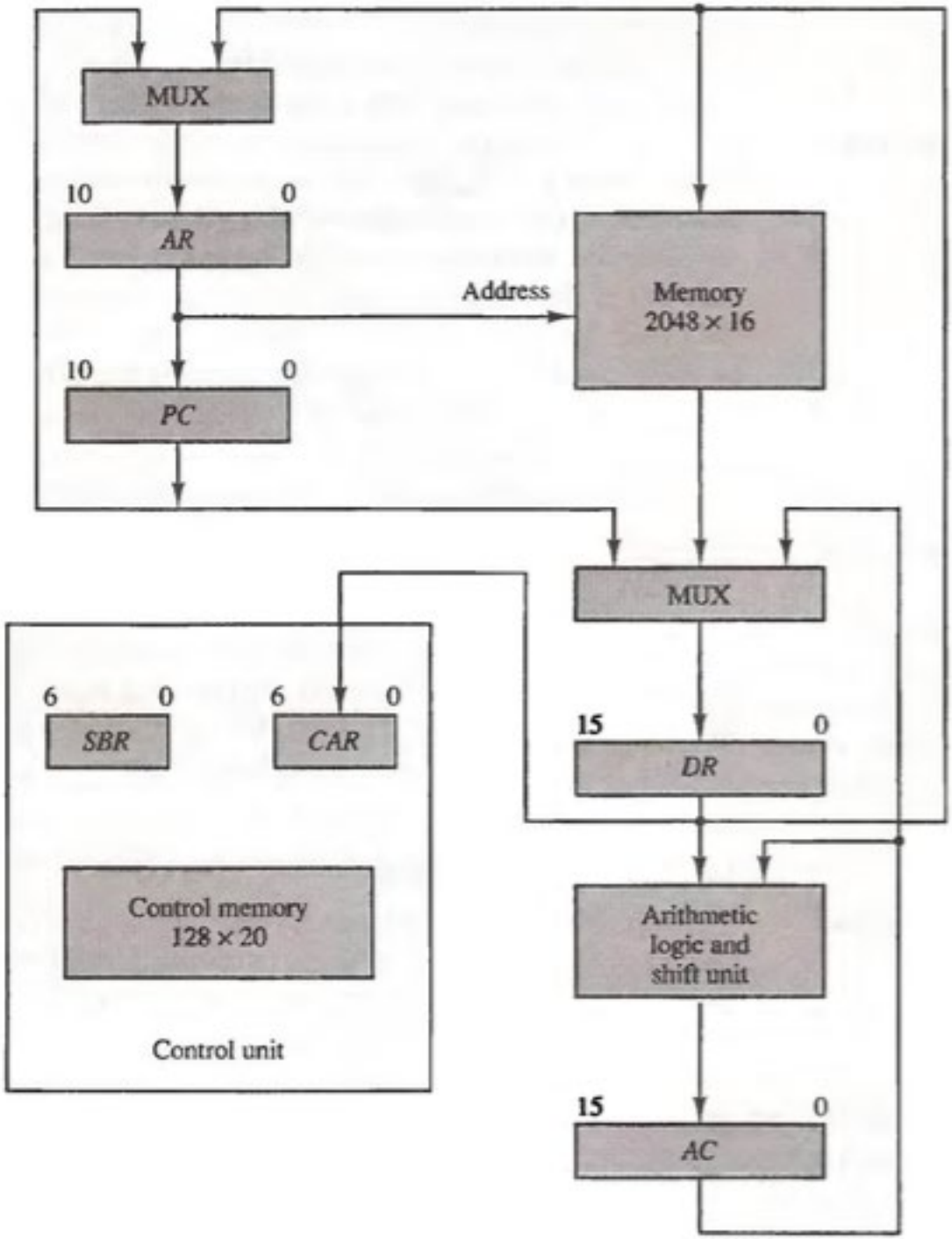
Microinstruction formats

- *Vertical* micro-programming
  - Each micro-instruction specifies *single* (*or few*) micro-operations to be performed



Example of microinstruction format

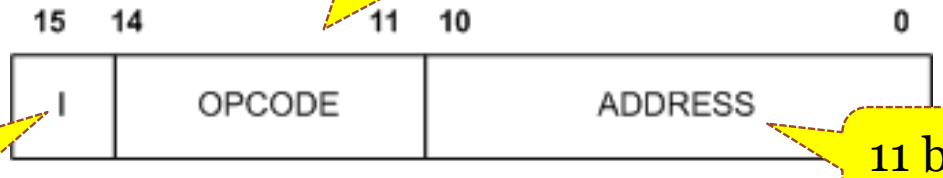
- To illustrate a vertical microinstruction format using an unreal machine



Unreal Computer Hardware Configuration

Machine instruction Format

4 bit opcode can have 16 possible instructions



Instruction Format

I ➔ Indirect Addressing

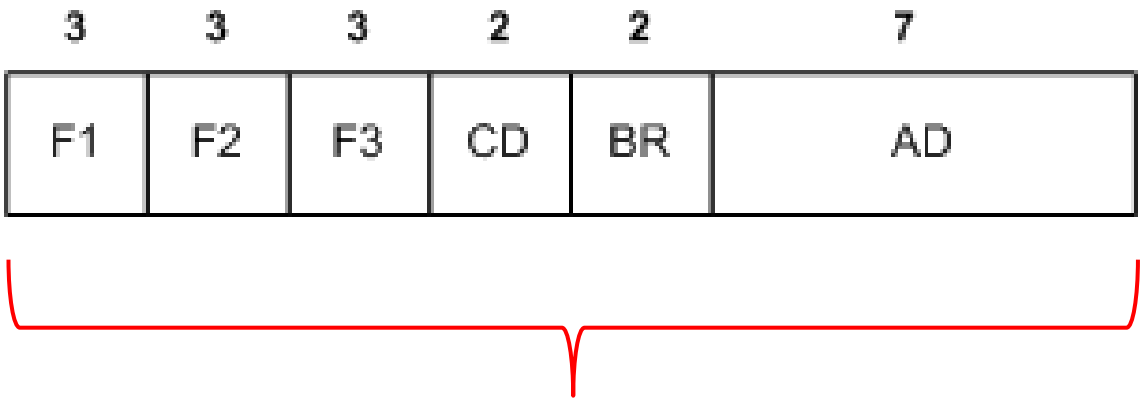
11 bit address field

Symbol	Opcode	Description
ADD	0000	AC <-- AC +M[EA]
BRANCH	0001	If (AC < 0) then (PC <-- EA)
STORE	0010	M[EA] <-- AC
EXCHANGE	0011	AC <-- M[EA], M[EA] <-- AC

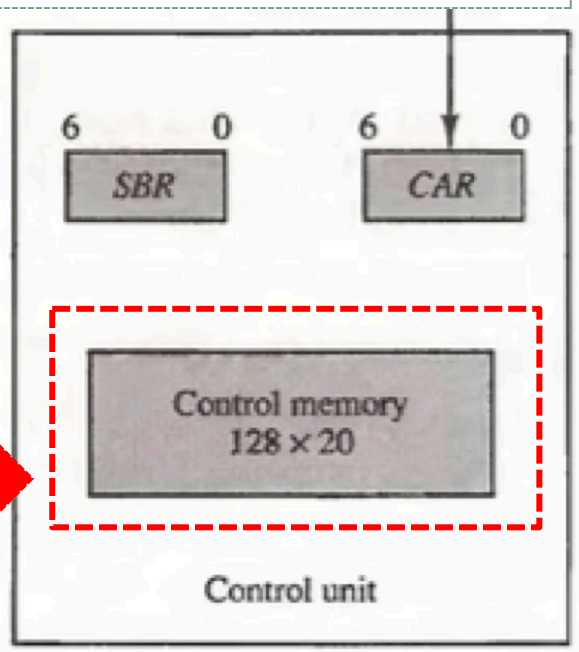
EA = effective address

Four computer instructions

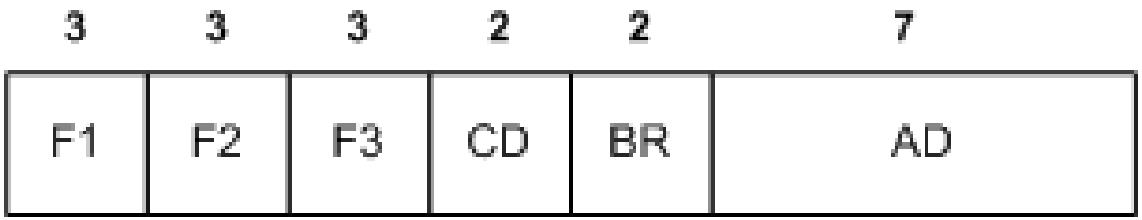
Vertical Microinstruction format



20-bit length



Vertical Microinstruction format

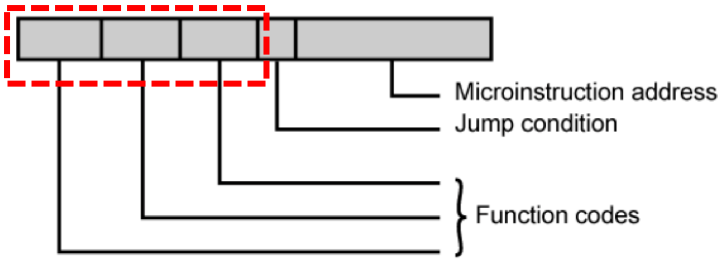


F1, F2, F3 : Microoperations field

CD: Condition for Branching

BR: Branching field

AD: Address field



(b) Vertical microinstruction

F1	Micro-operation	Symbol
000	None	NOP
001	AC ← AC + DR	ADD
010	AC ← 0	CLRAC
011	AC ← AC + 1	INCAC
100	AC ← DR	DRTAC
101	AR ← DR(0-10)	DRTAR
110	AR ← PC	PCTAR
111	M[AR] ← DR	WRITE

F2	Micro-operation	Symbol
000	None	NOP
001	AC ← AC - DR	SUB
010	AC ← AC ∨ DR	OR
011	AC ← AC ∧ DR	AND
100	DR ← M[AR]	READ
101	DR ← AC	ACTDR
110	DR ← DR + 1	INCDR
111	DR(0-10) ← PC	PCTDR

Microinstruction field description:

F1, F2, F3

F3	Micro-operation	Symbol
000	None	NOP
001	AC ← AC ⊕ DR	XOR
010	AC ← AC'	COM
011	AC ← shl AC	SHL
100	AC ← shr AC	SHR
101	PC ← PC + 1	INCPC
110	PC ← AR	ARTPC
111	Reserved	

* Mano, Morris M. (1993). *Computer System Architecture* (3rd Edition). p.224.

The condition for branching

Microinstruction field description:
CD, BR

CD	Condition	Symbol	Comments
00	Always = 1	U	Unconditional branch
01	DR(15)	I	Indirect address bit
10	AC(15)	S	Sign bit of AC
11	AC = 0	Z	Zero value in AC

The type of branch to be used

BR	Symbol	Function
00	JMP	CAR ← AD if condition = 1
		CAR ← CAR + 1 if condition = 0
01	CALL	CAR ← AD, SBR ← CAR + 1 if condition = 1
		CAR ← CAR + 1 if condition = 0
10	RET	CAR ← SBR (Return from subroutine)
11	MAP	CAR(2-5) ← DR(11-14), CAR(0,1,6) ← 0

* Mano, Morris M. (1993). *Computer System Architecture* (3rd Edition). p.224.

F2	Microoperation	Symbol
000	None	NOP
001	AC ← AC - DR	SUB
010	AC ← AC ∨ DR	OR
011	AC ← AC ∧ DR	AND
100	DR ← M[AR]	READ
101	DR ← AC	ACTDR
110	DR ← DR + 1	INCDR
111	DR(0-10) ← PC	PCTDR

F3	Microoperation	Symbol
000	None	NOP
001	AC ← AC ⊕ DR	XOR
010	AC ← AC'	COM
011	AC ← shl AC	SHL
100	AC ← shr AC	SHR
101	PC ← PC + 1	INCPC
110	PC ← AR	ARTPC
111	Reserved	

Example 23:

A microinstruction with 2 simultaneous micro-operations.

$DR \leftarrow M[AR]$ ⇢ $\{F2 = 100\}$

$PC \leftarrow PC + 1$ ⇢ $\{F3 = 101\}$

Since no F1 micro-operation, $\{F1 = 000\}$

Thus the vertical microinstruction fields:

F1	F2	F3	CD	BR	AD
000	100	101			

* Mano, Morris M. (1993). *Computer System Architecture* (3rd Edition). p.223.

No more than 3 micro-operations can be chosen for a single microinstruction.

F1:
F2: $DR \leftarrow M[AR]$
F3: $PC \leftarrow PC + 1$

F1	Microoperation	Symbol
000	None	NOP
001	AC ← AC + DR	ADD
010	AC ← 0	CLRAC
011	AC ← AC + 1	INCAC
100	AC ← DR	DRTAC
101	AR ← DR(0-10)	DRTAR
110	AR ← PC	PCTAR
111	M[AR] ← DR	WRITE

F2	Microoperation	Symbol
000	None	NOP
001	AC ← AC - DR	SUB
010	AC ← AC ∨ DR	OR
011	AC ← AC ∧ DR	AND
100	DR ← M[AR]	READ
101	DR ← AC	ACTDR
110	DR ← DR + 1	INCDR
111	DR(0-10) ← PC	PCTDR

F3	Microoperation	Symbol
000	None	NOP
001	AC ← AC ⊕ DR	XOR
010	AC ← AC'	COM
011	AC ← shl AC	SHL
100	AC ← shr AC	SHR
101	PC ← PC + 1	INCPC
110	PC ← AR	ARTPC
111	Reserved	

Symbolic Microprogram

Label	Microops	CD	BR	AD
	ORG 0			
ADD:	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ADD	U	JMP	FETCH
	ORG 4			
BRANCH:	NOP	S	JMP	OVER
	NOP	U	JMP	FETCH
OVER:	NOP	I	CALL	INDRCT
	ARTPC	U	JMP	FETCH
	ORG 8			
STORE:	NOP	I	CALL	INDRCT
	ACTDR	U	JMP	NEXT
	WRITE	U	JMP	FETCH
	ORG 12			
EXCHANGE:	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ACTDR, DRTAC	U	JMP	NEXT
	WRITE	U	JMP	FETCH
	ORG 64			
FETCH:	PCTAR	U	JMP	NEXT
	READ, INCPC	U	JMP	NEXT
	DRTAR	U	MAP	
INDRCT:	READ	U	JMP	NEXT
	DRTAR	U	RET	

* Mano, Morris M. (1993). *Computer System Architecture* (3rd Edition). p.228.

Binary Microprogram

5

Micro Routine	Address		Binary Microinstruction					
	Decimal	Binary	F1	F2	F3	CD	BR	AD
ADD	0	0000000	000	000	000	01	01	1000011
	1	0000001	000	100	000	00	00	0000010
	2	0000010	001	000	000	00	00	1000000
	3	0000011	000	000	000	00	00	1000000
BRANCH	4	0000100	000	000	000	10	00	0000110
	5	0000101	000	000	000	00	00	1000000
	6	0000110	000	000	000	01	01	1000011
	7	0000111	000	000	110	00	00	1000000
STORE	8	0001000	000	000	000	01	01	1000011
	9	0001001	000	101	000	00	00	0001010
	10	0001010	111	000	000	00	00	1000000
	11	0001011	000	000	000	00	00	1000000
EXCHANGE	12	0001100	000	000	000	01	01	1000011
	13	0001101	001	000	000	00	00	0001110
	14	0001110	100	101	000	00	00	0001111
	15	0001111	111	000	000	00	00	1000000
FETCH	64	1000000	110	000	000	00	00	1000001
	65	1000001	000	100	101	00	00	1000010
	66	1000010	101	000	000	00	11	0000000
INDRCT	67	1000011	000	100	000	00	00	1000100
	68	1000100	101	000	000	00	10	0000000

This microprogram can be implemented using ROM

* Mano, Morris M. (1993). *Computer System Architecture* (3rd Edition). p.230.

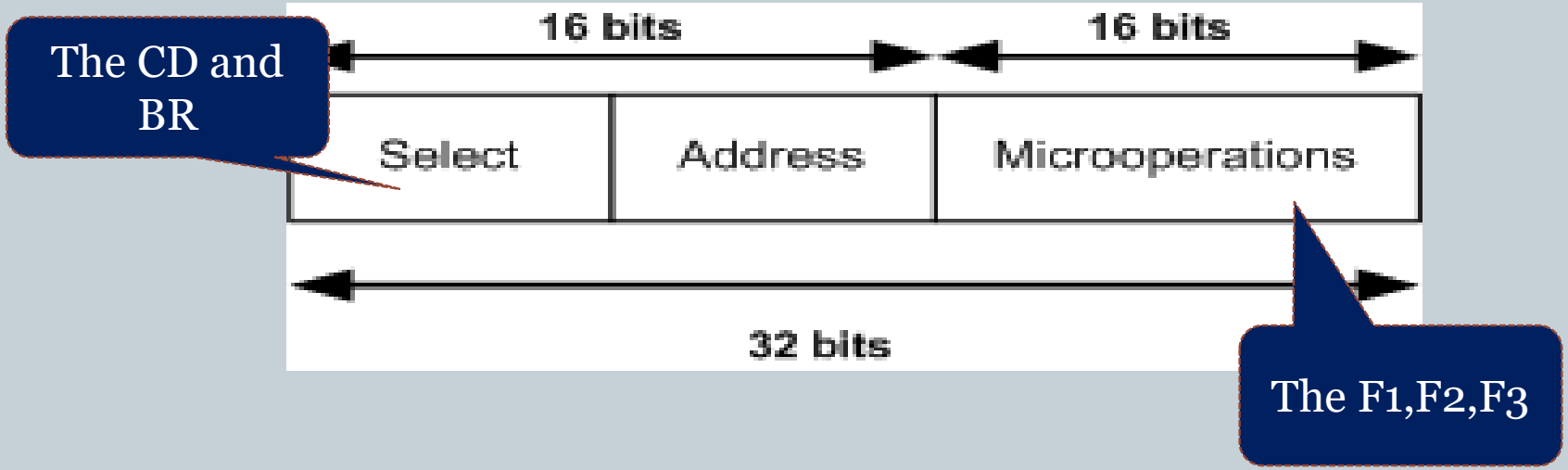
Calculation

Calculation example

- A system uses a control memory of 1024 words of 32 bits each. The microinstruction has 3 fields and a 16-bit micro operations.
  - A-How many bits are there in the branch address field and the selection field?
  - B- if there are 16 status bits in the system, how many bits of the branch logic are used to select a status bit?
  - C- how many bits are left to select an input for the multiplexers?

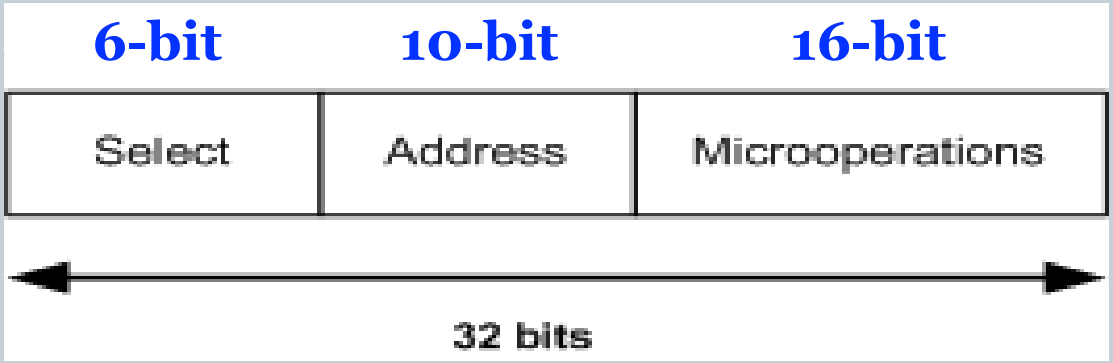
Calculation example - solution

- The keywords: A system uses a control memory of 1024 words of 32 bits each. The microinstruction has 3 fields and a 16-bit microoperations.



Calculation example - solution

- A-How many bits are there in the branch address field and the selection field?



- The Control memory has 1024 words (2^{10}). Thus, the required address field for each word is **10 bits**.
- The selection field will be: 16-10 = **6 bits**

Calculation example - solution

- B- if there are 16 status bits in the system, how many bits of the branch logic are used to select a status bit?
  - 16 status means **4 bits** are required for it (2^4)
- C- how many bits are left to select an input for the multiplexers?
  - Input for the multiplexer

    = selection field bits – status bits

    = 6 – 4

    = **2 bits**

Summary

- Explained the concept of *micro-operations* and define the principal instruction cycle phases in terms of micro-operations.
- Discussed how micro-operations are organized to control a processor.
- Understand micro-programmed control unit organization.

End of module 5